

MRMAC Reference Manual

Multi-Edit Revisited Macro Language

Dr. Michael 'iDoc' Raus

12 September 2026

This manual describes the executable MRMAC language in the current MR source tree. The original Multi-Edit reference guide supplies historical context, not the authority for MRMAC semantics. An outer-margin bar marks an MRMAC extension or a documented semantic change relative to MEMAC. A footnote identifies a close historical command that is not implemented. The bars are deliberately mechanical: they remain useful when a reader prints only selected pages.



Contents

I	Language Fundamentals	4
1	Purpose and Source Form	5
1.1	Primitive Map	5
1.2	Reference Entries	5
2	Values, Variables and Collections	8
2.1	Primitive Map	8
2.2	Reference Entries	11
3	Expressions and Conversion	48
3.1	Primitive Map	48
3.2	Reference Entries	49
4	Control Flow and Macro Composition	59
4.1	Primitive Map	59
4.2	Reference Entries	59
II	Editor Automation	63
5	Text and String Operations	64
5.1	Primitive Map	64
5.2	Reference Entries	64
6	Cursor, Tabs and Indentation	68
6.1	Primitive Map	68
6.2	Reference Entries	69
7	Blocks, Clipboard and Undo	81
7.1	Primitive Map	81
7.2	Reference Entries	82
8	Files, Search and Windows	89
8.1	Primitive Map	89
8.2	Reference Entries	90
III	Interaction and UI	105
9	Screen, Input and Key Dispatch	106
9.1	Primitive Map	106
9.2	Reference Entries	107
10	Typed Dialogs and Collections	119
10.1	Primitive Map	119
10.2	Reference Entries	119
11	Modeless MMP Interfaces	127
11.1	Primitive Map	127
11.2	Reference Entries	128

IV Runtime and Integration	146
12 Globals, Catalogs and Execution Sessions	147
12.1 Primitive Map	147
12.2 Reference Entries	147
13 Settings, Keymaps and Themes	153
13.1 Primitive Map	153
13.2 Reference Entries	153
14 External Environment and Editor Commands	160
14.1 Primitive Map	160
14.2 Reference Entries	161
V Debugging	168
15 Debugging MRMAC Macros	169
15.1 Starting a debug session	169
15.2 Source-view context menu	170
15.3 Breakpoints, running and stopping	170
15.4 Stepping	172
16 Inspecting State and Diagnosing Failures	174
16.1 Variables and collections	174
16.2 Watches and expression evaluation	175
16.3 Diagnostic model	177
17 Example Macros	178
17.1 Insert the current date	178
17.2 A compact modeless status panel	178
A MEMAC Compatibility Notes	179
B Command Index	180

Part I

Language Fundamentals

Chapter 1

Purpose and Source Form

MRMAC source is ordinary text compiled before it runs. A source file may name itself, define macro entries, and define closures for scheduled work. Begin here when writing a new macro: every source-form primitive is listed in the local map and then documented immediately below.

1.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
<code>\$MACRO</code>	keyword	Source structure.	p. 5
<code>\$MACRO_FILE</code>	keyword	Source structure.	p. 5
<code>\$CLOSURE</code>	keyword	Source structure.	p. 6
<code>DEF_TICK</code>	keyword	Source structure.	p. 6
<code>END_MACRO</code>	keyword	Macro terminator.	p. 6
<code>END_CLOSURE</code>	keyword	Closure terminator.	p. 6
<code>FROM</code>	header keyword	Macro context.	p. 7
<code>TO</code>	header keyword	Key binding.	p. 7
<code>TRANS</code>	header keyword	TRANS attribute.	p. 7
<code>DUMP</code>	header keyword	DUMP attribute.	p. 7
<code>PERM</code>	header keyword	PERM attribute.	p. 7

1.2 Reference Entries

`$MACRO`

```
$MACRO macroName FROM EDIT;  
    statement;  
END_MACRO;
```

Parameters: `macroName` names the entry; `FROM EDIT` selects the editor context.
Starts a named executable macro entry.

Example macro.

```
$MACRO REFERENCE_MACRO FROM EDIT;  
    MAKE_MESSAGE('reference');  
END_MACRO;
```

\$MACRO _FILE

```
$MACRO_FILE fileName;
```

Parameters: fileName is the single source-file identity.

Declares the identity of one macro source file.

Example macro.

```
$MACRO_FILE REFERENCE_FILE;
$MACRO REFERENCE_MACRO FROM EDIT;
END_MACRO;
```

\$CLOSURE

```
$CLOSURE closureName;
DEF_TICK(milliseconds);
    statement;
END_CLOSURE;
```

Parameters: closureName names the scheduled unit; milliseconds is a positive tick interval.

Starts a named closure with local execution state.

Example macro.

```
$CLOSURE REFERENCE_CLOSURE;
DEF_TICK(1000);
    MAKE_MESSAGE('tick');
END_CLOSURE;
```

DEF _TICK

```
DEF_TICK(milliseconds);
```

Parameters: milliseconds is a positive integer interval.

Declares the tick interval of the enclosing closure.

Example macro.

```
$CLOSURE REFERENCE_TICK;
DEF_TICK(1000);
END_CLOSURE;
```

END _MACRO

```
END_MACRO;
```

Parameters: None.

Terminates the current macro entry. It is required after the entry body.

Example macro.

```
$MACRO REFERENCE_END_MACRO FROM EDIT;
END_MACRO;
```

END _CLOSURE

```
END_CLOSURE;
```

Parameters: None.

Terminates the current closure entry after its tick declaration and body.

Example macro.

```
$CLOSURE REFERENCE_END_CLOSURE;  
DEF_TICK(1000);  
END_CLOSURE;
```

FROM

```
$MACRO macroName FROM mode;
```

Parameters: macroName names the macro; mode selects its context.

Selects the declared execution context. EDIT is the normal editor context.

Example macro.

```
$MACRO REFERENCE_FROM FROM EDIT;  
END_MACRO;
```

TO

```
$MACRO macroName TO <keySpec> FROM EDIT;
```

Parameters: macroName names the macro; keySpec is a supported key specification.

Associates an entry with a key specification when its source is loaded.

Example macro.

```
$MACRO REFERENCE_TO TO <F1> FROM EDIT;  
END_MACRO;
```

TRANS

```
$MACRO macroName TRANS FROM EDIT;
```

Parameters: macroName names the macro.

Sets the compiler-supported TRANS attribute on the macro header.

Example macro.

```
$MACRO REFERENCE_TRANS TRANS FROM EDIT;  
END_MACRO;
```

DUMP

```
$MACRO macroName DUMP FROM EDIT;
```

Parameters: macroName names the macro.

Sets the compiler-supported DUMP attribute on the macro header.

Example macro.

```
$MACRO REFERENCE_DUMP DUMP FROM EDIT;  
END_MACRO;
```

PERM

```
$MACRO macroName PERM FROM EDIT;
```

Parameters: macroName names the macro.

Sets the compiler-supported PERM attribute on the macro header.

Example macro.

```
$MACRO REFERENCE_PERM PERM FROM EDIT;  
END_MACRO;
```

Chapter 2

Values, Variables and Collections

A macro works with typed values. Declare local values before using them; declarations belong to the current execution, not to a source file. Integers, real numbers, strings and characters are scalar values. Hashes associate keys with values, and typed arrays contain one declared element type. Built-in constants and variables provide runtime values.

2.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
DEF_INT	keyword	Typed declaration.	p. 11
DEF_REAL	keyword	Typed declaration.	p. 11
DEF_STR	keyword	Typed declaration.	p. 12
DEF_CHAR	keyword	Typed declaration.	p. 12
DEF_HASH	keyword	Typed declaration.	p. 12
ASSIGNMENT	source form	Assignment.	p. 12
INDEXING	source form	Collection access.	p. 13
TRUE	constant	Symbolic constant.	p. 13
FALSE	constant	Symbolic constant.	p. 13
BLACK	constant	Symbolic constant.	p. 14
BLUE	constant	Symbolic constant.	p. 14
GREEN	constant	Symbolic constant.	p. 14
CYAN	constant	Symbolic constant.	p. 14
RED	constant	Symbolic constant.	p. 14
MAGENTA	constant	Symbolic constant.	p. 14
BROWN	constant	Symbolic constant.	p. 14
LIGHTGRAY	constant	Symbolic constant.	p. 14
DARKGRAY	constant	Symbolic constant.	p. 14
LIGHTBLUE	constant	Symbolic constant.	p. 14
LIGHTGREEN	constant	Symbolic constant.	p. 14
LIGHTCYAN	constant	Symbolic constant.	p. 14
LIGHTRED	constant	Symbolic constant.	p. 14
LIGHTMAGENTA	constant	Symbolic constant.	p. 14
YELLOW	constant	Symbolic constant.	p. 14
WHITE	constant	Symbolic constant.	p. 14
EDIT	constant	Symbolic constant.	p. 14
DOS_SHELL	constant	Symbolic constant.	p. 14
BACK_SPACE	constant	Symbolic constant.	p. 14
BLOCK_BEGIN	constant	Symbolic constant.	p. 15

Primitive	Role	Purpose	Reference
BLOCK_END	constant	Symbolic constant.	p. 15
BLOCK_OFF	constant	Symbolic constant.	p. 15
COL_BLOCK_BEGIN	constant	Symbolic constant.	p. 15
COPY_BLOCK	constant	Symbolic constant.	p. 16
CR	constant	Symbolic constant.	p. 16
DELETE_BLOCK	constant	Symbolic constant.	p. 16
DEL_CHAR	constant	Symbolic constant.	p. 16
DEL_LINE	constant	Symbolic constant.	p. 17
DOWN	constant	Symbolic constant.	p. 17
EOF	constant	Symbolic constant.	p. 17
EOL	constant	Symbolic constant.	p. 17
FIRST_WORD	constant	Symbolic constant.	p. 18
GOTO_MARK	constant	Symbolic constant.	p. 18
HOME	constant	Symbolic constant.	p. 18
INDENT	constant	Symbolic constant.	p. 18
KEY_RECORD	constant	Symbolic constant.	p. 19
LAST_PAGE_BREAK	constant	Symbolic constant.	p. 19
LEFT	constant	Symbolic constant.	p. 19
MARK_POS	constant	Symbolic constant.	p. 19
MOVE_BLOCK	constant	Symbolic constant.	p. 20
NEXT_PAGE_BREAK	constant	Symbolic constant.	p. 20
PAGE_DOWN	constant	Symbolic constant.	p. 20
PAGE_UP	constant	Symbolic constant.	p. 20
RIGHT	constant	Symbolic constant.	p. 21
SAVE_FILE	constant	Symbolic constant.	p. 21
STR_BLOCK_BEGIN	constant	Symbolic constant.	p. 21
TAB_LEFT	constant	Symbolic constant.	p. 21
TAB_RIGHT	constant	Symbolic constant.	p. 22
TOF	constant	Symbolic constant.	p. 22
UNDENT	constant	Symbolic constant.	p. 22
UNDO	constant	Symbolic constant.	p. 22
UP	constant	Symbolic constant.	p. 23
WORD_LEFT	constant	Symbolic constant.	p. 23
WORD_RIGHT	constant	Symbolic constant.	p. 23
ALL	constant	Symbolic constant.	p. 23
RETURN_INT	variable	Runtime variable.	p. 24
ERROR_LEVEL	variable	Runtime variable.	p. 24
FIRST_RUN	variable	Runtime variable.	p. 24
IGNORE_CASE	variable	Runtime variable.	p. 24
REG_EXP_STAT	variable	Runtime variable.	p. 24
FIRST_SAVE	variable	Runtime variable.	p. 25
BUFFER_ID	variable	Runtime variable.	p. 25
TMP_FILE	variable	Runtime variable.	p. 25
FILE_CHANGED	variable	Runtime variable.	p. 25
LAST_FILE_ATTR	variable	Runtime variable.	p. 26
LAST_FILE_SIZE	variable	Runtime variable.	p. 26
LAST_FILE_TIME	variable	Runtime variable.	p. 26
CUR_FILE_ATTR	variable	Runtime variable.	p. 26
CUR_FILE_SIZE	variable	Runtime variable.	p. 26
READ_ONLY	variable	Runtime variable.	p. 27
DOC_MODE	variable	Runtime variable.	p. 27

Primitive	Role	Purpose	Reference
PRINT_MARGIN	variable	Runtime variable.	p. 27
SHADOW_CHAR	variable	Runtime variable.	p. 27
REFRESH	variable	Runtime variable.	p. 28
MESSAGES	variable	Hero message-line switch.	p. 28
MOUSE	variable	Runtime variable.	p. 28
LOGO_SCREEN	variable	Runtime variable.	p. 28
EXPLOSIONS	variable	Runtime variable.	p. 29
TRUNCATE_SPACES	variable	Runtime variable.	p. 29
BACKUPS	variable	Runtime variable.	p. 29
AUTOSAVE	variable	Runtime variable.	p. 29
UNDO_STAT	variable	Runtime variable.	p. 30
FORMAT_STAT	variable	Runtime variable.	p. 30
WRAP_STAT	variable	Runtime variable.	p. 30
MEM_ALLOC	variable	Runtime variable.	p. 30
LEFT_MARGIN	variable	Runtime variable.	p. 30
RIGHT_MARGIN	variable	Runtime variable.	p. 31
FORMAT_RULER	variable	Runtime variable.	p. 31
INDENT_STYLE	variable	Runtime variable.	p. 31
INS_CURSOR	variable	Runtime variable.	p. 31
OVR_CURSOR	variable	Runtime variable.	p. 32
CTRL_HELP	variable	Runtime variable.	p. 32
MOUSE_H_SENSE	variable	Runtime variable.	p. 32
MOUSE_V_SENSE	variable	Runtime variable.	p. 32
WINDOW_ATTR	variable	Runtime variable.	p. 33
TEXT_COLOR	variable	Runtime variable.	p. 33
CHANGE_COLOR	variable	Runtime variable.	p. 33
BACK_COLOR	variable	Runtime variable.	p. 33
MENU_COLOR	variable	Runtime variable.	p. 33
STAT_COLOR	variable	Runtime variable.	p. 34
ERROR_COLOR	variable	Runtime variable.	p. 34
SHADOW_COLOR	variable	Runtime variable.	p. 34
STATUS_ROW	variable	Runtime variable.	p. 34
MESSAGE_ROW	variable	Runtime variable.	p. 35
MAX_WINDOW_ROW	variable	Runtime variable.	p. 35
MIN_WINDOW_ROW	variable	Runtime variable.	p. 35
NAME_LINE	variable	Runtime variable.	p. 35
PARAM_COUNT	variable	Runtime variable.	p. 35
CPU	variable	Runtime variable.	p. 36
C_COL	variable	Runtime variable.	p. 36
C_LINE	variable	Runtime variable.	p. 36
C_ROW	variable	Runtime variable.	p. 36
C_PAGE	variable	Runtime variable.	p. 37
PG_LINE	variable	Runtime variable.	p. 37
AT_EOF	variable	Runtime variable.	p. 37
AT_EOL	variable	Runtime variable.	p. 37
BLOCK_STAT	variable	Runtime variable.	p. 37
BLOCK_LINE1	variable	Runtime variable.	p. 38
BLOCK_LINE2	variable	Runtime variable.	p. 38
BLOCK_COL1	variable	Runtime variable.	p. 38
BLOCK_COL2	variable	Runtime variable.	p. 38
MARKING	variable	Runtime variable.	p. 39

Primitive	Role	Purpose	Reference
TAB_EXPAND	variable	Runtime variable.	p. 39
INSERT_MODE	variable	Runtime variable.	p. 39
INDENT_LEVEL	variable	Runtime variable.	p. 39
CUR_WINDOW	variable	Runtime variable.	p. 40
LINK_STAT	variable	Runtime variable.	p. 40
WIN_X1	variable	Runtime variable.	p. 40
WIN_Y1	variable	Runtime variable.	p. 40
WIN_X2	variable	Runtime variable.	p. 40
WIN_Y2	variable	Runtime variable.	p. 41
WINDOW_COUNT	variable	Runtime variable.	p. 41
VIRTUAL_DESKTOPS	variable	Runtime variable.	p. 41
CYCLIC_VIRTUAL_DESKTOPS	variable	Runtime variable.	p. 41
KEY1	variable	Runtime variable.	p. 42
KEY2	variable	Runtime variable.	p. 42
RETURN_STR	variable	Runtime variable.	p. 42
MPARM_STR	variable	Runtime variable.	p. 42
DATE	variable	Runtime variable.	p. 42
TIME	variable	Runtime variable.	p. 43
FIRST_MACRO	variable	Runtime variable.	p. 43
NEXT_MACRO	variable	Runtime variable.	p. 43
LAST_FILE_NAME	variable	Runtime variable.	p. 43
TMP_FILE_NAME	variable	Runtime variable.	p. 44
FILE_NAME	variable	Runtime variable.	p. 44
COMSPEC	variable	Runtime variable.	p. 44
TEMP_PATH	variable	Runtime variable.	p. 44
FOUND_STR	variable	Runtime variable.	p. 44
SEARCH_FILE	variable	Runtime variable.	p. 45
MR_PATH	variable	Runtime variable.	p. 45
GET_LINE	variable	Runtime variable.	p. 45
FORMAT_LINE	variable	Runtime variable.	p. 45
DEFAULT_FORMAT	variable	Runtime variable.	p. 46
PAGE_STR	variable	Runtime variable.	p. 46
WORD_DELIMITS	variable	Runtime variable.	p. 46
FOUND_X	variable	Runtime variable.	p. 46
FOUND_Y	variable	Runtime variable.	p. 47
CUR_CHAR	variable	Runtime variable.	p. 47

2.2 Reference Entries

DEF_INT

```
DEF_INT(name);
DEF_INT(name[]);
```

Parameters: name is a local integer scalar or typed-array name.
Declares an integer variable or array.

Example macro.

```
$MACRO REFERENCE_DEF_INT FROM EDIT;
    DEF_INT(Value);
    Value := 1;
END_MACRO;
```

DEF_REAL

```
DEF_REAL(name);  
DEF_REAL(name[]);
```

Parameters: name is a local real scalar or typed-array name.
Declares a real variable or array.

Example macro.

```
$MACRO REFERENCE_DEF_REAL FROM EDIT;  
    DEF_REAL(Value);  
    Value := 1.0;  
END_MACRO;
```

DEF_STR

```
DEF_STR(name);  
DEF_STR(name[]);
```

Parameters: name is a local string scalar or typed-array name.
Declares a string variable or array.

Example macro.

```
$MACRO REFERENCE_DEF_STR FROM EDIT;  
    DEF_STR(Value);  
    Value := 'reference';  
END_MACRO;
```

DEF_CHAR

```
DEF_CHAR(name);  
DEF_CHAR(name[]);
```

Parameters: name is a local character scalar or typed-array name.
Declares a character variable or array.

Example macro.

```
$MACRO REFERENCE_DEF_CHAR FROM EDIT;  
    DEF_CHAR(Value);  
    Value := 'r';  
END_MACRO;
```

DEF_HASH

```
DEF_HASH(name);
```

Parameters: name is a local hash name.
Declares a local hash.

Example macro.

```
$MACRO REFERENCE_DEF_HASH FROM EDIT;  
    DEF_HASH(State);  
    State['name'] := 'reference';  
END_MACRO;
```

Assignment

`target := expression;`

Parameters: `target` is a declared variable, array element or writable hash element; `expression` supplies a compatible value.

Assigns the evaluated right-hand expression to the left-hand `target`.

Example macro.

```
$MACRO REFERENCE_ASSIGNMENT FROM EDIT;
  DEF_INT(Value);
  Value := 1;
END_MACRO;
```

Array and Hash Indexing

`collection[indexOrKey]`

Parameters: `collection` is an array or hash; `indexOrKey` is a positive integer array index or string-like hash key.

Addresses an element of a typed array or hash. Array reads outside the current bounds are runtime errors.

Example macro.

```
$MACRO REFERENCE_INDEXING FROM EDIT;
  DEF_INT(Values[]);
  Values[1] := 42;
END_MACRO;
```

TRUE — constant

`TRUE`

Parameters: None. Result: integer constant.

Evaluates to integer 1, the true result used by conditional expressions.

Example macro.

```
$MACRO REFERENCE_CONSTANT_TRUE FROM EDIT;
  DEF_INT(Value);
  Value := TRUE;
END_MACRO;
```

FALSE — constant

`FALSE`

Parameters: None. Result: integer constant.

Evaluates to integer 0, the false result used by conditional expressions.

Example macro.

```
$MACRO REFERENCE_CONSTANT_FALSE FROM EDIT;
  DEF_INT(Value);
  Value := FALSE;
END_MACRO;
```

Colour Constants

BLACK	0	LIGHTGRAY	7
BLUE	1	DARKGRAY	8
GREEN	2	LIGHTBLUE	9
CYAN	3	LIGHTGREEN	10
RED	4	LIGHTCYAN	11
MAGENTA	5	LIGHTRED	12
BROWN	6	LIGHTMAGENTA	13
YELLOW	14	WHITE	15

Parameters: None. Result: integer constant.

These constants denote the fixed values of the 16-colour palette. Use them only where a primitive documents a palette-colour value.

Example macro.

```
$MACRO REFERENCE_COLOUR_CONSTANTS FROM EDIT;
    DEF_INT(Colour);
    Colour := LIGHTGREEN;
END_MACRO;
```

EDIT — constant

EDIT

Parameters: None. Result: integer constant.

Provides compiler-defined macro mode value 0. DOS_SHELL is historical source syntax; MRMAC provides no DOS shell runtime.

Example macro.

```
$MACRO REFERENCE_CONSTANT_EDIT FROM EDIT;
    DEF_INT(Value);
    Value := EDIT;
END_MACRO;
```

DOS_SHELL — constant

DOS_SHELL

Parameters: None. Result: integer constant.

Provides compiler-defined macro mode value 1. DOS_SHELL is historical source syntax; MRMAC provides no DOS shell runtime.

Example macro.

```
$MACRO REFERENCE_CONSTANT_DOS_SHELL FROM EDIT;
    DEF_INT(Value);
    Value := DOS_SHELL;
END_MACRO;
```

BACK_SPACE — constant

BACK_SPACE

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7001. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_BACK_SPACE FROM EDIT;
    DEF_INT(Value);
    Value := BACK_SPACE;
END_MACRO;
```

BLOCK__BEGIN — constant

BLOCK_BEGIN

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7002. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_BLOCK_BEGIN FROM EDIT;
    DEF_INT(Value);
    Value := BLOCK_BEGIN;
END_MACRO;
```

BLOCK__END — constant

BLOCK_END

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7003. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_BLOCK_END FROM EDIT;
    DEF_INT(Value);
    Value := BLOCK_END;
END_MACRO;
```

BLOCK__OFF — constant

BLOCK_OFF

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7004. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_BLOCK_OFF FROM EDIT;
    DEF_INT(Value);
    Value := BLOCK_OFF;
END_MACRO;
```

COL_BLOCK_BEGIN — constant

COL_BLOCK_BEGIN

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7005. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_COL_BLOCK_BEGIN FROM EDIT;
    DEF_INT(Value);
    Value := COL_BLOCK_BEGIN;
END_MACRO;
```

COPY_BLOCK — constant

COPY_BLOCK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7006. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_COPY_BLOCK FROM EDIT;
    DEF_INT(Value);
    Value := COPY_BLOCK;
END_MACRO;
```

CR — constant

CR

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7007. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_CR FROM EDIT;
    DEF_INT(Value);
    Value := CR;
END_MACRO;
```

DELETE_BLOCK — constant

DELETE_BLOCK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7008. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_DELETE_BLOCK FROM EDIT;
    DEF_INT(Value);
    Value := DELETE_BLOCK;
END_MACRO;
```

DEL_CHAR — constant

DEL_CHAR

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7009. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_DEL_CHAR FROM EDIT;  
    DEF_INT(Value);  
    Value := DEL_CHAR;  
END_MACRO;
```

DEL_LINE — constant

DEL_LINE

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700A. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_DEL_LINE FROM EDIT;  
    DEF_INT(Value);  
    Value := DEL_LINE;  
END_MACRO;
```

DOWN — constant

DOWN

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700B. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_DOWN FROM EDIT;  
    DEF_INT(Value);  
    Value := DOWN;  
END_MACRO;
```

EOF — constant

EOF

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700C. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_EOF FROM EDIT;  
    DEF_INT(Value);  
    Value := EOF;  
END_MACRO;
```

EOL — constant

EOL

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700D. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_EOL FROM EDIT;
    DEF_INT(Value);
    Value := EOL;
END_MACRO;
```

FIRST_WORD — constant

FIRST_WORD

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700E. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_FIRST_WORD FROM EDIT;
    DEF_INT(Value);
    Value := FIRST_WORD;
END_MACRO;
```

GOTO_MARK — constant

GOTO_MARK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x700F. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_GOTO_MARK FROM EDIT;
    DEF_INT(Value);
    Value := GOTO_MARK;
END_MACRO;
```

HOME — constant

HOME

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7010. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_HOME FROM EDIT;
    DEF_INT(Value);
    Value := HOME;
END_MACRO;
```

INDENT — constant

INDENT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7011. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_INDENT FROM EDIT;
    DEF_INT(Value);
    Value := INDENT;
END_MACRO;
```

KEY_RECORD — constant

KEY_RECORD

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7012. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_KEY_RECORD FROM EDIT;
    DEF_INT(Value);
    Value := KEY_RECORD;
END_MACRO;
```

LAST_PAGE_BREAK — constant

LAST_PAGE_BREAK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7013. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_LAST_PAGE_BREAK FROM EDIT;
    DEF_INT(Value);
    Value := LAST_PAGE_BREAK;
END_MACRO;
```

LEFT — constant

LEFT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7014. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_LEFT FROM EDIT;
    DEF_INT(Value);
    Value := LEFT;
END_MACRO;
```

MARK_POS — constant

MARK_POS

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7015. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_MARK_POS FROM EDIT;
    DEF_INT(Value);
    Value := MARK_POS;
END_MACRO;
```

MOVE_BLOCK — constant

MOVE_BLOCK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7016. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_MOVE_BLOCK FROM EDIT;
    DEF_INT(Value);
    Value := MOVE_BLOCK;
END_MACRO;
```

NEXT_PAGE_BREAK — constant

NEXT_PAGE_BREAK

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7017. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_NEXT_PAGE_BREAK FROM EDIT;
    DEF_INT(Value);
    Value := NEXT_PAGE_BREAK;
END_MACRO;
```

PAGE_DOWN — constant

PAGE_DOWN

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7018. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_PAGE_DOWN FROM EDIT;
    DEF_INT(Value);
    Value := PAGE_DOWN;
END_MACRO;
```

PAGE_UP — constant

PAGE_UP

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7019. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_PAGE_UP FROM EDIT;
    DEF_INT(Value);
    Value := PAGE_UP;
END_MACRO;
```

RIGHT — constant

RIGHT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701A. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_RIGHT FROM EDIT;
    DEF_INT(Value);
    Value := RIGHT;
END_MACRO;
```

SAVE_FILE — constant

SAVE_FILE

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701B. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_SAVE_FILE FROM EDIT;
    DEF_INT(Value);
    Value := SAVE_FILE;
END_MACRO;
```

STR_BLOCK_BEGIN — constant

STR_BLOCK_BEGIN

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701C. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_STR_BLOCK_BEGIN FROM EDIT;
    DEF_INT(Value);
    Value := STR_BLOCK_BEGIN;
END_MACRO;
```

TAB_LEFT — constant

TAB_LEFT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701D. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_TAB_LEFT FROM EDIT;
    DEF_INT(Value);
    Value := TAB_LEFT;
END_MACRO;
```

TAB_RIGHT — constant

TAB_RIGHT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701E. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_TAB_RIGHT FROM EDIT;
    DEF_INT(Value);
    Value := TAB_RIGHT;
END_MACRO;
```

TOF — constant

TOF

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x701F. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_TOF FROM EDIT;
    DEF_INT(Value);
    Value := TOF;
END_MACRO;
```

UNDENT — constant

UNDENT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7020. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_UNDENT FROM EDIT;
    DEF_INT(Value);
    Value := UNDENT;
END_MACRO;
```

UNDO — constant

UNDO

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7021. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_UNDO FROM EDIT;
    DEF_INT(Value);
    Value := UNDO;
END_MACRO;
```

UP — constant

UP

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7022. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_UP FROM EDIT;
    DEF_INT(Value);
    Value := UP;
END_MACRO;
```

WORD_LEFT — constant

WORD_LEFT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7023. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_WORD_LEFT FROM EDIT;
    DEF_INT(Value);
    Value := WORD_LEFT;
END_MACRO;
```

WORD_RIGHT — constant

WORD_RIGHT

Parameters: None. Result: integer constant.

Provides legacy integer command identifier 0x7024. Use it only where a procedure explicitly accepts a command identifier.

Example macro.

```
$MACRO REFERENCE_CONSTANT_WORD_RIGHT FROM EDIT;
    DEF_INT(Value);
    Value := WORD_RIGHT;
END_MACRO;
```

ALL — constant

ALL

Parameters: None. Result: integer constant.

Provides compiler-defined macro mode value 255. DOS_SHELL is historical source syntax; MRMAC provides no DOS shell runtime.

Example macro.

```
$MACRO REFERENCE_CONSTANT_ALL FROM EDIT;
    DEF_INT(Value);
    Value := ALL;
END_MACRO;
```

RETURN_INT — variable

RETURN_INT

Parameters: None. Result: integer value.

Provides the integer return slot of the current macro. Assign a value before RET to return an integer result.

Example macro.

```
$MACRO REFERENCE_VARIABLE_RETURN_INT FROM EDIT;
    DEF_INT(Value);
    Value := RETURN_INT;
END_MACRO;
```

ERROR_LEVEL — variable

ERROR_LEVEL

Parameters: None. Result: integer value.

Provides the current macro error level. The VM permits assignment to set the level for the active execution.

Example macro.

```
$MACRO REFERENCE_VARIABLE_ERROR_LEVEL FROM EDIT;
    DEF_INT(Value);
    Value := ERROR_LEVEL;
END_MACRO;
```

FIRST_RUN — variable

FIRST_RUN

Parameters: None. Result: integer value.

Returns 1 only on the first run of the current macro stack entry; otherwise 0.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FIRST_RUN FROM EDIT;
    DEF_INT(Value);
    Value := FIRST_RUN;
END_MACRO;
```

IGNORE_CASE — variable

IGNORE_CASE

Parameters: None. Result: integer value.

Provides the active search case-sensitivity flag as 1 or 0. The VM permits assignment.

Example macro.

```
$MACRO REFERENCE_VARIABLE_IGNORE_CASE FROM EDIT;
    DEF_INT(Value);
    Value := IGNORE_CASE;
END_MACRO;
```

REG_EXP_STAT — variable

REG_EXP_STAT

Parameters: None. Result: integer value.

Provides the active regular-expression status flag as 1 or 0. The VM permits assignment.

Example macro.

```
$MACRO REFERENCE_VARIABLE_REG_EXP_STAT FROM EDIT;  
    DEF_INT(Value);  
    Value := REG_EXP_STAT;  
END_MACRO;
```

FIRST_SAVE — variable

FIRST_SAVE

Parameters: None. Result: integer value.

Provides current or last-file metadata named first save.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FIRST_SAVE FROM EDIT;  
    DEF_INT(Value);  
    Value := FIRST_SAVE;  
END_MACRO;
```

BUFFER_ID — variable

BUFFER_ID

Parameters: None. Result: integer value.

Provides current or last-file metadata named buffer id.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BUFFER_ID FROM EDIT;  
    DEF_INT(Value);  
    Value := BUFFER_ID;  
END_MACRO;
```

TMP_FILE — variable

TMP_FILE

Parameters: None. Result: integer value.

Provides current or last-file metadata named tmp file.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TMP_FILE FROM EDIT;  
    DEF_INT(Value);  
    Value := TMP_FILE;  
END_MACRO;
```

FILE_CHANGED — variable

FILE_CHANGED

Parameters: None. Result: integer value.

Provides whether the active file has unsaved changes as 1 or 0. The VM permits assignment when an active file context exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FILE_CHANGED FROM EDIT;  
    DEF_INT(Value);  
    Value := FILE_CHANGED;  
END_MACRO;
```

LAST_FILE_ATTR — variable

LAST_FILE_ATTR

Parameters: None. Result: integer value.

Provides current or last-file metadata named last file attr.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LAST_FILE_ATTR FROM EDIT;  
    DEF_INT(Value);  
    Value := LAST_FILE_ATTR;  
END_MACRO;
```

LAST_FILE_SIZE — variable

LAST_FILE_SIZE

Parameters: None. Result: integer value.

Provides current or last-file metadata named last file size.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LAST_FILE_SIZE FROM EDIT;  
    DEF_INT(Value);  
    Value := LAST_FILE_SIZE;  
END_MACRO;
```

LAST_FILE_TIME — variable

LAST_FILE_TIME

Parameters: None. Result: integer value.

Provides current or last-file metadata named last file time.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LAST_FILE_TIME FROM EDIT;  
    DEF_INT(Value);  
    Value := LAST_FILE_TIME;  
END_MACRO;
```

CUR_FILE_ATTR — variable

CUR_FILE_ATTR

Parameters: None. Result: integer value.

Provides current or last-file metadata named cur file attr.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CUR_FILE_ATTR FROM EDIT;  
    DEF_INT(Value);  
    Value := CUR_FILE_ATTR;  
END_MACRO;
```

CUR_FILE_SIZE — variable

CUR_FILE_SIZE

Parameters: None. Result: integer value.

Provides current or last-file metadata named cur file size.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CUR_FILE_SIZE FROM EDIT;  
  DEF_INT(Value);  
  Value := CUR_FILE_SIZE;  
END_MACRO;
```

READ_ONLY — variable

READ_ONLY

Parameters: None. Result: integer value.

Provides current or last-file metadata named read only.

Example macro.

```
$MACRO REFERENCE_VARIABLE_READ_ONLY FROM EDIT;  
  DEF_INT(Value);  
  Value := READ_ONLY;  
END_MACRO;
```

DOC_MODE — variable

DOC_MODE

Parameters: None. Result: integer value.

Provides the current runtime value named doc mode.

Example macro.

```
$MACRO REFERENCE_VARIABLE_DOC_MODE FROM EDIT;  
  DEF_INT(Value);  
  Value := DOC_MODE;  
END_MACRO;
```

PRINT_MARGIN — variable

PRINT_MARGIN

Parameters: None. Result: read-only integer value.

Returns the configured right boundary for PDF text output in columns. The value applies when the current file or a marked block is exported. A value of zero disables the column limit, leaving the printable page width as the only horizontal boundary. Assigning to this variable is a runtime error.

Example macro.

```
$MACRO REFERENCE_VARIABLE_PRINT_MARGIN FROM EDIT;  
  DEF_INT(Value);  
  Value := PRINT_MARGIN;  
END_MACRO;
```

SHADOW_CHAR — variable

SHADOW_CHAR

Parameters: None. Result: integer value.

Provides the current runtime value named shadow char.

Example macro.

```
$MACRO REFERENCE_VARIABLE_SHADOW_CHAR FROM EDIT;
    DEF_INT(Value);
    Value := SHADOW_CHAR;
END_MACRO;
```

REFRESH — variable

REFRESH

Parameters: None. Result: integer value.

Provides the current runtime value named refresh.

Example macro.

```
$MACRO REFERENCE_VARIABLE_REFRESH FROM EDIT;
    DEF_INT(Value);
    Value := REFRESH;
END_MACRO;
```

MESSAGES — variable

MESSAGES

Parameters: None. Result: integer value.

Reads the Hero message-line switch: one means enabled and zero means disabled. Assigning zero disables that destination; assigning a nonzero integer enables it. This does not mute ordinary warnings or validation feedback, change Hero log output, or alter the file-size threshold. The central settings model records a dirty change only when the value actually changes.

Do not confuse this runtime variable with the retired serialized setting of the same name. The old settings key is accepted without changing runtime state; current settings use the Hero keys described under MRSETUP.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MESSAGES FROM EDIT;
    DEF_INT(Value);
    Value := MESSAGES;
    MESSAGES := 0;
    MAKE_MESSAGE('Ordinary feedback remains available.');
```

MESSAGES := Value;

```
END_MACRO;
```

MOUSE — variable

MOUSE

Parameters: None. Result: integer value.

Provides the current runtime value named mouse.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MOUSE FROM EDIT;
    DEF_INT(Value);
    Value := MOUSE;
END_MACRO;
```

LOGO_SCREEN — variable

LOGO_SCREEN

Parameters: None. Result: integer value.

Provides the current runtime value named logo screen.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LOGO_SCREEN FROM EDIT;
    DEF_INT(Value);
    Value := LOGO_SCREEN;
END_MACRO;
```

EXPLOSIONS — variable

EXPLOSIONS

Parameters: None. Result: integer value.

Provides the current runtime value named explosions.

Example macro.

```
$MACRO REFERENCE_VARIABLE_EXPLOSIONS FROM EDIT;
    DEF_INT(Value);
    Value := EXPLOSIONS;
END_MACRO;
```

TRUNCATE_SPACES — variable

TRUNCATE_SPACES

Parameters: None. Result: integer value.

Provides the active editor-format value named truncate spaces; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TRUNCATE_SPACES FROM EDIT;
    DEF_INT(Value);
    Value := TRUNCATE_SPACES;
END_MACRO;
```

BACKUPS — variable

BACKUPS

Parameters: None. Result: integer value.

Provides the active editor-format value named backups; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BACKUPS FROM EDIT;
    DEF_INT(Value);
    Value := BACKUPS;
END_MACRO;
```

AUTOSAVE — variable

AUTOSAVE

Parameters: None. Result: integer value.

Provides the active editor-format value named autosave; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_AUTOSAVE FROM EDIT;
    DEF_INT(Value);
    Value := AUTOSAVE;
END_MACRO;
```

UNDO_STAT — variable

UNDO_STAT

Parameters: None. Result: integer value.

Provides the current runtime value named undo stat.

Example macro.

```
$MACRO REFERENCE_VARIABLE_UNDO_STAT FROM EDIT;
    DEF_INT(Value);
    Value := UNDO_STAT;
END_MACRO;
```

FORMAT_STAT — variable

FORMAT_STAT

Parameters: None. Result: integer value.

Provides the current runtime value named format stat.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FORMAT_STAT FROM EDIT;
    DEF_INT(Value);
    Value := FORMAT_STAT;
END_MACRO;
```

WRAP_STAT — variable

WRAP_STAT

Parameters: None. Result: integer value.

Provides the active editor-format value named wrap stat; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WRAP_STAT FROM EDIT;
    DEF_INT(Value);
    Value := WRAP_STAT;
END_MACRO;
```

MEM_ALLOC — variable

MEM_ALLOC

Parameters: None. Result: integer value.

Provides the current runtime value named mem alloc.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MEM_ALLOC FROM EDIT;
    DEF_INT(Value);
    Value := MEM_ALLOC;
END_MACRO;
```

LEFT_MARGIN — variable

LEFT_MARGIN

Parameters: None. Result: integer value.

Provides the active editor-format value named left margin; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LEFT_MARGIN FROM EDIT;  
    DEF_INT(Value);  
    Value := LEFT_MARGIN;  
END_MACRO;
```

RIGHT_MARGIN — variable

RIGHT_MARGIN

Parameters: None. Result: integer value.

Provides the active editor-format value named right margin; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_RIGHT_MARGIN FROM EDIT;  
    DEF_INT(Value);  
    Value := RIGHT_MARGIN;  
END_MACRO;
```

FORMAT_RULER — variable

FORMAT_RULER

Parameters: None. Result: integer value.

Provides the active editor-format value named format ruler; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FORMAT_RULER FROM EDIT;  
    DEF_INT(Value);  
    Value := FORMAT_RULER;  
END_MACRO;
```

INDENT_STYLE — variable

INDENT_STYLE

Parameters: None. Result: integer value.

Provides the active editor-format value named indent style; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_INDENT_STYLE FROM EDIT;  
    DEF_INT(Value);  
    Value := INDENT_STYLE;  
END_MACRO;
```

INS_CURSOR — variable

INS_CURSOR

Parameters: None. Result: integer value.

Provides the current runtime value named ins cursor.

Example macro.

```
$MACRO REFERENCE_VARIABLE_INS_CURSOR FROM EDIT;  
  DEF_INT(Value);  
  Value := INS_CURSOR;  
END_MACRO;
```

OVR_CURSOR — variable

OVR_CURSOR

Parameters: None. Result: integer value.

Provides the current runtime value named ovr cursor.

Example macro.

```
$MACRO REFERENCE_VARIABLE_OVR_CURSOR FROM EDIT;  
  DEF_INT(Value);  
  Value := OVR_CURSOR;  
END_MACRO;
```

CTRL_HELP — variable

CTRL_HELP

Parameters: None. Result: integer value.

Provides the current runtime value named ctrl help.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CTRL_HELP FROM EDIT;  
  DEF_INT(Value);  
  Value := CTRL_HELP;  
END_MACRO;
```

MOUSE_H_SENSE — variable

MOUSE_H_SENSE

Parameters: None. Result: integer value.

Provides the current runtime value named mouse h sense.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MOUSE_H_SENSE FROM EDIT;  
  DEF_INT(Value);  
  Value := MOUSE_H_SENSE;  
END_MACRO;
```

MOUSE_V_SENSE — variable

MOUSE_V_SENSE

Parameters: None. Result: integer value.

Provides the current runtime value named mouse v sense.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MOUSE_V_SENSE FROM EDIT;  
    DEF_INT(Value);  
    Value := MOUSE_V_SENSE;  
END_MACRO;
```

WINDOW_ATTR — variable

WINDOW_ATTR

Parameters: None. Result: integer value.

Provides the current configured window attr as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WINDOW_ATTR FROM EDIT;  
    DEF_INT(Value);  
    Value := WINDOW_ATTR;  
END_MACRO;
```

TEXT_COLOR — variable

TEXT_COLOR

Parameters: None. Result: integer value.

Provides the current configured text color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TEXT_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := TEXT_COLOR;  
END_MACRO;
```

CHANGE_COLOR — variable

CHANGE_COLOR

Parameters: None. Result: integer value.

Provides the current configured change color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CHANGE_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := CHANGE_COLOR;  
END_MACRO;
```

BACK_COLOR — variable

BACK_COLOR

Parameters: None. Result: integer value.

Provides the current configured back color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BACK_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := BACK_COLOR;  
END_MACRO;
```

MENU_COLOR — variable

MENU_COLOR

Parameters: None. Result: integer value.

Provides the current configured menu color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MENU_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := MENU_COLOR;  
END_MACRO;
```

STAT_COLOR — variable

STAT_COLOR

Parameters: None. Result: integer value.

Provides the current configured stat color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_STAT_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := STAT_COLOR;  
END_MACRO;
```

ERROR_COLOR — variable

ERROR_COLOR

Parameters: None. Result: integer value.

Provides the current configured error color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_ERROR_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := ERROR_COLOR;  
END_MACRO;
```

SHADOW_COLOR — variable

SHADOW_COLOR

Parameters: None. Result: integer value.

Provides the current configured shadow color as an integer colour or display attribute.

Example macro.

```
$MACRO REFERENCE_VARIABLE_SHADOW_COLOR FROM EDIT;  
    DEF_INT(Value);  
    Value := SHADOW_COLOR;  
END_MACRO;
```

STATUS_ROW — variable

STATUS_ROW

Parameters: None. Result: integer value.

Provides the current runtime value named status row.

Example macro.

```
$MACRO REFERENCE_VARIABLE_STATUS_ROW FROM EDIT;  
    DEF_INT(Value);  
    Value := STATUS_ROW;  
END_MACRO;
```

MESSAGE_ROW — variable

MESSAGE_ROW

Parameters: None. Result: integer value.

Provides the current runtime value named message row.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MESSAGE_ROW FROM EDIT;  
    DEF_INT(Value);  
    Value := MESSAGE_ROW;  
END_MACRO;
```

MAX_WINDOW_ROW — variable

MAX_WINDOW_ROW

Parameters: None. Result: integer value.

Provides the current runtime value named max window row.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MAX_WINDOW_ROW FROM EDIT;  
    DEF_INT(Value);  
    Value := MAX_WINDOW_ROW;  
END_MACRO;
```

MIN_WINDOW_ROW — variable

MIN_WINDOW_ROW

Parameters: None. Result: integer value.

Provides the current runtime value named min window row.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MIN_WINDOW_ROW FROM EDIT;  
    DEF_INT(Value);  
    Value := MIN_WINDOW_ROW;  
END_MACRO;
```

NAME_LINE — variable

NAME_LINE

Parameters: None. Result: integer value.

Provides the current runtime value named name line.

Example macro.

```
$MACRO REFERENCE_VARIABLE_NAME_LINE FROM EDIT;  
    DEF_INT(Value);  
    Value := NAME_LINE;  
END_MACRO;
```

PARAM_COUNT — variable

PARAM_COUNT

Parameters: None. Result: integer value.

Provides the number of process parameters visible to the runtime.

Example macro.

```
$MACRO REFERENCE_VARIABLE_PARAM_COUNT FROM EDIT;  
    DEF_INT(Value);  
    Value := PARAM_COUNT;  
END_MACRO;
```

CPU — variable

CPU

Parameters: None. Result: integer value.

Provides the current runtime value named cpu.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CPU FROM EDIT;  
    DEF_INT(Value);  
    Value := CPU;  
END_MACRO;
```

C_COL — variable

C_COL

Parameters: None. Result: integer value.

Provides the current editor cursor column.

Example macro.

```
$MACRO REFERENCE_VARIABLE_C_COL FROM EDIT;  
    DEF_INT(Value);  
    Value := C_COL;  
END_MACRO;
```

C_LINE — variable

C_LINE

Parameters: None. Result: integer value.

Provides the current editor cursor line number.

Example macro.

```
$MACRO REFERENCE_VARIABLE_C_LINE FROM EDIT;  
    DEF_INT(Value);  
    Value := C_LINE;  
END_MACRO;
```

C_ROW — variable

C_ROW

Parameters: None. Result: integer value.

Provides the current editor row within the visible view.

Example macro.

```
$MACRO REFERENCE_VARIABLE_C_ROW FROM EDIT;  
    DEF_INT(Value);  
    Value := C_ROW;  
END_MACRO;
```

C_PAGE — variable

C_PAGE

Parameters: None. Result: integer value.

Provides the current editor page number.

Example macro.

```
$MACRO REFERENCE_VARIABLE_C_PAGE FROM EDIT;  
    DEF_INT(Value);  
    Value := C_PAGE;  
END_MACRO;
```

PG_LINE — variable

PG_LINE

Parameters: None. Result: integer value.

Provides the current line number within the current page.

Example macro.

```
$MACRO REFERENCE_VARIABLE_PG_LINE FROM EDIT;  
    DEF_INT(Value);  
    Value := PG_LINE;  
END_MACRO;
```

AT_EOF — variable

AT_EOF

Parameters: None. Result: integer value.

Returns 1 when the cursor is at end of file; otherwise 0.

Example macro.

```
$MACRO REFERENCE_VARIABLE_AT_EOF FROM EDIT;  
    DEF_INT(Value);  
    Value := AT_EOF;  
END_MACRO;
```

AT_EOL — variable

AT_EOL

Parameters: None. Result: integer value.

Returns 1 when the cursor is at end of line; otherwise 0.

Example macro.

```
$MACRO REFERENCE_VARIABLE_AT_EOL FROM EDIT;  
    DEF_INT(Value);  
    Value := AT_EOL;  
END_MACRO;
```

BLOCK_STAT — variable

BLOCK_STAT

Parameters: None. Result: integer value.

Provides current selection state named block stat.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BLOCK_STAT FROM EDIT;  
    DEF_INT(Value);  
    Value := BLOCK_STAT;  
END_MACRO;
```

BLOCK_LINE1 — variable

BLOCK_LINE1

Parameters: None. Result: integer value.

Provides current selection state named block line1.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BLOCK_LINE1 FROM EDIT;  
    DEF_INT(Value);  
    Value := BLOCK_LINE1;  
END_MACRO;
```

BLOCK_LINE2 — variable

BLOCK_LINE2

Parameters: None. Result: integer value.

Provides current selection state named block line2.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BLOCK_LINE2 FROM EDIT;  
    DEF_INT(Value);  
    Value := BLOCK_LINE2;  
END_MACRO;
```

BLOCK_COL1 — variable

BLOCK_COL1

Parameters: None. Result: integer value.

Provides current selection state named block col1.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BLOCK_COL1 FROM EDIT;  
    DEF_INT(Value);  
    Value := BLOCK_COL1;  
END_MACRO;
```

BLOCK_COL2 — variable

BLOCK_COL2

Parameters: None. Result: integer value.

Provides current selection state named block col2.

Example macro.

```
$MACRO REFERENCE_VARIABLE_BLOCK_COL2 FROM EDIT;
    DEF_INT(Value);
    Value := BLOCK_COL2;
END_MACRO;
```

MARKING — variable

MARKING

Parameters: None. Result: integer value.

Provides current selection state named marking.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MARKING FROM EDIT;
    DEF_INT(Value);
    Value := MARKING;
END_MACRO;
```

TAB_EXPAND — variable

TAB_EXPAND

Parameters: None. Result: integer value.

Provides the active editor-format value named tab expand; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TAB_EXPAND FROM EDIT;
    DEF_INT(Value);
    Value := TAB_EXPAND;
END_MACRO;
```

INSERT_MODE — variable

INSERT_MODE

Parameters: None. Result: integer value.

Provides the active editor-format value named insert mode; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_INSERT_MODE FROM EDIT;
    DEF_INT(Value);
    Value := INSERT_MODE;
END_MACRO;
```

INDENT_LEVEL — variable

INDENT_LEVEL

Parameters: None. Result: integer value.

Provides the active editor-format value named indent level; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_INDENT_LEVEL FROM EDIT;  
    DEF_INT(Value);  
    Value := INDENT_LEVEL;  
END_MACRO;
```

CUR_WINDOW — variable

CUR_WINDOW

Parameters: None. Result: integer value.

Provides current editor-window state named cur window.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CUR_WINDOW FROM EDIT;  
    DEF_INT(Value);  
    Value := CUR_WINDOW;  
END_MACRO;
```

LINK_STAT — variable

LINK_STAT

Parameters: None. Result: integer value.

Provides current editor-window state named link stat.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LINK_STAT FROM EDIT;  
    DEF_INT(Value);  
    Value := LINK_STAT;  
END_MACRO;
```

WIN_X1 — variable

WIN_X1

Parameters: None. Result: integer value.

Provides current editor-window state named win x1.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WIN_X1 FROM EDIT;  
    DEF_INT(Value);  
    Value := WIN_X1;  
END_MACRO;
```

WIN_Y1 — variable

WIN_Y1

Parameters: None. Result: integer value.

Provides current editor-window state named win y1.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WIN_Y1 FROM EDIT;  
    DEF_INT(Value);  
    Value := WIN_Y1;  
END_MACRO;
```

WIN_X2 — variable

WIN_X2

Parameters: None. Result: integer value.

Provides current editor-window state named win x2.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WIN_X2 FROM EDIT;
    DEF_INT(Value);
    Value := WIN_X2;
END_MACRO;
```

WIN_Y2 — variable

WIN_Y2

Parameters: None. Result: integer value.

Provides current editor-window state named win y2.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WIN_Y2 FROM EDIT;
    DEF_INT(Value);
    Value := WIN_Y2;
END_MACRO;
```

WINDOW_COUNT — variable

WINDOW_COUNT

Parameters: None. Result: integer value.

Provides current editor-window state named window count.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WINDOW_COUNT FROM EDIT;
    DEF_INT(Value);
    Value := WINDOW_COUNT;
END_MACRO;
```

VIRTUAL_DESKTOPS — variable

VIRTUAL_DESKTOPS

Parameters: None. Result: integer value.

Provides the current runtime value named virtual desktops.

Example macro.

```
$MACRO REFERENCE_VARIABLE_VIRTUAL_DESKTOPS FROM EDIT;
    DEF_INT(Value);
    Value := VIRTUAL_DESKTOPS;
END_MACRO;
```

CYCLIC_VIRTUAL_DESKTOPS — variable

CYCLIC_VIRTUAL_DESKTOPS

Parameters: None. Result: integer value.

Provides the current runtime value named cyclic virtual desktops.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CYCLIC_VIRTUAL_DESKTOPS FROM EDIT;
  DEF_INT(Value);
  Value := CYCLIC_VIRTUAL_DESKTOPS;
END_MACRO;
```

KEY1 — variable

KEY1

Parameters: None. Result: integer value.

Provides the current runtime value named key1.

Example macro.

```
$MACRO REFERENCE_VARIABLE_KEY1 FROM EDIT;
  DEF_INT(Value);
  Value := KEY1;
END_MACRO;
```

KEY2 — variable

KEY2

Parameters: None. Result: integer value.

Provides the current runtime value named key2.

Example macro.

```
$MACRO REFERENCE_VARIABLE_KEY2 FROM EDIT;
  DEF_INT(Value);
  Value := KEY2;
END_MACRO;
```

RETURN_STR — variable

RETURN_STR

Parameters: None. Result: string value.

Provides the string return slot of the current macro. Assign a value before RET to return a string result.

Example macro.

```
$MACRO REFERENCE_VARIABLE_RETURN_STR FROM EDIT;
  DEF_STR(Value);
  Value := RETURN_STR;
END_MACRO;
```

MPARM_STR — variable

MPARM_STR

Parameters: None. Result: string value.

Provides the current macro parameter string. The VM permits assignment for the active execution.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MPARM_STR FROM EDIT;
  DEF_STR(Value);
  Value := MPARM_STR;
END_MACRO;
```

DATE — variable

DATE

Parameters: None. Result: string value.

Provides the current local date formatted by the runtime.

Example macro.

```
$MACRO REFERENCE_VARIABLE_DATE FROM EDIT;  
    DEF_STR(Value);  
    Value := DATE;  
END_MACRO;
```

TIME — variable

TIME

Parameters: None. Result: string value.

Provides the current local time formatted by the runtime.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TIME FROM EDIT;  
    DEF_STR(Value);  
    Value := TIME;  
END_MACRO;
```

FIRST_MACRO — variable

FIRST_MACRO

Parameters: None. Result: string value.

Starts loaded-macro enumeration and returns the first visible macro name, or an empty string.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FIRST_MACRO FROM EDIT;  
    DEF_STR(Value);  
    Value := FIRST_MACRO;  
END_MACRO;
```

NEXT_MACRO — variable

NEXT_MACRO

Parameters: None. Result: string value.

Continues loaded-macro enumeration and returns the next visible macro name, or an empty string.

Example macro.

```
$MACRO REFERENCE_VARIABLE_NEXT_MACRO FROM EDIT;  
    DEF_STR(Value);  
    Value := NEXT_MACRO;  
END_MACRO;
```

LAST_FILE_NAME — variable

LAST_FILE_NAME

Parameters: None. Result: string value.

Provides current or last-file metadata named last file name.

Example macro.

```
$MACRO REFERENCE_VARIABLE_LAST_FILE_NAME FROM EDIT;
    DEF_STR(Value);
    Value := LAST_FILE_NAME;
END_MACRO;
```

TMP_FILE_NAME — variable

TMP_FILE_NAME

Parameters: None. Result: string value.

Provides current or last-file metadata named tmp file name.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TMP_FILE_NAME FROM EDIT;
    DEF_STR(Value);
    Value := TMP_FILE_NAME;
END_MACRO;
```

FILE_NAME — variable

FILE_NAME

Parameters: None. Result: string value.

Provides the current file name. The VM permits assignment when an active file context exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FILE_NAME FROM EDIT;
    DEF_STR(Value);
    Value := FILE_NAME;
END_MACRO;
```

COMSPEC — variable

COMSPEC

Parameters: None. Result: string value.

Provides the runtime environment value named comspec.

Example macro.

```
$MACRO REFERENCE_VARIABLE_COMSPEC FROM EDIT;
    DEF_STR(Value);
    Value := COMSPEC;
END_MACRO;
```

TEMP_PATH — variable

TEMP_PATH

Parameters: None. Result: string value.

Provides the runtime environment value named temp path.

Example macro.

```
$MACRO REFERENCE_VARIABLE_TEMP_PATH FROM EDIT;
    DEF_STR(Value);
    Value := TEMP_PATH;
END_MACRO;
```

FOUND_STR — variable

FOUND_STR

Parameters: None. Result: string value.

Provides the text of the current search match, or an empty string when no match exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FOUND_STR FROM EDIT;  
    DEF_STR(Value);  
    Value := FOUND_STR;  
END_MACRO;
```

SEARCH_FILE — variable

SEARCH_FILE

Parameters: None. Result: string value.

Provides the file name of the current search match, or an empty string when no match exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_SEARCH_FILE FROM EDIT;  
    DEF_STR(Value);  
    Value := SEARCH_FILE;  
END_MACRO;
```

MR_PATH — variable

MR_PATH

Parameters: None. Result: string value.

Provides the runtime environment value named mr path.

Example macro.

```
$MACRO REFERENCE_VARIABLE_MR_PATH FROM EDIT;  
    DEF_STR(Value);  
    Value := MR_PATH;  
END_MACRO;
```

GET_LINE — variable

GET_LINE

Parameters: None. Result: string value.

Provides the complete current editor line as a string.

Example macro.

```
$MACRO REFERENCE_VARIABLE_GET_LINE FROM EDIT;  
    DEF_STR(Value);  
    Value := GET_LINE;  
END_MACRO;
```

FORMAT_LINE — variable

FORMAT_LINE

Parameters: None. Result: string value.

Provides the active editor-format value named format line; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FORMAT_LINE FROM EDIT;
    DEF_STR(Value);
    Value := FORMAT_LINE;
END_MACRO;
```

DEFAULT_FORMAT — variable

DEFAULT_FORMAT

Parameters: None. Result: string value.

Provides the active editor-format value named default format; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_DEFAULT_FORMAT FROM EDIT;
    DEF_STR(Value);
    Value := DEFAULT_FORMAT;
END_MACRO;
```

PAGE_STR — variable

PAGE_STR

Parameters: None. Result: string value.

Provides the active editor-format value named page str; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_PAGE_STR FROM EDIT;
    DEF_STR(Value);
    Value := PAGE_STR;
END_MACRO;
```

WORD_DELIMITS — variable

WORD_DELIMITS

Parameters: None. Result: string value.

Provides the active editor-format value named word delimits; writable settings use the configured runtime boundary.

Example macro.

```
$MACRO REFERENCE_VARIABLE_WORD_DELIMITS FROM EDIT;
    DEF_STR(Value);
    Value := WORD_DELIMITS;
END_MACRO;
```

FOUND_X — variable

FOUND_X

Parameters: None. Result: integer value.

Provides the column of the current search match, or 0 when no match exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FOUND_X FROM EDIT;  
    DEF_INT(Value);  
    Value := FOUND_X;  
END_MACRO;
```

FOUND_Y — variable

FOUND_Y

Parameters: None. Result: integer value.

Provides the line of the current search match, or 0 when no match exists.

Example macro.

```
$MACRO REFERENCE_VARIABLE_FOUND_Y FROM EDIT;  
    DEF_INT(Value);  
    Value := FOUND_Y;  
END_MACRO;
```

CUR_CHAR — variable

CUR_CHAR

Parameters: None. Result: character value.

Provides the character at the current editor cursor position.

Example macro.

```
$MACRO REFERENCE_VARIABLE_CUR_CHAR FROM EDIT;  
    DEF_CHAR(Value);  
    Value := CUR_CHAR;  
END_MACRO;
```

Chapter 3

Expressions and Conversion

An expression computes a value from literals, variables, collection access, operators and intrinsics. Expressions are used in assignments, conditions and procedure arguments. The following entries document the intrinsic operations that create, inspect, convert and search values.

3.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
RVAL	intrinsic	Expression operation.	p. 49
VAL	intrinsic	Expression operation.	p. 49
ASCII	intrinsic	Expression operation.	p. 49
BAR_MENU	intrinsic	Expression operation.	p. 50
CAPS	intrinsic	Expression operation.	p. 50
CHAR	intrinsic	Expression operation.	p. 50
CHECK_KEY	intrinsic	Expression operation.	p. 51
COPY	intrinsic	Expression operation.	p. 51
EXISTS	intrinsic	Expression operation.	p. 51
HAS_VALUE	intrinsic	Expression operation.	p. 51
INT_R	intrinsic	Expression operation.	p. 52
KEYS	intrinsic	Expression operation.	p. 52
LEN	intrinsic	Expression operation.	p. 52
LENGTH	intrinsic	Expression operation.	p. 53
PARSE_INT	intrinsic	Expression operation.	p. 53
PARSE_STR	intrinsic	Expression operation.	p. 53
POS	intrinsic	Expression operation.	p. 53
REAL_I	intrinsic	Expression operation.	p. 54
REMOVE_SPACE	intrinsic	Expression operation.	p. 54
RSTR	intrinsic	Expression operation.	p. 54
SCREEN_LENGTH	intrinsic	Expression operation.	p. 55
SCREEN_WIDTH	intrinsic	Expression operation.	p. 55
STR	intrinsic	Expression operation.	p. 55
STRING_IN	intrinsic	Expression operation.	p. 55
STR_DEL	intrinsic	Expression operation.	p. 56
STR_INS	intrinsic	Expression operation.	p. 56
UTF8	intrinsic	Expression operation.	p. 57
VALUES	intrinsic	Expression operation.	p. 57
V_MENU	intrinsic	Expression operation.	p. 57
WHEREX	intrinsic	Expression operation.	p. 58

Primitive	Role	Purpose	Reference
WHEREY	intrinsic	Expression operation.	p. 58
XPOS	intrinsic	Expression operation.	p. 58

3.2 Reference Entries

RVAL

`RVAL(target, text)`

Parameters:

target Writable typed target variable.

text String-like text.

Parses text into the named real target and returns a status value.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_RVAL FROM EDIT;
    DEF_REAL(Target);
    DEF_INT(Result);
    Result := RVAL(Target, '42.0');
END_MACRO;
```

VAL

`VAL(target, text)`

Parameters:

target Writable typed target variable.

text String-like text.

Parses text into the named integer target and returns a status value.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_VAL FROM EDIT;
    DEF_INT(Target);
    DEF_INT(Result);
    Result := VAL(Target, '42');
END_MACRO;
```

ASCII

`ASCII(text)`

Parameters:

text String-like text.

Returns the numeric value of the first character.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_ASCII FROM EDIT;  
    DEF_INT(Result);  
    Result := ASCII('reference');  
END_MACRO;
```

BAR_MENU

BAR_MENU(column, row, width, items, title)

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

items String-like encoded menu item list.

title User-visible title.

Runs a horizontal menu and returns the selected item index.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_BAR_MENU FROM EDIT;  
    DEF_INT(Result);  
    Result := BAR_MENU(0, 0, 0, 'reference', 'reference');  
END_MACRO;
```

CAPS

CAPS(text)

Parameters:

text String-like text.

Returns an upper-case copy of the text.
Returns a string.

Example macro.

```
$MACRO REFERENCE_CAPS FROM EDIT;  
    DEF_STR(Result);  
    Result := CAPS('reference');  
END_MACRO;
```

CHAR

CHAR(integerValue)

Parameters:

integerValue Integer source value for the conversion.

Converts an integer to a character value.
Returns a character.

Example macro.

```
$MACRO REFERENCE_CHAR FROM EDIT;  
    DEF_STR(Result);  
    Result := CHAR(0);  
END_MACRO;
```

CHECK_KEY

CHECK_KEY

Parameters: None.

Reports whether input is available.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_CHECK_KEY FROM EDIT;  
    DEF_INT(Result);  
    Result := CHECK_KEY;  
END_MACRO;
```

COPY

COPY(text, start, count)

Parameters:

text String-like text.

start Zero-based string start position.

count Number of characters to process.

Returns a substring using the specified start and count.

Returns a string.

Example macro.

```
$MACRO REFERENCE_COPY FROM EDIT;  
    DEF_STR(Result);  
    Result := COPY('reference', 0, 0);  
END_MACRO;
```

EXISTS

EXISTS(hash, key)

Parameters:

hash Hash value.

key String-like hash key.

Tests whether a hash contains a key.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_EXISTS FROM EDIT;  
    DEF_HASH(State);  
    DEF_INT(Result);  
    Result := EXISTS(State, 'reference');  
END_MACRO;
```

HAS_VALUE

HAS_VALUE(hash, key)

Parameters:

hash Hash value.

key String-like hash key.

Tests whether a hash key has a non-default value.

Returns an integer.

Example macro.

```

$MACRO REFERENCE_HAS_VALUE FROM EDIT;
    DEF_HASH(State);
    DEF_INT(Result);
    Result := HAS_VALUE(State, 'reference');
END_MACRO;

```

INT_R

INT_R(realValue)

Parameters:

realValue Real source value for the conversion or formatting operation.

Converts a real value to an integer.

Returns an integer.

Example macro.

```

$MACRO REFERENCE_INT_R FROM EDIT;
    DEF_INT(Result);
    Result := INT_R(0.0);
END_MACRO;

```

KEYS

KEYS(hash)

Parameters:

hash Hash value.

Returns the hash keys as a string array.

Returns a string array.

Example macro.

```

$MACRO REFERENCE_KEYS FROM EDIT;
    DEF_HASH(State);
    DEF_INT(Result);
    Result := LEN(KEYS(State));
END_MACRO;

```

LEN

LEN(value)

Parameters:

value String-like value or typed array whose length is requested.

Returns the number of characters or collection elements.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_LEN FROM EDIT;  
    DEF_STR(Text);  
    DEF_INT(Result);  
    Text := 'reference';  
    Result := LEN(Text);  
END_MACRO;
```

LENGTH

LENGTH(text)

Parameters:

text String-like text.

Returns the number of characters in the string-like value.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_LENGTH FROM EDIT;  
    DEF_INT(Result);  
    Result := LENGTH('reference');  
END_MACRO;
```

PARSE_INT

PARSE_INT(text, delimiter)

Parameters:

text String-like text.

delimiter String-like delimiter separating parsed fields.

Returns the selected integer component from delimiter parsing.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_PARSE_INT FROM EDIT;  
    DEF_INT(Result);  
    Result := PARSE_INT('reference', 'reference');  
END_MACRO;
```

PARSE_STR

PARSE_STR(text, delimiter)

Parameters:

text String-like text.

delimiter String-like delimiter separating parsed fields.

Returns the selected string component from delimiter parsing.
Returns a string.

Example macro.

```
$MACRO REFERENCE_PARSE_STR FROM EDIT;  
    DEF_STR(Result);  
    Result := PARSE_STR('reference', 'reference');  
END_MACRO;
```

POS

POS(*needle*, *text*)

Parameters:

needle String-like text sought in the supplied text.

text String-like text.

Returns the first position of the needle in the text, or zero when absent.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_POS FROM EDIT;  
  DEF_INT(Result);  
  Result := POS('reference', 'reference');  
END_MACRO;
```

REAL_I

REAL_I(*integerValue*)

Parameters:

integerValue Integer source value for the conversion.

Converts an integer to a real value.

Returns a real value.

Example macro.

```
$MACRO REFERENCE_REAL_I FROM EDIT;  
  DEF_REAL(Result);  
  Result := REAL_I(0);  
END_MACRO;
```

REMOVE_SPACE

REMOVE_SPACE(*text*)

Parameters:

text String-like text.

Normalises spacing in a string.

Returns a string.

Example macro.

```
$MACRO REFERENCE_REMOVE_SPACE FROM EDIT;  
  DEF_STR(Result);  
  Result := REMOVE_SPACE('reference');  
END_MACRO;
```

RSTR

RSTR(*realValue*, *significantDigits*, *decimalPlaces*)

Parameters:

realValue Real source value for the conversion or formatting operation.

significantDigits Integer number of significant digits to emit.

decimalPlaces Integer number of decimal places to emit.

Formats a real value with the requested precision.

Returns a string.

Example macro.

```
$MACRO REFERENCE_RSTR FROM EDIT;  
    DEF_STR(Result);  
    Result := RSTR(0.0, 0, 0);  
END_MACRO;
```

SCREEN_LENGTH

SCREEN_LENGTH

Parameters: None.

Returns the current screen row count.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_SCREEN_LENGTH FROM EDIT;  
    DEF_INT(Result);  
    Result := SCREEN_LENGTH;  
END_MACRO;
```

SCREEN_WIDTH

SCREEN_WIDTH

Parameters: None.

Returns the current screen column count.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_SCREEN_WIDTH FROM EDIT;  
    DEF_INT(Result);  
    Result := SCREEN_WIDTH;  
END_MACRO;
```

STR

STR(integerValue)

Parameters:

integerValue Integer source value for the conversion.

Converts an integer to its decimal string representation.

Returns a string.

Example macro.

```
$MACRO REFERENCE_STR FROM EDIT;  
    DEF_STR(Result);  
    Result := STR(0);  
END_MACRO;
```

STRING_IN

STRING_IN(column, row, width, initialText, prompt)

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

initialText Initial text value.

prompt String or character positional argument.

Runs string input and returns the entered text.

Returns a string.

Example macro.

```
$MACRO REFERENCE_STRING_IN FROM EDIT;  
  DEF_STR(Result);  
  Result := STRING_IN(0, 0, 0, 'reference', 'reference');  
END_MACRO;
```

STR_DEL

STR_DEL(text, start, count)

Parameters:

text String-like text.

start Zero-based string start position.

count Number of characters to process.

Returns text with count characters removed from start.

Returns a string.

Example macro.

```
$MACRO REFERENCE_STR_DEL FROM EDIT;  
  DEF_STR(Result);  
  Result := STR_DEL('reference', 0, 0);  
END_MACRO;
```

STR_INS

STR_INS(text, insertText, start)

Parameters:

text String-like text.

insertText String-like text inserted into the source text.

start Zero-based string start position.

Returns text with insertText inserted at start.

Returns a string.

Example macro.

```
$MACRO REFERENCE_STR_INS FROM EDIT;
    DEF_STR(Result);
    Result := STR_INS('reference', 'reference', 0);
END_MACRO;
```

UTF8

UTF8(codePoint)

Parameters:

codePoint Integer Unicode code point.

Encodes one Unicode code point as a UTF-8 string.
Returns a string.

Example macro.

```
$MACRO REFERENCE_UTF8 FROM EDIT;
    DEF_STR(Result);
    Result := UTF8(0);
END_MACRO;
```

VALUES

VALUES(hash)

Parameters:

hash Hash value.

Returns the hash values as a string array.
Returns a string array.

Example macro.

```
$MACRO REFERENCE_VALUES FROM EDIT;
    DEF_HASH(State);
    DEF_INT(Result);
    Result := LEN(VALUES(State));
END_MACRO;
```

V_MENU

V_MENU(column, row, height, items, title)

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

height Control or region height.

items String-like encoded menu item list.

title User-visible title.

Runs a vertical menu and returns the selected item index.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_V_MENU FROM EDIT;  
    DEF_INT(Result);  
    Result := V_MENU(0, 0, 0, 'reference', 'reference');  
END_MACRO;
```

WHEREX

WHEREX

Parameters: None.

Returns the current screen column.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_WHEREX FROM EDIT;  
    DEF_INT(Result);  
    Result := WHEREX;  
END_MACRO;
```

WHEREY

WHEREY

Parameters: None.

Returns the current screen row.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_WHEREY FROM EDIT;  
    DEF_INT(Result);  
    Result := WHEREY;  
END_MACRO;
```

XPOS

XPOS(needle, text, start)

Parameters:

needle String-like text sought in the supplied text.

text String-like text.

start Zero-based string start position.

Searches from start and returns the matching position, or zero.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_XPOS FROM EDIT;  
    DEF_INT(Result);  
    Result := XPOS('reference', 'reference', 0);  
END_MACRO;
```

Chapter 4

Control Flow and Macro Composition

Statements normally run in source order. Control-flow forms choose a branch, repeat a block, transfer to a label or return from a local call. Composition forms expose invocation parameters and run macros already loaded into the runtime catalogue.

4.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
IF	keyword	Control flow.	p. 59
WHILE	keyword	Control flow.	p. 59
GOTO	keyword	Control flow.	p. 60
CALL	keyword	Control flow.	p. 60
RET	keyword	Control flow.	p. 60
PARAM_STR	intrinsic	Control flow.	p. 60
THEN	keyword	True branch.	p. 61
ELSE	keyword	False branch.	p. 61
END	keyword	Block terminator.	p. 61
DO	keyword	Loop body.	p. 62
LABEL	source form	Branch label.	p. 62

4.2 Reference Entries

IF

```
IF integerExpression THEN
    statement;
ELSE
    statement;
END;
```

Parameters: integerExpression is non-zero for the true branch.
Selects one of two statement blocks.

Example macro.

```
$MACRO REFERENCE_IF FROM EDIT;
    IF TRUE THEN
        MAKE_MESSAGE('true');
    END;
END_MACRO;
```

WHILE

```
WHILE integerExpression DO
    statement;
END;
```

Parameters: integerExpression is evaluated before each iteration.
Repeats a statement block while its condition is non-zero.

Example macro.

```
$MACRO REFERENCE_WHILE FROM EDIT;
    DEF_INT(Count);
    WHILE Count < 1 DO
        Count := Count + 1;
    END;
END_MACRO;
```

GOTO

```
GOTO label;
```

Parameters: label names a label in the same macro.
Branches to a local label.

Example macro.

```
$MACRO REFERENCE_GOTO FROM EDIT;
    GOTO Done;
Done:
END_MACRO;
```

CALL

```
CALL label;
```

Parameters: label names a label in the same macro.
Calls a local label and records a return point.

Example macro.

```
$MACRO REFERENCE_CALL FROM EDIT;
    CALL Done;
Done:
    RET;
END_MACRO;
```

RET

```
RET;
```

Parameters: No parameters.
Returns from a local label call.

Example macro.

```
$MACRO REFERENCE_RET FROM EDIT;
    CALL Done;
Done:
    RET;
END_MACRO;
```

PARAM_STR

PARAM_STR(index)

Parameters:

index Zero-based parameter or selection index.

Returns one macro invocation parameter.

Returns a string.

Example macro.

```
$MACRO REFERENCE_PARAM_STR FROM EDIT;  
    DEF_STR(Result);  
    Result := PARAM_STR(0);  
END_MACRO;
```

THEN

```
IF expression THEN  
    statement;  
END;
```

Parameters: expression is an integer truth value; statement is a macro statement.

Separates an IF condition from its true branch.

Example macro.

```
$MACRO REFERENCE_THEN FROM EDIT;  
    IF TRUE THEN  
        MAKE_MESSAGE('true');  
    END;  
END_MACRO;
```

ELSE

```
ELSE  
    statement;
```

Parameters: statement is a macro statement.

Introduces the optional false branch of the nearest IF block.

Example macro.

```
$MACRO REFERENCE_ELSE FROM EDIT;  
    IF FALSE THEN  
        MAKE_MESSAGE('true');  
    ELSE  
        MAKE_MESSAGE('false');  
    END;  
END_MACRO;
```

END

```
END;
```

Parameters: None.

Terminates the nearest IF or WHILE block.

Example macro.

```
$MACRO REFERENCE_END FROM EDIT;  
    IF TRUE THEN  
        MAKE_MESSAGE('done');  
    END;  
END_MACRO;
```

DO

```
WHILE expression DO  
    statement;  
END;
```

Parameters: expression is an integer truth value; statement is a macro statement.
Separates a WHILE condition from its repeated body.

Example macro.

```
$MACRO REFERENCE_DO FROM EDIT;  
    WHILE FALSE DO  
        MAKE_MESSAGE('not reached');  
    END;  
END_MACRO;
```

Label

labelName:

Parameters: labelName is a unique local label identifier.
Defines the target used by GOTO and CALL in the same macro.

Example macro.

```
$MACRO REFERENCE_LABEL_FORM FROM EDIT;  
    GOTO REFERENCE_LABEL_DONE;  
REFERENCE_LABEL_DONE:  
END_MACRO;
```

Part II

Editor Automation

Chapter 5

Text and String Operations

Text primitives act through the current editor context. They obtain, insert and delete text and derive useful string or path values without exposing a document object. Each entry identifies the exact operation, its parameters and one complete macro.

5.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
GET_EXTENSION	intrinsic	Text operation.	p. 64
GET_PATH	intrinsic	Text operation.	p. 64
GET_WORD	intrinsic	Text operation.	p. 65
TRUNCATE_EXTENSION	intrinsic	Text operation.	p. 65
TRUNCATE_PATH	intrinsic	Text operation.	p. 65
BACK_SPACE	procedure	Text operation.	p. 66
CR	procedure	Text operation.	p. 66
DEL_CHAR	procedure	Text operation.	p. 66
DEL_LINE	procedure	Text operation.	p. 66
DEL_CHARS	procedure	Text operation.	p. 66
PUT_LINE	procedure	Text operation.	p. 67
TEXT	procedure	Text operation.	p. 67

5.2 Reference Entries

GET_EXTENSION

GET_EXTENSION(path)

Parameters:

path A string-like file-system path.

Returns the filename extension.
Returns a string.

Example macro.

```
$MACRO REFERENCE_GET_EXTENSION FROM EDIT;  
  DEF_STR(Result);  
  Result := GET_EXTENSION('reference');  
END_MACRO;
```

GET_PATH

GET_PATH(path)

Parameters:

path A string-like file-system path.

Returns the directory portion of a path.
Returns a string.

Example macro.

```
$MACRO REFERENCE_GET_PATH FROM EDIT;  
  DEF_STR(Result);  
  Result := GET_PATH('reference');  
END_MACRO;
```

GET_WORD

GET_WORD(delimiters)

Parameters:

delimiters String-like set of word-delimiter characters.

Returns the current editor word using the supplied delimiters.
Returns a string.

Example macro.

```
$MACRO REFERENCE_GET_WORD FROM EDIT;  
  DEF_STR(Result);  
  Result := GET_WORD('reference');  
END_MACRO;
```

TRUNCATE_EXTENSION

TRUNCATE_EXTENSION(path)

Parameters:

path A string-like file-system path.

Removes the filename extension.
Returns a string.

Example macro.

```
$MACRO REFERENCE_TRUNCATE_EXTENSION FROM EDIT;  
  DEF_STR(Result);  
  Result := TRUNCATE_EXTENSION('reference');  
END_MACRO;
```

TRUNCATE_PATH

TRUNCATE_PATH(path)

Parameters:

path A string-like file-system path.

Removes the directory portion.
Returns a string.

Example macro.

```
$MACRO REFERENCE_TRUNCATE_PATH FROM EDIT;  
    DEF_STR(Result);  
    Result := TRUNCATE_PATH('reference');  
END_MACRO;
```

BACK_SPACE

BACK_SPACE;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_BACK_SPACE FROM EDIT;  
    BACK_SPACE;  
END_MACRO;
```

CR

CR;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_CR FROM EDIT;  
    CR;  
END_MACRO;
```

DEL_CHAR

DEL_CHAR;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_DEL_CHAR FROM EDIT;  
    DEL_CHAR;  
END_MACRO;
```

DEL_LINE

DEL_LINE;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_DEL_LINE FROM EDIT;  
    DEL_LINE;  
END_MACRO;
```

DEL_CHARS

DEL_CHARS(count);

Parameters:

count Number of characters to process.

Deletes count characters at the current editor position.

Example macro.

```
$MACRO REFERENCE_DEL_CHARS FROM EDIT;  
    DEL_CHARS(0);  
END_MACRO;
```

PUT_LINE

PUT_LINE(text);

Parameters:

text String-like text.

Replaces or inserts the current editor line.

Example macro.

```
$MACRO REFERENCE_PUT_LINE FROM EDIT;  
    PUT_LINE('reference');  
END_MACRO;
```

TEXT

TEXT(text);

Parameters:

text String-like text.

Inserts text at the current editor cursor.

Example macro.

```
$MACRO REFERENCE_TEXT FROM EDIT;  
    TEXT('reference');  
END_MACRO;
```

Chapter 6

Cursor, Tabs and Indentation

Cursor primitives move within the active editor and view. Formatting primitives use the editor tab and indentation settings; they do not construct a private formatting model. Use the map to distinguish movement, marks, paging and indentation.

6.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
EXPAND_TABS	procedure	Cursor or formatting.	p. 69
TABS_TO_SPACES	procedure	Cursor or formatting.	p. 69
BACK_HOME	procedure	Cursor or formatting.	p. 70
BACK_WORD	procedure	Cursor or formatting.	p. 70
BOTTOM_OF_WINDOW	procedure	Cursor or formatting.	p. 70
CENTER_LINE	procedure	Cursor or formatting.	p. 70
CENTER_LINE_ON_SCREEN	procedure	Cursor or formatting.	p. 70
DOWN	procedure	Cursor or formatting.	p. 71
EOF	procedure	Cursor or formatting.	p. 71
EOL	procedure	Cursor or formatting.	p. 71
FIRST_WORD	procedure	Cursor or formatting.	p. 71
GET_RANDOM_MARK	procedure	Cursor or formatting.	p. 72
GOTO_MARK	procedure	Cursor or formatting.	p. 72
HOME	procedure	Cursor or formatting.	p. 72
INDENT	procedure	Cursor or formatting.	p. 72
INDENT_BLOCK	procedure	Cursor or formatting.	p. 72
JUSTIFY_PARAGRAPH	procedure	Cursor or formatting.	p. 73
LAST_PAGE_BREAK	procedure	Cursor or formatting.	p. 73
LEFT	procedure	Cursor or formatting.	p. 73
MARK_POS	procedure	Cursor or formatting.	p. 73
MOVE_VIEWPORT_LEFT	procedure	Cursor or formatting.	p. 74
MOVE_VIEWPORT_RIGHT	procedure	Cursor or formatting.	p. 74
NEXT_PAGE_BREAK	procedure	Cursor or formatting.	p. 74
PAGE_DOWN	procedure	Cursor or formatting.	p. 74
PAGE_UP	procedure	Cursor or formatting.	p. 74
POP_MARK	procedure	Cursor or formatting.	p. 75
REFORMAT_DOCUMENT	procedure	Cursor or formatting.	p. 75
REFORMAT_PARAGRAPH	procedure	Cursor or formatting.	p. 75
REVERSE_TAB	procedure	Cursor or formatting.	p. 75
RIGHT	procedure	Cursor or formatting.	p. 76

Primitive	Role	Purpose	Reference
SCROLL_DOWN	procedure	Cursor or formatting.	p. 76
SCROLL_UP	procedure	Cursor or formatting.	p. 76
SET_INDENT_LEVEL	procedure	Cursor or formatting.	p. 76
SET_LEFT_MARGIN	procedure	Cursor or formatting.	p. 76
SET_RANDOM_MARK	procedure	Cursor or formatting.	p. 77
SET_RIGHT_MARGIN	procedure	Cursor or formatting.	p. 77
TAB_LEFT	procedure	Cursor or formatting.	p. 77
TAB_RIGHT	procedure	Cursor or formatting.	p. 77
TOF	procedure	Cursor or formatting.	p. 78
TOGGLE_FORMAT_RULER	procedure	Cursor or formatting.	p. 78
TOGGLE_WORD_WRAP	procedure	Cursor or formatting.	p. 78
TOP_OF_WINDOW	procedure	Cursor or formatting.	p. 78
UNDENT	procedure	Cursor or formatting.	p. 78
UNDENT_BLOCK	procedure	Cursor or formatting.	p. 79
UP	procedure	Cursor or formatting.	p. 79
WORD_LEFT	procedure	Cursor or formatting.	p. 79
WORD_RIGHT	procedure	Cursor or formatting.	p. 79
WORD_WRAP_LINE	procedure	Cursor or formatting.	p. 80
GOTO_COL	procedure	Cursor or formatting.	p. 80
GOTO_LINE	procedure	Cursor or formatting.	p. 80

6.2 Reference Entries

EXPAND_TABS

```
EXPAND_TABS(textVariable);
EXPAND_TABS(textVariable, tabWidthVariable);
```

Parameters:

textVariable Writable string variable.

tabWidthVariable Integer variable holding the tab width.

Expands tab characters in the supplied string variable in place.

Example macro.

```
$MACRO REFERENCE_EXPAND_TABS FROM EDIT;
  DEF_STR(Text);
  DEF_INT(TabWidth);
  Text := 'a\tb';
  TabWidth := 4;
  EXPAND_TABS(Text, TabWidth);
END_MACRO;
```

TABS_TO_SPACES

```
TABS_TO_SPACES(textVariable);
```

Parameters:

textVariable Writable string variable.

Replaces tabs in the supplied string variable in place.

Example macro.

```
$MACRO REFERENCE_TABS_TO_SPACES FROM EDIT;  
  DEF_STR(Text);  
  Text := 'a\tb';  
  TABS_TO_SPACES(Text);  
END_MACRO;
```

BACK_HOME

BACK_HOME;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_BACK_HOME FROM EDIT;  
  BACK_HOME;  
END_MACRO;
```

BACK_WORD

BACK_WORD;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_BACK_WORD FROM EDIT;  
  BACK_WORD;  
END_MACRO;
```

BOTTOM_OF_WINDOW

BOTTOM_OF_WINDOW;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_BOTTOM_OF_WINDOW FROM EDIT;  
  BOTTOM_OF_WINDOW;  
END_MACRO;
```

CENTER_LINE

CENTER_LINE;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_CENTER_LINE FROM EDIT;  
  CENTER_LINE;  
END_MACRO;
```

CENTER_LINE_ON_SCREEN

CENTER_LINE_ON_SCREEN;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_CENTER_LINE_ON_SCREEN FROM EDIT;  
    CENTER_LINE_ON_SCREEN;  
END_MACRO;
```

DOWN

DOWN;

Parameters: None.

Moves the editor cursor one position in the named direction.

Example macro.

```
$MACRO REFERENCE_DOWN FROM EDIT;  
    DOWN;  
END_MACRO;
```

EOF

EOF;

Parameters: None.

Moves the editor cursor to the named line or file boundary.

Example macro.

```
$MACRO REFERENCE_EOF FROM EDIT;  
    EOF;  
END_MACRO;
```

EOL

EOL;

Parameters: None.

Moves the editor cursor to the named line or file boundary.

Example macro.

```
$MACRO REFERENCE_EOL FROM EDIT;  
    EOL;  
END_MACRO;
```

FIRST_WORD

FIRST_WORD;

Parameters: None.

Moves the editor cursor by word using the named direction or boundary.

Example macro.

```
$MACRO REFERENCE_FIRST_WORD FROM EDIT;  
    FIRST_WORD;  
END_MACRO;
```

GET_RANDOM_MARK

```
GET_RANDOM_MARK(markNumber);
```

Parameters:

markNumber Integer positional argument.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_GET_RANDOM_MARK FROM EDIT;  
    GET_RANDOM_MARK(0);  
END_MACRO;
```

GOTO_MARK

```
GOTO_MARK;
```

Parameters: None.

Performs the named operation on the editor mark stack.

Example macro.

```
$MACRO REFERENCE_GOTO_MARK FROM EDIT;  
    GOTO_MARK;  
END_MACRO;
```

HOME

```
HOME;
```

Parameters: None.

Moves the editor cursor to the named line or file boundary.

Example macro.

```
$MACRO REFERENCE_HOME FROM EDIT;  
    HOME;  
END_MACRO;
```

INDENT

```
INDENT;
```

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_INDENT FROM EDIT;  
    INDENT;  
END_MACRO;
```

INDENT_BLOCK

INDENT_BLOCK;

Parameters: None.

Indents the visible committed block through the editor's block command route. A stream block is first converted to a line block covering its touched lines. For line blocks, the least-indented nonblank line determines the delta to the next configured tab stop; every nonblank line receives that same delta, so relative indentation is preserved. Whitespace-only lines are unchanged. Column blocks retain their rectangular shift semantics. The document must be writable and a visible block must exist. The text change is undoable.

Example macro.

```
$MACRO REFERENCE_INDENT_BLOCK FROM EDIT;  
    INDENT_BLOCK;  
END_MACRO;
```

JUSTIFY_PARAGRAPH

JUSTIFY_PARAGRAPH;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_JUSTIFY_PARAGRAPH FROM EDIT;  
    JUSTIFY_PARAGRAPH;  
END_MACRO;
```

LAST_PAGE_BREAK

LAST_PAGE_BREAK;

Parameters: None.

Moves the editor view by the named page operation.

Example macro.

```
$MACRO REFERENCE_LAST_PAGE_BREAK FROM EDIT;  
    LAST_PAGE_BREAK;  
END_MACRO;
```

LEFT

LEFT;

Parameters: None.

Moves the editor cursor one position in the named direction.

Example macro.

```
$MACRO REFERENCE_LEFT FROM EDIT;  
    LEFT;  
END_MACRO;
```

MARK_POS

MARK_POS;

Parameters: None.

Performs the named operation on the editor mark stack.

Example macro.

```
$MACRO REFERENCE_MARK_POS FROM EDIT;  
    MARK_POS;  
END_MACRO;
```

MOVE__VIEWPORT__LEFT

```
MOVE_VIEWPORT_LEFT;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MOVE_VIEWPORT_LEFT FROM EDIT;  
    MOVE_VIEWPORT_LEFT;  
END_MACRO;
```

MOVE__VIEWPORT__RIGHT

```
MOVE_VIEWPORT_RIGHT;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MOVE_VIEWPORT_RIGHT FROM EDIT;  
    MOVE_VIEWPORT_RIGHT;  
END_MACRO;
```

NEXT__PAGE__BREAK

```
NEXT_PAGE_BREAK;
```

Parameters: None.

Moves the editor view by the named page operation.

Example macro.

```
$MACRO REFERENCE_NEXT_PAGE_BREAK FROM EDIT;  
    NEXT_PAGE_BREAK;  
END_MACRO;
```

PAGE__DOWN

```
PAGE_DOWN;
```

Parameters: None.

Moves the editor view by the named page operation.

Example macro.

```
$MACRO REFERENCE_PAGE_DOWN FROM EDIT;  
    PAGE_DOWN;  
END_MACRO;
```

PAGE_UP

PAGE_UP;

Parameters: None.

Moves the editor view by the named page operation.

Example macro.

```
$MACRO REFERENCE_PAGE_UP FROM EDIT;  
    PAGE_UP;  
END_MACRO;
```

POP_MARK

POP_MARK;

Parameters: None.

Performs the named operation on the editor mark stack.

Example macro.

```
$MACRO REFERENCE_POP_MARK FROM EDIT;  
    POP_MARK;  
END_MACRO;
```

REFORMAT_DOCUMENT

REFORMAT_DOCUMENT;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_REFORMAT_DOCUMENT FROM EDIT;  
    REFORMAT_DOCUMENT;  
END_MACRO;
```

REFORMAT_PARAGRAPH

REFORMAT_PARAGRAPH;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_REFORMAT_PARAGRAPH FROM EDIT;  
    REFORMAT_PARAGRAPH;  
END_MACRO;
```

REVERSE_TAB

REVERSE_TAB;

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_REVERSE_TAB FROM EDIT;  
    REVERSE_TAB;  
END_MACRO;
```

RIGHT

RIGHT;

Parameters: None.

Moves the editor cursor one position in the named direction.

Example macro.

```
$MACRO REFERENCE_RIGHT FROM EDIT;  
    RIGHT;  
END_MACRO;
```

SCROLL_DOWN

SCROLL_DOWN;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SCROLL_DOWN FROM EDIT;  
    SCROLL_DOWN;  
END_MACRO;
```

SCROLL_UP

SCROLL_UP;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SCROLL_UP FROM EDIT;  
    SCROLL_UP;  
END_MACRO;
```

SET_INDENT_LEVEL

SET_INDENT_LEVEL;

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_SET_INDENT_LEVEL FROM EDIT;  
    SET_INDENT_LEVEL;  
END_MACRO;
```

SET__LEFT__MARGIN

SET_LEFT_MARGIN;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SET_LEFT_MARGIN FROM EDIT;  
    SET_LEFT_MARGIN;  
END_MACRO;
```

SET__RANDOM__MARK

SET_RANDOM_MARK(markNumber);

Parameters:

markNumber Integer positional argument.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SET_RANDOM_MARK FROM EDIT;  
    SET_RANDOM_MARK(0);  
END_MACRO;
```

SET__RIGHT__MARGIN

SET_RIGHT_MARGIN;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SET_RIGHT_MARGIN FROM EDIT;  
    SET_RIGHT_MARGIN;  
END_MACRO;
```

TAB__LEFT

TAB_LEFT;

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_TAB_LEFT FROM EDIT;  
    TAB_LEFT;  
END_MACRO;
```

TAB__RIGHT

TAB_RIGHT;

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_TAB_RIGHT FROM EDIT;  
    TAB_RIGHT;  
END_MACRO;
```

TOF

TOF;

Parameters: None.

Moves the editor cursor to the named line or file boundary.

Example macro.

```
$MACRO REFERENCE_TOF FROM EDIT;  
    TOF;  
END_MACRO;
```

TOGGLE_FORMAT_RULER

TOGGLE_FORMAT_RULER;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_TOGGLE_FORMAT_RULER FROM EDIT;  
    TOGGLE_FORMAT_RULER;  
END_MACRO;
```

TOGGLE_WORD_WRAP

TOGGLE_WORD_WRAP;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_TOGGLE_WORD_WRAP FROM EDIT;  
    TOGGLE_WORD_WRAP;  
END_MACRO;
```

TOP_OF_WINDOW

TOP_OF_WINDOW;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_TOP_OF_WINDOW FROM EDIT;  
    TOP_OF_WINDOW;  
END_MACRO;
```

UNDENT

UNDENT;

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_UNDENT FROM EDIT;  
    UNDENT;  
END_MACRO;
```

UNDENT_BLOCK

UNDENT_BLOCK;

Parameters: None.

Undents the visible block towards the previous configured tab stop. Stream blocks are first converted to line blocks covering the touched lines. The least-indented nonblank line determines one shared reduction for a line block; relative indentation is preserved and whitespace-only lines are skipped. Column blocks retain their rectangular shift semantics. A missing visible block or read-only document prevents the mutation. The text change is undoable.

Example macro.

```
$MACRO REFERENCE_UNDENT_BLOCK FROM EDIT;  
    UNDENT_BLOCK;  
END_MACRO;
```

UP

UP;

Parameters: None.

Moves the editor cursor one position in the named direction.

Example macro.

```
$MACRO REFERENCE_UP FROM EDIT;  
    UP;  
END_MACRO;
```

WORD_LEFT

WORD_LEFT;

Parameters: None.

Moves the editor cursor by word using the named direction or boundary.

Example macro.

```
$MACRO REFERENCE_WORD_LEFT FROM EDIT;  
    WORD_LEFT;  
END_MACRO;
```

WORD_RIGHT

WORD_RIGHT;

Parameters: None.

Moves the editor cursor by word using the named direction or boundary.

Example macro.

```
$MACRO REFERENCE_WORD_RIGHT FROM EDIT;  
    WORD_RIGHT;  
END_MACRO;
```

WORD_WRAP_LINE

```
WORD_WRAP_LINE;
```

Parameters: None.

Performs the named tab, indentation or line-wrap action using active editor settings.

Example macro.

```
$MACRO REFERENCE_WORD_WRAP_LINE FROM EDIT;  
    WORD_WRAP_LINE;  
END_MACRO;
```

GOTO_COL

```
GOTO_COL(column);
```

Parameters:

column Screen or editor column, as required by the primitive.

Moves the editor cursor to the specified column.

Example macro.

```
$MACRO REFERENCE_GOTO_COL FROM EDIT;  
    GOTO_COL(0);  
END_MACRO;
```

GOTO_LINE

```
GOTO_LINE(line);
```

Parameters:

line One-based editor line number.

Moves the editor cursor to the specified line.

Example macro.

```
$MACRO REFERENCE_GOTO_LINE FROM EDIT;  
    GOTO_LINE(0);  
END_MACRO;
```

Chapter 7

Blocks, Clipboard and Undo

A block is the current editor selection. Block primitives start, extend, inspect, copy, move, delete and persist that selection. Clipboard and undo operations use editor-owned state in the active execution context.

7.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
BLOCK_TEXT	intrinsic	Block operation.	p. 82
APPEND_BLOCK	procedure	Block operation.	p. 82
BLOCK_BEGIN	procedure	Block operation.	p. 82
BLOCK_END	procedure	Block operation.	p. 82
BLOCK_MARK_LINES	procedure	Block operation.	p. 82
BLOCK_OFF	procedure	Block operation.	p. 83
BLOCK_STAT	procedure	Block operation.	p. 83
BLOCK_TOGGLE_MARKING	procedure	Block operation.	p. 83
BLOCK_TOGGLE_VISIBILITY	procedure	Block operation.	p. 83
COL_BLOCK_BEGIN	procedure	Block operation.	p. 84
COPY_BLOCK	procedure	Block operation.	p. 84
COPY_BLOCK_TO_CLIPBOARD	procedure	Block operation.	p. 84
CUT_APPEND_BLOCK	procedure	Block operation.	p. 84
CUT_BLOCK	procedure	Block operation.	p. 84
DELETE_BLOCK	procedure	Block operation.	p. 85
DEL_CHAR_OR_BLOCK	procedure	Block operation.	p. 85
DEL_EOL	procedure	Block operation.	p. 85
DEL_WORD	procedure	Block operation.	p. 85
END_OF_BLOCK	procedure	Block operation.	p. 86
EXTEND_BLOCK_BY_MOTION	procedure	Block operation.	p. 86
MARK_WORD_RIGHT	procedure	Block operation.	p. 86
MOVE_BLOCK	procedure	Block operation.	p. 86
PASTE_BLOCK	procedure	Block operation.	p. 86
PASTE_FROM_CLIPBOARD	procedure	Block operation.	p. 87
REDO	procedure	Block operation.	p. 87
SORT_COLUMN_BLOCK_TOGGLE	procedure	Block operation.	p. 87
START_OF_BLOCK	procedure	Block operation.	p. 87
STR_BLOCK_BEGIN	procedure	Block operation.	p. 88
UNDO	procedure	Block operation.	p. 88
LOAD_BLOCK	procedure	Block operation.	p. 88

Primitive	Role	Purpose	Reference
SAVE_BLOCK	procedure	Block operation.	p. 88

7.2 Reference Entries

BLOCK_TEXT

BLOCK_TEXT

Parameters: None.

Returns the selected block text.

Returns a string.

Example macro.

```
$MACRO REFERENCE_BLOCK_TEXT FROM EDIT;
  DEF_STR(Result);
  Result := BLOCK_TEXT;
END_MACRO;
```

APPEND_BLOCK

APPEND_BLOCK;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_APPEND_BLOCK FROM EDIT;
  APPEND_BLOCK;
END_MACRO;
```

This entry is an MRMAC editor-command extension.

BLOCK_BEGIN

BLOCK_BEGIN;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_BEGIN FROM EDIT;
  BLOCK_BEGIN;
END_MACRO;
```

BLOCK_END

BLOCK_END;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_END FROM EDIT;
  BLOCK_END;
END_MACRO;
```

BLOCK_MARK_LINES

BLOCK_MARK_LINES;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_MARK_LINES FROM EDIT;  
    BLOCK_MARK_LINES;  
END_MACRO;
```

BLOCK_OFF

BLOCK_OFF;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_OFF FROM EDIT;  
    BLOCK_OFF;  
END_MACRO;
```

BLOCK_STAT

BLOCK_STAT;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_STAT FROM EDIT;  
    BLOCK_STAT;  
END_MACRO;
```

BLOCK_TOGGLE_MARKING

BLOCK_TOGGLE_MARKING;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_TOGGLE_MARKING FROM EDIT;  
    BLOCK_TOGGLE_MARKING;  
END_MACRO;
```

BLOCK_TOGGLE_VISIBILITY

BLOCK_TOGGLE_VISIBILITY;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_BLOCK_TOGGLE_VISIBILITY FROM EDIT;  
    BLOCK_TOGGLE_VISIBILITY;  
END_MACRO;
```

COL_BLOCK_BEGIN

COL_BLOCK_BEGIN;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_COL_BLOCK_BEGIN FROM EDIT;  
    COL_BLOCK_BEGIN;  
END_MACRO;
```

COPY_BLOCK

COPY_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_COPY_BLOCK FROM EDIT;  
    COPY_BLOCK;  
END_MACRO;
```

COPY_BLOCK_TO_CLIPBOARD

COPY_BLOCK_TO_CLIPBOARD;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_COPY_BLOCK_TO_CLIPBOARD FROM EDIT;  
    COPY_BLOCK_TO_CLIPBOARD;  
END_MACRO;
```

CUT_APPEND_BLOCK

CUT_APPEND_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_CUT_APPEND_BLOCK FROM EDIT;  
    CUT_APPEND_BLOCK;  
END_MACRO;
```

CUT_BLOCK

CUT_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_CUT_BLOCK FROM EDIT;  
    CUT_BLOCK;  
END_MACRO;
```

DELETE_BLOCK

DELETE_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DELETE_BLOCK FROM EDIT;  
    DELETE_BLOCK;  
END_MACRO;
```

DEL_CHAR_OR_BLOCK

DEL_CHAR_OR_BLOCK;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_DEL_CHAR_OR_BLOCK FROM EDIT;  
    DEL_CHAR_OR_BLOCK;  
END_MACRO;
```

DEL_EOL

DEL_EOL;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_DEL_EOL FROM EDIT;  
    DEL_EOL;  
END_MACRO;
```

DEL_WORD

DEL_WORD;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_DEL_WORD FROM EDIT;  
    DEL_WORD;  
END_MACRO;
```

END_OF_BLOCK

```
END_OF_BLOCK;
```

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_END_OF_BLOCK FROM EDIT;  
    END_OF_BLOCK;  
END_MACRO;
```

EXTEND_BLOCK_BY_MOTION

```
EXTEND_BLOCK_BY_MOTION(motionName);
```

Parameters:

motionName String or character positional argument.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_EXTEND_BLOCK_BY_MOTION FROM EDIT;  
    EXTEND_BLOCK_BY_MOTION('reference');  
END_MACRO;
```

MARK_WORD_RIGHT

```
MARK_WORD_RIGHT;
```

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_MARK_WORD_RIGHT FROM EDIT;  
    MARK_WORD_RIGHT;  
END_MACRO;
```

MOVE_BLOCK

```
MOVE_BLOCK;
```

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MOVE_BLOCK FROM EDIT;  
    MOVE_BLOCK;  
END_MACRO;
```

PASTE_BLOCK

PASTE_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_PASTE_BLOCK FROM EDIT;  
    PASTE_BLOCK;  
END_MACRO;
```

PASTE_FROM_CLIPBOARD

PASTE_FROM_CLIPBOARD;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_PASTE_FROM_CLIPBOARD FROM EDIT;  
    PASTE_FROM_CLIPBOARD;  
END_MACRO;
```

REDO

REDO;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_REDO FROM EDIT;  
    REDO;  
END_MACRO;
```

SORT_COLUMN_BLOCK_TOGGLE

SORT_COLUMN_BLOCK_TOGGLE;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SORT_COLUMN_BLOCK_TOGGLE FROM EDIT;  
    SORT_COLUMN_BLOCK_TOGGLE;  
END_MACRO;
```

START_OF_BLOCK

START_OF_BLOCK;

Parameters: None.

Performs the named selection or block operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_START_OF_BLOCK FROM EDIT;  
    START_OF_BLOCK;  
END_MACRO;
```

STR_BLOCK_BEGIN

```
STR_BLOCK_BEGIN;
```

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_STR_BLOCK_BEGIN FROM EDIT;  
    STR_BLOCK_BEGIN;  
END_MACRO;
```

UNDO

```
UNDO;
```

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_UNDO FROM EDIT;  
    UNDO;  
END_MACRO;
```

LOAD_BLOCK

```
LOAD_BLOCK(path);
```

Parameters:

path A string-like file-system path.

Loads a block from path.

Example macro.

```
$MACRO REFERENCE_LOAD_BLOCK FROM EDIT;  
    LOAD_BLOCK('reference');  
END_MACRO;
```

SAVE_BLOCK

```
SAVE_BLOCK(path);
```

Parameters:

path A string-like file-system path.

Saves the current block to path.

Example macro.

```
$MACRO REFERENCE_SAVE_BLOCK FROM EDIT;  
    SAVE_BLOCK('reference');  
END_MACRO;
```

Chapter 8

Files, Search and Windows

File primitives operate on named files or the active editor file. Search primitives use editor search state. Window primitives select or arrange editor windows and workspaces. Their effects are on editor resources, rather than a local macro value.

8.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
<code>COPY_FILE</code>	procedure	File, search or window.	p. 90
<code>FILE_ATTR</code>	intrinsic	File, search or window.	p. 90
<code>FILE_EXISTS</code>	intrinsic	File, search or window.	p. 91
<code>FIRST_FILE</code>	intrinsic	File, search or window.	p. 91
<code>NEXT_FILE</code>	intrinsic	File, search or window.	p. 91
<code>RENAME_FILE</code>	intrinsic	File, search or window.	p. 92
<code>SEARCH_BWD</code>	intrinsic	File, search or window.	p. 92
<code>SEARCH_FWD</code>	intrinsic	File, search or window.	p. 92
<code>SWITCH_FILE</code>	intrinsic	File, search or window.	p. 92
<code>CREATE_WINDOW</code>	procedure	File, search or window.	p. 93
<code>DELETE_WINDOW</code>	procedure	File, search or window.	p. 93
<code>DESKTOP_MOVE_WINDOW_LEFT</code>	procedure	File, search or window.	p. 93
<code>DESKTOP_MOVE_WINDOW_RIGHT</code>	procedure	File, search or window.	p. 93
<code>DESKTOP_VIEWPORT_LEFT</code>	procedure	File, search or window.	p. 94
<code>DESKTOP_VIEWPORT_RIGHT</code>	procedure	File, search or window.	p. 94
<code>ERASE_WINDOW</code>	procedure	File, search or window.	p. 94
<code>FORCE_SAVE</code>	procedure	File, search or window.	p. 94
<code>LINK_WINDOW</code>	procedure	File, search or window.	p. 94
<code>LIST_MATCHED_FILES</code>	procedure	File, search or window.	p. 95
<code>LOAD_WORKSPACE</code>	procedure	File, search or window.	p. 95
<code>MODIFY_WINDOW</code>	procedure	File, search or window.	p. 95
<code>MOVE_WIN_TO_NEXT_DESKTOP</code>	procedure	File, search or window.	p. 95
<code>MOVE_WIN_TO_PREV_DESKTOP</code>	procedure	File, search or window.	p. 96
<code>MULTI_FILE_SEARCH</code>	procedure	File, search or window.	p. 96
<code>MULTI_FILE_SEARCH_REPLACE</code>	procedure	File, search or window.	p. 96
<code>NEW_SCREEN</code>	procedure	File, search or window.	p. 96
<code>NEXT_SEARCH_RESULT</code>	procedure	File, search or window.	p. 96
<code>REDRAW</code>	procedure	File, search or window.	p. 97
<code>REPEAT_SEARCH</code>	procedure	File, search or window.	p. 97
<code>REVERT_FILE</code>	procedure	File, search or window.	p. 97

Primitive	Role	Purpose	Reference
SAVE_ALL	procedure	File, search or window.	p. 97
SAVE_WORKSPACE	procedure	File, search or window.	p. 98
SEARCH	procedure	File, search or window.	p. 98
SEARCH_REPLACE	procedure	File, search or window.	p. 98
SWITCH_WINDOW	procedure	File, search or window.	p. 98
UNLINK_WINDOW	procedure	File, search or window.	p. 98
WINDOW_CASCADE	procedure	File, search or window.	p. 99
WINDOW_CLOSE	procedure	File, search or window.	p. 99
WINDOW_COPY_BLOCK	procedure	File, search or window.	p. 99
WINDOW_LIST	procedure	File, search or window.	p. 99
WINDOW_MOVE_BLOCK	procedure	File, search or window.	p. 100
WINDOW_NEXT	procedure	File, search or window.	p. 100
WINDOW_OPEN	procedure	File, search or window.	p. 100
WINDOW_PREVIOUS	procedure	File, search or window.	p. 100
WINDOW_SPLIT_HORIZONTAL	procedure	File, search or window.	p. 100
WINDOW_SPLIT_VERTICAL	procedure	File, search or window.	p. 101
WINDOW_TILE	procedure	File, search or window.	p. 101
WINDOW_ZOOM	procedure	File, search or window.	p. 101
ZOOM	procedure	File, search or window.	p. 101
DEL_FILE	procedure	File, search or window.	p. 102
LOAD_FILE	procedure	File, search or window.	p. 102
REPLACE	procedure	File, search or window.	p. 102
SAVE_FILE	procedure	File, search or window.	p. 102
SET_FILE_ATTR	procedure	File, search or window.	p. 103
SIZE_WINDOW	procedure	File, search or window.	p. 103
WINDOW_COPY	procedure	File, search or window.	p. 103
WINDOW_MOVE	procedure	File, search or window.	p. 103

8.2 Reference Entries

COPY_FILE

```
COPY_FILE(sourcePath, targetPath);
COPY_FILE(sourcePath, targetPath, overwrite);
```

Parameters:

sourcePath Source file path.

targetPath Destination file path.

overwrite Non-zero permits overwrite.

Copies a file and returns a status value.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_COPY_FILE FROM EDIT;
    DEF_INT(Result);
    Result := COPY_FILE('reference', 'reference', 0);
END_MACRO;
```

FILE_ATTR**FILE_ATTR**(path)

Parameters:

path A string-like file-system path.

Returns the file attribute mask.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_FILE_ATTR FROM EDIT;  
  DEF_INT(Result);  
  Result := FILE_ATTR('reference');  
END_MACRO;
```

FILE_EXISTS**FILE_EXISTS**(path)

Parameters:

path A string-like file-system path.

Returns non-zero when the path exists.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_FILE_EXISTS FROM EDIT;  
  DEF_INT(Result);  
  Result := FILE_EXISTS('reference');  
END_MACRO;
```

FIRST_FILE**FIRST_FILE**(pattern)

Parameters:

pattern File-name pattern.

Starts pattern enumeration and returns the first status value.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_FIRST_FILE FROM EDIT;  
  DEF_INT(Result);  
  Result := FIRST_FILE('reference');  
END_MACRO;
```

NEXT_FILE**NEXT_FILE**

Parameters: None.

Advances the active file enumeration.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_NEXT_FILE FROM EDIT;  
    DEF_INT(Result);  
    Result := NEXT_FILE;  
END_MACRO;
```

RENAME__FILE

RENAME_FILE(sourcePath, targetPath)

Parameters:

sourcePath Source file path.

targetPath Destination file path.

Renames a file and returns a status value.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_RENAME_FILE FROM EDIT;  
    DEF_INT(Result);  
    Result := RENAME_FILE('reference', 'reference');  
END_MACRO;
```

SEARCH__BWD

SEARCH_BWD(searchText, options)

Parameters:

searchText Text to search for.

options Integer search-option mask.

Searches backward and returns a status value.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_SEARCH_BWD FROM EDIT;  
    DEF_INT(Result);  
    Result := SEARCH_BWD('reference', 0);  
END_MACRO;
```

SEARCH__FWD

SEARCH_FWD(searchText, options)

Parameters:

searchText Text to search for.

options Integer search-option mask.

Searches forward and returns a status value.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_SEARCH_FWD FROM EDIT;  
    DEF_INT(Result);  
    Result := SEARCH_FWD('reference', 0);  
END_MACRO;
```

SWITCH_FILE

SWITCH_FILE(fileName)

Parameters:

fileName Name of an open file.

Selects an already open file and returns a status value.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_SWITCH_FILE FROM EDIT;  
    DEF_INT(Result);  
    Result := SWITCH_FILE('reference');  
END_MACRO;
```

CREATE_WINDOW

CREATE_WINDOW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_CREATE_WINDOW FROM EDIT;  
    CREATE_WINDOW;  
END_MACRO;
```

DELETE_WINDOW

DELETE_WINDOW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DELETE_WINDOW FROM EDIT;  
    DELETE_WINDOW;  
END_MACRO;
```

DESKTOP_MOVE_WINDOW_LEFT

DESKTOP_MOVE_WINDOW_LEFT;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DESKTOP_MOVE_WINDOW_LEFT FROM EDIT;  
    DESKTOP_MOVE_WINDOW_LEFT;  
END_MACRO;
```

DESKTOP_MOVE_WINDOW_RIGHT

DESKTOP_MOVE_WINDOW_RIGHT;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DESKTOP_MOVE_WINDOW_RIGHT FROM EDIT;  
    DESKTOP_MOVE_WINDOW_RIGHT;  
END_MACRO;
```

DESKTOP_VIEWPORT_LEFT

```
DESKTOP_VIEWPORT_LEFT;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DESKTOP_VIEWPORT_LEFT FROM EDIT;  
    DESKTOP_VIEWPORT_LEFT;  
END_MACRO;
```

DESKTOP_VIEWPORT_RIGHT

```
DESKTOP_VIEWPORT_RIGHT;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_DESKTOP_VIEWPORT_RIGHT FROM EDIT;  
    DESKTOP_VIEWPORT_RIGHT;  
END_MACRO;
```

ERASE_WINDOW

```
ERASE_WINDOW;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_ERASE_WINDOW FROM EDIT;  
    ERASE_WINDOW;  
END_MACRO;
```

FORCE_SAVE

```
FORCE_SAVE;
```

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_FORCE_SAVE FROM EDIT;  
    FORCE_SAVE;  
END_MACRO;
```

LINK_WINDOW

LINK_WINDOW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_LINK_WINDOW FROM EDIT;  
    LINK_WINDOW;  
END_MACRO;
```

LIST_MATCHED_FILES

LIST_MATCHED_FILES;

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_LIST_MATCHED_FILES FROM EDIT;  
    LIST_MATCHED_FILES;  
END_MACRO;
```

LOAD_WORKSPACE

LOAD_WORKSPACE;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_LOAD_WORKSPACE FROM EDIT;  
    LOAD_WORKSPACE;  
END_MACRO;
```

MODIFY_WINDOW

MODIFY_WINDOW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MODIFY_WINDOW FROM EDIT;  
    MODIFY_WINDOW;  
END_MACRO;
```

MOVE_WIN_TO_NEXT_DESKTOP

MOVE_WIN_TO_NEXT_DESKTOP;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MOVE_WIN_TO_NEXT_DESKTOP FROM EDIT;  
    MOVE_WIN_TO_NEXT_DESKTOP;  
END_MACRO;
```

MOVE_WIN_TO_PREV_DESKTOP

```
MOVE_WIN_TO_PREV_DESKTOP;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MOVE_WIN_TO_PREV_DESKTOP FROM EDIT;  
    MOVE_WIN_TO_PREV_DESKTOP;  
END_MACRO;
```

MULTI_FILE_SEARCH

```
MULTI_FILE_SEARCH;
```

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MULTI_FILE_SEARCH FROM EDIT;  
    MULTI_FILE_SEARCH;  
END_MACRO;
```

MULTI_FILE_SEARCH_REPLACE

```
MULTI_FILE_SEARCH_REPLACE;
```

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_MULTI_FILE_SEARCH_REPLACE FROM EDIT;  
    MULTI_FILE_SEARCH_REPLACE;  
END_MACRO;
```

NEW_SCREEN

```
NEW_SCREEN;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_NEW_SCREEN FROM EDIT;  
    NEW_SCREEN;  
END_MACRO;
```

NEXT_SEARCH_RESULT

NEXT_SEARCH_RESULT;

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_NEXT_SEARCH_RESULT FROM EDIT;  
    NEXT_SEARCH_RESULT;  
END_MACRO;
```

REDRAW

REDRAW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_REDRAW FROM EDIT;  
    REDRAW;  
END_MACRO;
```

REPEAT_SEARCH

REPEAT_SEARCH;

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_REPEAT_SEARCH FROM EDIT;  
    REPEAT_SEARCH;  
END_MACRO;
```

REVERT_FILE

REVERT_FILE;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_REVERT_FILE FROM EDIT;  
    REVERT_FILE;  
END_MACRO;
```

SAVE_ALL

SAVE_ALL;

Parameters: None.

Performs the named editor action through the active MR editor command route.

Example macro.

```
$MACRO REFERENCE_SAVE_ALL FROM EDIT;  
    SAVE_ALL;  
END_MACRO;
```

SAVE__WORKSPACE

```
SAVE_WORKSPACE;
```

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_SAVE_WORKSPACE FROM EDIT;  
    SAVE_WORKSPACE;  
END_MACRO;
```

SEARCH

```
SEARCH;
```

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_SEARCH FROM EDIT;  
    SEARCH;  
END_MACRO;
```

SEARCH__REPLACE

```
SEARCH_REPLACE;
```

Parameters: None.

Invokes the named MR search operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_SEARCH_REPLACE FROM EDIT;  
    SEARCH_REPLACE;  
END_MACRO;
```

SWITCH__WINDOW

```
SWITCH_WINDOW(windowIndex);
```

Parameters:

windowIndex Integer positional argument.

Activates windowIndex.

Example macro.

```
$MACRO REFERENCE_SWITCH_WINDOW FROM EDIT;  
    SWITCH_WINDOW(0);  
END_MACRO;
```

UNLINK_WINDOW

UNLINK_WINDOW;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_UNLINK_WINDOW FROM EDIT;  
    UNLINK_WINDOW;  
END_MACRO;
```

WINDOW_CASCADE

WINDOW_CASCADE;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_CASCADE FROM EDIT;  
    WINDOW_CASCADE;  
END_MACRO;
```

WINDOW_CLOSE

WINDOW_CLOSE;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_CLOSE FROM EDIT;  
    WINDOW_CLOSE;  
END_MACRO;
```

WINDOW_COPY_BLOCK

WINDOW_COPY_BLOCK;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_COPY_BLOCK FROM EDIT;  
    WINDOW_COPY_BLOCK;  
END_MACRO;
```

WINDOW_LIST

WINDOW_LIST;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_LIST FROM EDIT;  
    WINDOW_LIST;  
END_MACRO;
```

WINDOW_MOVE_BLOCK

```
WINDOW_MOVE_BLOCK;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_MOVE_BLOCK FROM EDIT;  
    WINDOW_MOVE_BLOCK;  
END_MACRO;
```

WINDOW_NEXT

```
WINDOW_NEXT;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_NEXT FROM EDIT;  
    WINDOW_NEXT;  
END_MACRO;
```

WINDOW_OPEN

```
WINDOW_OPEN;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_OPEN FROM EDIT;  
    WINDOW_OPEN;  
END_MACRO;
```

WINDOW_PREVIOUS

```
WINDOW_PREVIOUS;
```

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_PREVIOUS FROM EDIT;  
    WINDOW_PREVIOUS;  
END_MACRO;
```

WINDOW_SPLIT_HORIZONTAL

WINDOW_SPLIT_HORIZONTAL;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_SPLIT_HORIZONTAL FROM EDIT;  
    WINDOW_SPLIT_HORIZONTAL;  
END_MACRO;
```

WINDOW_SPLIT_VERTICAL

WINDOW_SPLIT_VERTICAL;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_SPLIT_VERTICAL FROM EDIT;  
    WINDOW_SPLIT_VERTICAL;  
END_MACRO;
```

WINDOW_TILE

WINDOW_TILE;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_TILE FROM EDIT;  
    WINDOW_TILE;  
END_MACRO;
```

WINDOW_ZOOM

WINDOW_ZOOM;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_WINDOW_ZOOM FROM EDIT;  
    WINDOW_ZOOM;  
END_MACRO;
```

ZOOM

ZOOM;

Parameters: None.

Performs the named MR window, screen or desktop operation in the active editor context.

Example macro.

```
$MACRO REFERENCE_ZOOM FROM EDIT;  
    ZOOM;  
END_MACRO;
```

DEL _FILE

```
DEL_FILE(path);
```

Parameters:

path A string-like file-system path.

Deletes path through the editor file route.

Example macro.

```
$MACRO REFERENCE_DEL_FILE FROM EDIT;  
    DEL_FILE('reference');  
END_MACRO;
```

LOAD _FILE

```
LOAD_FILE(path);
```

Parameters:

path A string-like file-system path.

Loads path into the editor.

Example macro.

```
$MACRO REFERENCE_LOAD_FILE FROM EDIT;  
    LOAD_FILE('reference');  
END_MACRO;
```

REPLACE

```
REPLACE(replacement);
```

Parameters:

replacement Replacement text.

Performs replacement using the current search state.

Example macro.

```
$MACRO REFERENCE_REPLACE FROM EDIT;  
    REPLACE('reference');  
END_MACRO;
```

SAVE _FILE

```
SAVE_FILE;
```

Parameters: None.

Saves the active editor file.

Example macro.

```
$MACRO REFERENCE_SAVE_FILE FROM EDIT;  
    SAVE_FILE;  
END_MACRO;
```

SET__FILE__ATTR

```
SET_FILE_ATTR(path, attributes);
```

Parameters:

path A string-like file-system path.

attributes Integer file attribute mask.

Applies the requested file attribute mask.

Example macro.

```
$MACRO REFERENCE_SET_FILE_ATTR FROM EDIT;  
    SET_FILE_ATTR('reference', 0);  
END_MACRO;
```

SIZE__WINDOW

```
SIZE_WINDOW(left, top, right, bottom);
```

Parameters:

left Left screen column.

top Top screen row.

right Right screen column.

bottom Bottom screen row.

Sets current window bounds.

Example macro.

```
$MACRO REFERENCE_SIZE_WINDOW FROM EDIT;  
    SIZE_WINDOW(0, 0, 0, 0);  
END_MACRO;
```

WINDOW__COPY

```
WINDOW_COPY(windowIndex);
```

Parameters:

windowIndex Integer positional argument.

Copies the current window to windowIndex.

Example macro.

```
$MACRO REFERENCE_WINDOW_COPY FROM EDIT;  
    WINDOW_COPY(0);  
END_MACRO;
```

WINDOW_MOVE

```
WINDOW_MOVE(windowIndex);
```

Parameters:

windowIndex Integer positional argument.

Moves the current window to windowIndex.

Example macro.

```
$MACRO REFERENCE_WINDOW_MOVE FROM EDIT;  
    WINDOW_MOVE(0);  
END_MACRO;
```

Part III

Interaction and UI

Chapter 9

Screen, Input and Key Dispatch

Interaction primitives request or stage user-facing work through the VM and UI boundary. They do not give macro source a drawing context, event object or direct keymap store. Use this chapter for screen output, input, feedback and key bindings.

9.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
BEEP	procedure	User interaction.	p. 107
BRAIN	procedure	User interaction.	p. 107
CLEAR_SCREEN	procedure	User interaction.	p. 107
CLR_LINE	procedure	User interaction.	p. 108
CMD_TO_KEY	procedure	User interaction.	p. 110
DESKTOP_BLIT	procedure	MR desktop background rendering.	p. 109
DESKTOP_CLEAR	procedure	MR desktop background rendering.	p. 109
DESKTOP_PUT_CHAR	procedure	MR desktop background rendering.	p. 110
DESKTOP_PUT_STRING	procedure	MR desktop background rendering.	p. 110
DESKTOP_SET_COLOR	procedure	MR desktop background rendering.	p. 110
FLABEL	procedure	User interaction.	p. 111
GOTOXY	procedure	User interaction.	p. 111
KEY_IN	procedure	User interaction.	p. 111
KEY_RECORD	procedure	User interaction.	p. 112
KILL_BOX	procedure	User interaction.	p. 112
MACRO_TO_KEY	procedure	User interaction.	p. 112
MAKE_MESSAGE	procedure	User interaction.	p. 112
MARQUEE	procedure	User interaction.	p. 113
MARQUEE_ERROR	procedure	User interaction.	p. 113
MARQUEE_WARNING	procedure	User interaction.	p. 113
PASS_KEY	procedure	User interaction.	p. 113
PLAY_KEY_MACRO	procedure	User interaction.	p. 114
POP_LABELS	procedure	User interaction.	p. 114
PUSH_KEY	procedure	User interaction.	p. 114
PUSH_LABELS	procedure	User interaction.	p. 114

Primitive	Role	Purpose	Reference
PUT_BOX	procedure	User interaction.	p. 115
PUT_COL_NUM	procedure	User interaction.	p. 115
PUT_LINE_NUM	procedure	User interaction.	p. 115
READ_KEY	procedure	User interaction.	p. 115
REGISTER_MENU_ITEM	procedure	User interaction.	p. 116
REMOVE_MENU_ITEM	procedure	User interaction.	p. 116
SCROLL_BOX_DN	procedure	User interaction.	p. 116
SCROLL_BOX_UP	procedure	User interaction.	p. 117
SET_CLIPBOARD_TEXT	procedure	User interaction.	p. 117
UNASSIGN_ALL_KEYS	procedure	User interaction.	p. 117
UNASSIGN_KEY	procedure	User interaction.	p. 117
WORKING	procedure	User interaction.	p. 118
WRITE	procedure	User interaction.	p. 118

9.2 Reference Entries

BEEP

BEEP;

Parameters: None.

Requests the standard editor beep.

Example macro.

```
$MACRO REFERENCE_BEEP FROM EDIT;
    BEEP;
END_MACRO;
```

BRAIN

BRAIN(enabled);

Parameters:

enabled Integer positional argument.

Enables or disables the typed busy indicator.

Example macro.

```
$MACRO REFERENCE_BRAIN FROM EDIT;
    BRAIN(0);
END_MACRO;
```

CLEAR_SCREEN

```
CLEAR_SCREEN();
CLEAR_SCREEN(attribute);
```

Parameters:

attribute Screen attribute code.

Clears the staged screen using the optional attribute.

Example macro.

```
$MACRO REFERENCE_CLEAR_SCREEN FROM EDIT;  
    CLEAR_SCREEN(0);  
END_MACRO;
```

CLR_LINE

```
CLR_LINE();  
CLR_LINE(column, row, attribute);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

attribute Screen attribute code.

Clears the current line or the specified screen range.

Example macro.

```
$MACRO REFERENCE_CLR_LINE FROM EDIT;  
    CLR_LINE(0, 0, 0);  
END_MACRO;
```

DESKTOP_BLIT

DESKTOP_BLIT(path, column, row, mode);

Parameters:

path Path of a binary-read UTF-8 ANSI image file.

column One-based column in the MR desktop background.

row One-based row in the MR desktop background.

mode Rendering mode: 0 for direct ANSI colour or 1 for desktop-adaptive monochrome.

Reads and renders an ANSI image without an intermediate colour-array or image conversion. Standard ANSI foreground colours 30 through 37 and 90 through 97, and background colours 40 through 47 and 100 through 107, are mapped directly to their equivalent TVision/BIOS colour nibbles in mode 0. Reset, default colour, bold foreground and inverse-video SGR parameters are supported.

Mode 1 maps all 16 decoded ANSI colours to distinct, fixed, luminance-ordered monochrome ranks. Equal upper/lower block samples use the CP437 **light-shade**, **medium-shade**, **dark-shade** and **full-block** progression. A deterministic 4 by 4 ordered dither selects between adjacent progression levels. Unequal upper/lower samples remain two independent luminance planes: one common ordered threshold selects the light-shade, upper-half, lower-half or full-block glyph, so the two samples are never averaged. Ordinary non-block glyphs remain unchanged. The configured desktop attribute supplies the only output colour pair, and minimum luminance reproduces the native desktop shade instead of introducing a solid rectangular background. The conversion is fixed and does not depend on image histograms, placement or clipping.

The parser accepts CR, LF, backspace, tab, UTF-8 text representable in the active TVision code page, CSI cursor positioning and movement (H, f, A through D, G, d), cursor save/restore, full erase 2J, and cursor visibility ?25h/?25l. LF begins the next image row. Unsupported or malformed control sequences and unrepresentable characters are reported instead of being discarded. Input is limited to 16 MiB, dimensions below 4096 cells per axis and at most 1,048,576 image cells.

The decoded rectangle is clipped to the desktop background. The operation does not change the current desktop drawing attribute. It performs external file I/O and therefore executes on the UI-thread route rather than as a staged background macro operation. Windows remain above the canvas, and the menu and FLabel lines are outside its projection bounds. The canvas is volatile runtime state shared by all virtual desktops.

Example macro.

```
$MACRO REFERENCE_DESKTOP_BLIT FROM EDIT;
    DESKTOP_CLEAR();
    DESKTOP_BLIT('misc/example.ans', 1, 1, 0);
END_MACRO;
```

DESKTOP_CLEAR

DESKTOP_CLEAR();

Parameters: None.

Removes all retained MRMAC canvas cells and exposes the native MR desktop background again. The current desktop drawing attribute is preserved, so a subsequent character or string operation continues with the attribute selected before the clear. Windows and the virtual-desktop marker remain above the canvas; the menu and FLabel lines remain outside its projection bounds.

Example macro.

```
$MACRO REFERENCE_DESKTOP_CLEAR FROM EDIT;
    DESKTOP_SET_COLOR(31);
    DESKTOP_PUT_STRING('temporary text', 1, 1);
    DESKTOP_CLEAR();
    DESKTOP_PUT_STRING('attribute retained', 1, 1);
END_MACRO;
```

DESKTOP_PUT_CHAR

```
DESKTOP_PUT_CHAR(character, column, row);
```

Parameters:

character Character value or a string containing exactly one byte.

column One-based column in the MR desktop background.

row One-based row in the MR desktop background.

Writes one character cell using the current attribute selected by `DESKTOP_SET_COLOR`. An out-of-bounds cell is clipped. The retained cell is redrawn after expose, resize and virtual-desktop changes without covering windows, the menu or the FLabel line.¹

Example macro.

```
$MACRO REFERENCE_DESKTOP_PUT_CHAR FROM EDIT;
    DESKTOP_SET_COLOR(30);
    DESKTOP_PUT_CHAR('*', 10, 4);
END_MACRO;
```

DESKTOP_PUT_STRING

```
DESKTOP_PUT_STRING(text, column, row);
```

Parameters:

text String-like sequence of character bytes.

column One-based starting column in the MR desktop background.

row One-based row in the MR desktop background.

Writes the bytes of text from left to right using the current desktop drawing attribute. Text is clipped to the desktop background. The retained cells are shared by all virtual desktops and do not extend into the menu or FLabel line.

Example macro.

```
$MACRO REFERENCE_DESKTOP_PUT_STRING FROM EDIT;
    DESKTOP_SET_COLOR(31);
    DESKTOP_PUT_STRING('MR desktop', 3, 2);
END_MACRO;
```

DESKTOP_SET_COLOR

```
DESKTOP_SET_COLOR(attribute);
```

Parameters:

attribute Complete TVision character attribute in the range 0 through 255.

Selects the attribute used by subsequent `DESKTOP_PUT_CHAR` and `DESKTOP_PUT_STRING` operations in applied command order. The attribute and desktop cells are volatile runtime state and are not persisted. `DESKTOP_BLIT` resolves an attribute for every cell and does not change this value.²

Example macro.

```
$MACRO REFERENCE_DESKTOP_SET_COLOR FROM EDIT;
    DESKTOP_SET_COLOR(31);
    DESKTOP_PUT_STRING('coloured text', 1, 1);
END_MACRO;
```

¹The historical MEMAC `DRAW_CHAR` procedure is not available in MRMAC.

²The historical MEMAC `DRAW_ATTR` procedure is not available in MRMAC.

CMD_TO_KEY

```
CMD_TO_KEY(keySpec, commandId, mode);
```

Parameters:

keySpec MRMAC key specification.

commandId MR application command identifier.

mode Editor mode constant.

Binds an MR command id to keySpec in mode.

Example macro.

```
$MACRO REFERENCE_CMD_TO_KEY FROM EDIT;  
    CMD_TO_KEY('reference', 0, 0);  
END_MACRO;
```

FLABEL

```
FLABEL(label, keyCode, mode);
```

Parameters:

label User-visible field label.

keyCode Integer key code.

mode Editor mode constant.

Assigns a function-label text to a key and mode.

Example macro.

```
$MACRO REFERENCE_FLABEL FROM EDIT;  
    FLABEL('reference', 0, 0);  
END_MACRO;
```

GOTOXY

```
GOTOXY(column, row);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

Moves the staged screen cursor.

Example macro.

```
$MACRO REFERENCE_GOTOXY FROM EDIT;  
    GOTOXY(0, 0);  
END_MACRO;
```

KEY_IN

```
KEY_IN(keySpec);
```

Parameters:

keySpec MRMAC key specification.

Tests the supplied key specification against current input.

Example macro.

```
$MACRO REFERENCE_KEY_IN FROM EDIT;
    KEY_IN('reference');
END_MACRO;
```

KEY_RECORD

```
KEY_RECORD;
```

Parameters: None.

Toggles macro-key recording.

Example macro.

```
$MACRO REFERENCE_KEY_RECORD FROM EDIT;
    KEY_RECORD;
END_MACRO;
```

KILL_BOX

```
KILL_BOX;
```

Parameters: None.

Removes the current staged screen box.

Example macro.

```
$MACRO REFERENCE_KILL_BOX FROM EDIT;
    KILL_BOX;
END_MACRO;
```

MACRO_TO_KEY

```
MACRO_TO_KEY(keySpec, macroSpec, mode);
```

Parameters:

keySpec MRMAC key specification.

macroSpec Macro specification or callback target.

mode Editor mode constant.

Binds macroSpec to keySpec in mode.

Example macro.

```
$MACRO REFERENCE_MACRO_TO_KEY FROM EDIT;
    MACRO_TO_KEY('reference', 'reference', 0);
END_MACRO;
```

MAKE_MESSAGE

```
MAKE_MESSAGE(message);
```

Parameters:

message String-like user-visible message.

Writes a message through the MR message-line path.

Example macro.

```
$MACRO REFERENCE_MAKE_MESSAGE FROM EDIT;  
    MAKE_MESSAGE('reference');  
END_MACRO;
```

MARQUEE

```
MARQUEE(message);
```

Parameters:

message String-like user-visible message.

Displays an informational marquee message.

Example macro.

```
$MACRO REFERENCE_MARQUEE FROM EDIT;  
    MARQUEE('reference');  
END_MACRO;
```

MARQUEE_ERROR

```
MARQUEE_ERROR(message);
```

Parameters:

message String-like user-visible message.

Displays an error marquee message.

Example macro.

```
$MACRO REFERENCE_MARQUEE_ERROR FROM EDIT;  
    MARQUEE_ERROR('reference');  
END_MACRO;
```

MARQUEE_WARNING

```
MARQUEE_WARNING(message);
```

Parameters:

message String-like user-visible message.

Displays a warning marquee message.

Example macro.

```
$MACRO REFERENCE_MARQUEE_WARNING FROM EDIT;  
    MARQUEE_WARNING('reference');  
END_MACRO;
```

PASS_KEY

```
PASS_KEY(keyCode, keyFlags);
```

Parameters:

keyCode Integer key code.

keyFlags Integer key modifier mask.

Passes a key with its modifier mask to the editor.

Example macro.

```
$MACRO REFERENCE_PASS_KEY FROM EDIT;  
    PASS_KEY(0, 0);  
END_MACRO;
```

PLAY_KEY_MACRO

```
PLAY_KEY_MACRO(keyCode, repeatCount);  
PLAY_KEY_MACRO(keyCode, repeatCount, mode);
```

Parameters:

keyCode Integer key code.

repeatCount Number of key macro repetitions.

Replays a recorded key macro.

Example macro.

```
$MACRO REFERENCE_PLAY_KEY_MACRO FROM EDIT;  
    PLAY_KEY_MACRO(0, 0);  
END_MACRO;
```

POP_LABELS

```
POP_LABELS;
```

Parameters: None.

Restores the saved function-label state.

Example macro.

```
$MACRO REFERENCE_POP_LABELS FROM EDIT;  
    POP_LABELS;  
END_MACRO;
```

PUSH_KEY

```
PUSH_KEY(keyCode, keyFlags);
```

Parameters:

keyCode Integer key code.

keyFlags Integer key modifier mask.

Queues a key with its modifier mask.

Example macro.

```
$MACRO REFERENCE_PUSH_KEY FROM EDIT;  
    PUSH_KEY(0, 0);  
END_MACRO;
```

PUSH_LABELS

```
PUSH_LABELS;
```

Parameters: None.

Saves the current function-label state.

Example macro.

```
$MACRO REFERENCE_PUSH_LABELS FROM EDIT;  
    PUSH_LABELS;  
END_MACRO;
```

PUT_BOX

```
PUT_BOX(left, top, right, bottom, foreground, background, text, border);
```

Parameters:

left Left screen column.

top Top screen row.

right Right screen column.

bottom Bottom screen row.

foreground Foreground colour code.

background Background colour code.

text String-like text.

border Integer border style or attribute.

Draws a staged text box at the supplied screen bounds.

Example macro.

```
$MACRO REFERENCE_PUT_BOX FROM EDIT;  
    PUT_BOX(0, 0, 0, 0, 0, 0, 'reference', 0);  
END_MACRO;
```

PUT_COL_NUM

```
PUT_COL_NUM(column);
```

Parameters:

column Screen or editor column, as required by the primitive.

Displays the supplied column number.

Example macro.

```
$MACRO REFERENCE_PUT_COL_NUM FROM EDIT;  
    PUT_COL_NUM(0);  
END_MACRO;
```

PUT_LINE_NUM

```
PUT_LINE_NUM(line);
```

Parameters:

line One-based editor line number.

Displays the supplied line number.

Example macro.

```
$MACRO REFERENCE_PUT_LINE_NUM FROM EDIT;  
    PUT_LINE_NUM(0);  
END_MACRO;
```

READ_KEY

READ_KEY;

Parameters: None.

Waits for and dispatches the next input key in the active interaction context.

Example macro.

```
$MACRO REFERENCE_READ_KEY FROM EDIT;
    READ_KEY;
END_MACRO;
```

REGISTER_MENU_ITEM

```
REGISTER_MENU_ITEM(menuName, itemLabel);
REGISTER_MENU_ITEM(menuName, itemLabel, macroSpec);
```

Parameters:

menuName Named menu definition.

itemLabel String or character positional argument.

macroSpec Macro specification or callback target.

Registers an owned runtime menu item and optional macro callback.

Example macro.

```
$MACRO REFERENCE_REGISTER_MENU_ITEM FROM EDIT;
    REGISTER_MENU_ITEM('reference', 'reference', 'reference');
END_MACRO;
```

REMOVE_MENU_ITEM

```
REMOVE_MENU_ITEM(menuName, itemLabel);
```

Parameters:

menuName Named menu definition.

itemLabel String or character positional argument.

Removes an owned runtime menu item.

Example macro.

```
$MACRO REFERENCE_REMOVE_MENU_ITEM FROM EDIT;
    REMOVE_MENU_ITEM('reference', 'reference');
END_MACRO;
```

SCROLL_BOX_DN

```
SCROLL_BOX_DN(left, top, right, bottom, attribute);
```

Parameters:

left Left screen column.

top Top screen row.

right Right screen column.

bottom Bottom screen row.

attribute Screen attribute code.

Scrolls the supplied staged screen rectangle down.

Example macro.

```
$MACRO REFERENCE_SCROLL_BOX_DN FROM EDIT;  
    SCROLL_BOX_DN(0, 0, 0, 0, 0);  
END_MACRO;
```

SCROLL_BOX_UP

```
SCROLL_BOX_UP(left, top, right, bottom, attribute);
```

Parameters:

left Left screen column.

top Top screen row.

right Right screen column.

bottom Bottom screen row.

attribute Screen attribute code.

Scrolls the supplied staged screen rectangle up.

Example macro.

```
$MACRO REFERENCE_SCROLL_BOX_UP FROM EDIT;  
    SCROLL_BOX_UP(0, 0, 0, 0, 0);  
END_MACRO;
```

SET_CLIPBOARD_TEXT

```
SET_CLIPBOARD_TEXT(text);
```

Parameters:

text String-like text.

Replaces system clipboard text.

Example macro.

```
$MACRO REFERENCE_SET_CLIPBOARD_TEXT FROM EDIT;  
    SET_CLIPBOARD_TEXT('reference');  
END_MACRO;
```

UNASSIGN_ALL_KEYS

```
UNASSIGN_ALL_KEYS;
```

Parameters: None.

Removes all active macro key bindings.

Example macro.

```
$MACRO REFERENCE_UNASSIGN_ALL_KEYS FROM EDIT;  
    UNASSIGN_ALL_KEYS;  
END_MACRO;
```

UNASSIGN_KEY

```
UNASSIGN_KEY(keySpec, mode);
```

Parameters:

keySpec MRMAC key specification.

mode Editor mode constant.

Removes the binding for keySpec in mode.

Example macro.

```
$MACRO REFERENCE_UNASSIGN_KEY FROM EDIT;  
    UNASSIGN_KEY('reference', 0);  
END_MACRO;
```

WORKING

```
WORKING;
```

Parameters: None.

Shows the standard working indication.

Example macro.

```
$MACRO REFERENCE_WORKING FROM EDIT;  
    WORKING;  
END_MACRO;
```

WRITE

```
WRITE(text, column, row, foreground, background);
```

Parameters:

text String-like text.

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

foreground Foreground colour code.

background Background colour code.

Writes staged screen text with foreground and background attributes.

Example macro.

```
$MACRO REFERENCE_WRITE FROM EDIT;  
    WRITE('reference', 0, 0, 0, 0);  
END_MACRO;
```

Chapter 10

Typed Dialogs and Collections

Typed dialogs are built from explicit controls and named collections. A macro declares a dialog, adds controls, fills list, tree or table data, then executes it. The resulting control id and values remain ordinary MRMAC values.

10.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
UI_BUTTON	procedure	Dialog construction.	p. 119
UI_DIALOG	procedure	Dialog construction.	p. 120
UI_DISPLAY	procedure	Dialog construction.	p. 120
UI_EXEC	intrinsic	Dialog construction.	p. 120
UI_GRID	procedure	Dialog construction.	p. 121
UI_INDEX	intrinsic	Dialog construction.	p. 121
UI_INPUT	procedure	Dialog construction.	p. 121
UI_LABEL	procedure	Dialog construction.	p. 122
UI_LISTBOX	procedure	Dialog construction.	p. 122
UI_LIST_ADD	procedure	Dialog construction.	p. 122
UI_LIST_CLEAR	procedure	Dialog construction.	p. 123
UI_MESSAGEBOX	procedure	Dialog construction.	p. 123
UI_TABLE	procedure	Dialog construction.	p. 123
UI_TABLE_CLEAR	procedure	Dialog construction.	p. 123
UI_TABLE_COLUMN	procedure	Dialog construction.	p. 124
UI_TABLE_ROW	procedure	Dialog construction.	p. 124
UI_TEXT	intrinsic	Dialog construction.	p. 124
UI_TREE	procedure	Dialog construction.	p. 125
UI_TREE_CLEAR	procedure	Dialog construction.	p. 125
UI_TREE_NODE	procedure	Dialog construction.	p. 125

10.2 Reference Entries

UI_BUTTON

```
UI_BUTTON(column, row, width, id, text);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

id Typed-dialog control identifier.

text String-like text.

Adds a typed dialog button with the supplied control id.

Example macro.

```
$MACRO REFERENCE_UI_BUTTON FROM EDIT;
    UI_BUTTON(0, 0, 0, 0, 'reference');
END_MACRO;
```

UI_DIALOG

```
UI_DIALOG(left, top, width, height, title);
```

Parameters:

left Left screen column.

top Top screen row.

width Control or region width.

height Control or region height.

title User-visible title.

Starts a typed dialog definition at the supplied rectangle and gives it title.

Example macro.

```
$MACRO REFERENCE_UI_DIALOG FROM EDIT;
    UI_DIALOG(0, 0, 0, 0, 'reference');
END_MACRO;
```

UI_DISPLAY

```
UI_DISPLAY(column, row, width, text);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

text String-like text.

Adds a read-only display field to the current typed dialog.

Example macro.

```
$MACRO REFERENCE_UI_DISPLAY FROM EDIT;
    UI_DISPLAY(0, 0, 0, 'reference');
END_MACRO;
```

UI_EXEC

```
UI_EXEC
```

Parameters: None.

Executes the current typed dialog and returns its selected control id.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_UI_EXEC FROM EDIT;
    DEF_INT(Result);
    Result := UI_EXEC;
END_MACRO;
```

UI_GRID

```
UI_GRID(column, row, width, height, id, tableName, initialValue, flags);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

id Typed-dialog control identifier.

tableName Named table definition.

initialValue Initial selected value.

flags Integer behaviour flags defined for the control.

Adds a typed grid control backed by tableName.

Example macro.

```
$MACRO REFERENCE_UI_GRID FROM EDIT;
    UI_GRID(0, 0, 0, 0, 0, 'reference', 'reference', 0);
END_MACRO;
```

UI_INDEX

```
UI_INDEX(controlId)
```

Parameters:

controlId Typed-dialog control identifier.

Returns the selection index of a typed-dialog control.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_UI_INDEX FROM EDIT;
    DEF_INT(Result);
    Result := UI_INDEX(0);
END_MACRO;
```

UI_INPUT

```
UI_INPUT(column, row, width, id, fieldName, initialText);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

id Typed-dialog control identifier.

fieldName Named retained field.

initialText Initial text value.

Adds a named editable text field to the current typed dialog.

Example macro.

```
$MACRO REFERENCE_UI_INPUT FROM EDIT;
    UI_INPUT(0, 0, 0, 0, 'reference', 'reference');
END_MACRO;
```

UI_LABEL

```
UI_LABEL(column, row, text);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

text String-like text.

Adds static label text to the current typed dialog.

Example macro.

```
$MACRO REFERENCE_UI_LABEL FROM EDIT;
    UI_LABEL(0, 0, 'reference');
END_MACRO;
```

UI_LISTBOX

```
UI_LISTBOX(column, row, width, height, id, listName, initialValue, flags);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

id Typed-dialog control identifier.

listName Named list definition.

initialValue Initial selected value.

flags Integer behaviour flags defined for the control.

Adds a typed list-box control backed by listName.

Example macro.

```
$MACRO REFERENCE_UI_LISTBOX FROM EDIT;
    UI_LISTBOX(0, 0, 0, 0, 0, 'reference', 'reference', 0);
END_MACRO;
```

UI_LIST_ADD

```
UI_LIST_ADD(listName, value);
```

Parameters:

listName Named list definition.

value Value written or selected by the primitive.

Appends one selectable value to the named typed-dialog list.

Example macro.

```
$MACRO REFERENCE_UI_LIST_ADD FROM EDIT;  
    UI_LIST_ADD('reference', 'reference');  
END_MACRO;
```

UI_LIST_CLEAR

```
UI_LIST_CLEAR(listName);
```

Parameters:

listName Named list definition.

Clears all values in the named typed-dialog list.

Example macro.

```
$MACRO REFERENCE_UI_LIST_CLEAR FROM EDIT;  
    UI_LIST_CLEAR('reference');  
END_MACRO;
```

UI_MESSAGEBOX

```
UI_MESSAGEBOX(message);
```

Parameters:

message String-like user-visible message.

Shows a typed modal message box.

Example macro.

```
$MACRO REFERENCE_UI_MESSAGEBOX FROM EDIT;  
    UI_MESSAGEBOX('reference');  
END_MACRO;
```

UI_TABLE

```
UI_TABLE(column, row, width, height, id, tableName, initialValue, flags);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

id Typed-dialog control identifier.

tableName Named table definition.

initialValue Initial selected value.

flags Integer behaviour flags defined for the control.

Adds a typed table control backed by tableName.

Example macro.

```
$MACRO REFERENCE_UI_TABLE FROM EDIT;  
    UI_TABLE(0, 0, 0, 0, 0, 'reference', 'reference', 0);  
END_MACRO;
```

UI_TABLE_CLEAR

```
UI_TABLE_CLEAR(tableName);
```

Parameters:

tableName Named table definition.

Clears all rows and columns of the named typed-dialog table.

Example macro.

```
$MACRO REFERENCE_UI_TABLE_CLEAR FROM EDIT;  
    UI_TABLE_CLEAR('reference');  
END_MACRO;
```

UI_TABLE_COLUMN

```
UI_TABLE_COLUMN(tableName, columnName, width);
```

Parameters:

tableName Named table definition.

columnName String or character positional argument.

width Control or region width.

Adds one named column to the typed-dialog table.

Example macro.

```
$MACRO REFERENCE_UI_TABLE_COLUMN FROM EDIT;  
    UI_TABLE_COLUMN('reference', 'reference', 0);  
END_MACRO;
```

UI_TABLE_ROW

```
UI_TABLE_ROW(tableName, rowId, values);
```

Parameters:

tableName Named table definition.

rowId String-like stable row identifier.

values String-like encoded cell values for the row.

Adds or replaces one identified typed-dialog table row.

Example macro.

```
$MACRO REFERENCE_UI_TABLE_ROW FROM EDIT;  
    UI_TABLE_ROW('reference', 'reference', 'reference');  
END_MACRO;
```

UI_TEXT

```
UI_TEXT(controlId)
```

Parameters:

controlId Typed-dialog control identifier.

Returns the text value of a typed-dialog control.
Returns a string.

Example macro.

```
$MACRO REFERENCE_UI_TEXT FROM EDIT;  
    DEF_STR(Result);  
    Result := UI_TEXT(0);  
END_MACRO;
```

UI_TREE

```
UI_TREE(column, row, width, height, id, treeName, initialValue, flags);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

id Typed-dialog control identifier.

treeName Named tree definition.

initialValue Initial selected value.

flags Integer behaviour flags defined for the control.

Adds a typed tree control backed by treeName.

Example macro.

```
$MACRO REFERENCE_UI_TREE FROM EDIT;  
    UI_TREE(0, 0, 0, 0, 0, 'reference', 'reference', 0);  
END_MACRO;
```

UI_TREE_CLEAR

```
UI_TREE_CLEAR(treeName);
```

Parameters:

treeName Named tree definition.

Clears the named typed-dialog tree data source.

Example macro.

```
$MACRO REFERENCE_UI_TREE_CLEAR FROM EDIT;  
    UI_TREE_CLEAR('reference');  
END_MACRO;
```

UI_TREE_NODE

```
UI_TREE_NODE(treeName, nodeId, parentId, text, flags);
```

Parameters:

treeName Named tree definition.

nodeId String-like tree-node identifier.

parentId String-like parent tree-node identifier.

text String-like text.

flags Integer behaviour flags defined for the control.

Adds or replaces one typed-dialog tree node.

Example macro.

```
$MACRO REFERENCE_UI_TREE_NODE FROM EDIT;  
    UI_TREE_NODE('reference', 'reference', 'reference', 'reference', 0);  
END_MACRO;
```

Chapter 11

Modeless MMP Interfaces

Modeless MRMAC Primitives, abbreviated MMP, build retained modeless interfaces from typed MRMAC values. A macro declares controls, shows a named window, and later updates its retained fields. MMP never exposes a native view, pointer or event object.

11.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
UI_MODELESS_CLOSE	procedure	Modeless UI.	p. 128
UI_MODELESS_DISPLAY	procedure	Modeless UI.	p. 128
UI_MODELESS_ON	procedure	Modeless UI.	p. 129
UI_MODELESS_SHOW	procedure	Modeless UI.	p. 129
UI_MODELESS_UPDATE	procedure	Modeless UI.	p. 129
MMP_ACTION_BUTTON	procedure	Modeless UI.	p. 129
MMP_ACTION_MENU	procedure	Modeless UI.	p. 130
MMP_BOOL_FIELD	procedure	Modeless UI.	p. 130
MMP_BOOL_SET	procedure	Modeless UI.	p. 131
MMP_BOOL_VALUE	intrinsic	Modeless UI.	p. 131
MMP_CANVAS	procedure	Modeless UI.	p. 131
MMP_CANVAS_BOX	procedure	Modeless UI.	p. 132
MMP_CANVAS_CLEAR	procedure	Modeless UI.	p. 132
MMP_CANVAS_COMMIT	procedure	Modeless UI.	p. 132
MMP_CANVAS_FILL	procedure	Modeless UI.	p. 133
MMP_CANVAS_GLYPH	procedure	Modeless UI.	p. 133
MMP_CANVAS_HOTSPOT	procedure	Modeless UI.	p. 133
MMP_CANVAS_LINE	procedure	Modeless UI.	p. 134
MMP_CANVAS_TEXT	procedure	Modeless UI.	p. 134
MMP_INT_FIELD	procedure	Modeless UI.	p. 135
MMP_INT_SET	procedure	Modeless UI.	p. 135
MMP_INT_VALUE	intrinsic	Modeless UI.	p. 135
MMP_LOG_APPEND	procedure	Modeless UI.	p. 136
MMP_LOG_CLEAR	procedure	Modeless UI.	p. 136
MMP_LOG_COUNT	intrinsic	Modeless UI.	p. 136
MMP_LOG_FIELD	procedure	Modeless UI.	p. 136
MMP_MENU_CLEAR	procedure	Modeless UI.	p. 137
MMP_MENU_ITEM	procedure	Modeless UI.	p. 137
MMP_PROGRESS_FIELD	procedure	Modeless UI.	p. 137
MMP_PROGRESS_SET	procedure	Modeless UI.	p. 138

Primitive	Role	Purpose	Reference
MMP_PROGRESS_VALUE	intrinsic	Modeless UI.	p. 138
MMP_SELECT_FIELD	procedure	Modeless UI.	p. 138
MMP_SELECT_OPTION	procedure	Modeless UI.	p. 139
MMP_SELECT_SET	procedure	Modeless UI.	p. 139
MMP_SELECT_VALUE	intrinsic	Modeless UI.	p. 139
MMP_STATUS_FIELD	procedure	Modeless UI.	p. 140
MMP_STATUS_SET	procedure	Modeless UI.	p. 140
MMP_STYLE_ACCENT	intrinsic	Modeless UI.	p. 140
MMP_STYLE_MUTED	intrinsic	Modeless UI.	p. 141
MMP_STYLE_SURFACE	intrinsic	Modeless UI.	p. 141
MMP_STYLE_TEXT	intrinsic	Modeless UI.	p. 141
MMP_TEXT_FIELD	procedure	Modeless UI.	p. 141
MMP_TEXT_SET	procedure	Modeless UI.	p. 142
MMP_TEXT_VALUE	intrinsic	Modeless UI.	p. 142
MMP_TIMER_START	procedure	Modeless UI.	p. 142
MMP_TIMER_STOP	procedure	Modeless UI.	p. 143
MMP_WINDOW_EXISTS	intrinsic	Modeless UI.	p. 143
MMP_WINDOW_HEIGHT	intrinsic	Modeless UI.	p. 143
MMP_WINDOW_INSTANCE	intrinsic	Modeless UI.	p. 144
MMP_WINDOW_WIDTH	intrinsic	Modeless UI.	p. 144
MMP_WINDOW_X	intrinsic	Modeless UI.	p. 144
MMP_WINDOW_Y	intrinsic	Modeless UI.	p. 144

11.2 Reference Entries

UI_MODELESS_CLOSE

```
UI_MODELESS_CLOSE(windowName);
```

Parameters:

windowName Logical modeless window name.

Requests that the named modeless window close through the UI event path.

Example macro.

```
$MACRO REFERENCE_UI_MODELESS_CLOSE FROM EDIT;
    UI_MODELESS_CLOSE('reference');
END_MACRO;
```

UI_MODELESS_DISPLAY

```
UI_MODELESS_DISPLAY(windowName, displayId, text);
```

Parameters:

windowName Logical modeless window name.

displayId Integer retained display-line identifier.

text String-like text.

Replaces one retained modeless display line.

Example macro.

```
$MACRO REFERENCE_UI_MODELESS_DISPLAY FROM EDIT;  
    UI_MODELESS_DISPLAY('reference', 0, 'reference');  
END_MACRO;
```

UI__MODELESS__ON

```
UI_MODELESS_ON(controlId, macroSpec);
```

Parameters:

controlId Typed-dialog control identifier.

macroSpec Macro specification or callback target.

Binds a typed-dialog control id to a macro callback.

Example macro.

```
$MACRO REFERENCE_UI_MODELESS_ON FROM EDIT;  
    UI_MODELESS_ON(0, 'reference');  
END_MACRO;
```

UI__MODELESS__SHOW

```
UI_MODELESS_SHOW(windowName);
```

Parameters:

windowName Logical modeless window name.

Shows the named retained modeless window.

Example macro.

```
$MACRO REFERENCE_UI_MODELESS_SHOW FROM EDIT;  
    UI_MODELESS_SHOW('reference');  
END_MACRO;
```

UI__MODELESS__UPDATE

```
UI_MODELESS_UPDATE(windowName);
```

Parameters:

windowName Logical modeless window name.

Requests a retained modeless window refresh without taking focus.

Example macro.

```
$MACRO REFERENCE_UI_MODELESS_UPDATE FROM EDIT;  
    UI_MODELESS_UPDATE('reference');  
END_MACRO;
```

MMP_ACTION_BUTTON

```
MMP_ACTION_BUTTON(column, row, width, height, label, macroSpec);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

label User-visible field label.

macroSpec Macro specification or callback target.

Adds a modeless action button and its macro callback.

Example macro.

```
$MACRO REFERENCE_MMP_ACTION_BUTTON FROM EDIT;
    MMP_ACTION_BUTTON(0, 0, 0, 0, 'reference', 'reference');
END_MACRO;
```

MMP_ACTION_MENU

```
MMP_ACTION_MENU(column, row, width, height, id, menuName, initialValue, macroSpec);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

id Typed-dialog control identifier.

menuName Named menu definition.

initialValue Initial selected value.

macroSpec Macro specification or callback target.

Adds a modeless action menu backed by menuName.

Example macro.

```
$MACRO REFERENCE_MMP_ACTION_MENU FROM EDIT;
    MMP_ACTION_MENU(0, 0, 0, 0, 0, 'reference', 'reference', 'reference');
END_MACRO;
```

MMP_BOOL_FIELD

```
MMP_BOOL_FIELD(column, row, fieldName, label, initialValue);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

fieldName Named retained field.

label User-visible field label.

initialValue Initial selected value.

Adds a retained named boolean field.

Example macro.

```
$MACRO REFERENCE_MMP_BOOL_FIELD FROM EDIT;  
    MMP_BOOL_FIELD(0, 0, 'reference', 'reference', 0);  
END_MACRO;
```

MMP_BOOL_SET

```
MMP_BOOL_SET(windowName, fieldName, value);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

value Value written or selected by the primitive.

Writes a retained modeless boolean field.

Example macro.

```
$MACRO REFERENCE_MMP_BOOL_SET FROM EDIT;  
    MMP_BOOL_SET('reference', 'reference', 0);  
END_MACRO;
```

MMP_BOOL_VALUE

```
MMP_BOOL_VALUE(windowName, fieldName)
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Reads the retained boolean value of a modeless field.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_BOOL_VALUE FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_BOOL_VALUE('reference', 'reference');  
END_MACRO;
```

MMP_CANVAS

```
MMP_CANVAS(left, top, width, height, canvasName);
```

Parameters:

left Left screen column.

top Top screen row.

width Control or region width.

height Control or region height.

canvasName Named retained canvas.

Adds a named retained canvas to the current modeless definition.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS FROM EDIT;
    MMP_CANVAS(0, 0, 0, 0, 'reference');
END_MACRO;
```

MMP_CANVAS_BOX

```
MMP_CANVAS_BOX(windowName, canvasName, left, top, width, height, style);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

left Left screen column.

top Top screen row.

width Control or region width.

height Control or region height.

style MRMAC style or attribute code.

Draws a retained outlined canvas rectangle.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_BOX FROM EDIT;
    MMP_CANVAS_BOX('reference', 'reference', 0, 0, 0, 0, 0);
END_MACRO;
```

MMP_CANVAS_CLEAR

```
MMP_CANVAS_CLEAR(windowName, canvasName, style);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

style MRMAC style or attribute code.

Clears the retained scene of the named canvas using style.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_CLEAR FROM EDIT;
    MMP_CANVAS_CLEAR('reference', 'reference', 0);
END_MACRO;
```

MMP_CANVAS_COMMIT

```
MMP_CANVAS_COMMIT(windowName, canvasName);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

Redraws only the named retained canvas.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_COMMIT FROM EDIT;  
    MMP_CANVAS_COMMIT('reference', 'reference');  
END_MACRO;
```

MMP_CANVAS_FILL

```
MMP_CANVAS_FILL(windowName, canvasName, left, top, width, height, style);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

left Left screen column.

top Top screen row.

width Control or region width.

height Control or region height.

style MRMAC style or attribute code.

Fills a retained canvas rectangle.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_FILL FROM EDIT;  
    MMP_CANVAS_FILL('reference', 'reference', 0, 0, 0, 0, 0);  
END_MACRO;
```

MMP_CANVAS_GLYPH

```
MMP_CANVAS_GLYPH(windowName, canvasName, column, row, style, glyph);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

style MRMAC style or attribute code.

glyph String-like glyph text used by the retained canvas.

Places one retained glyph at a canvas coordinate.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_GLYPH FROM EDIT;  
    MMP_CANVAS_GLYPH('reference', 'reference', 0, 0, 0, 'reference');  
END_MACRO;
```

MMP_CANVAS_HOTSPOT

```
MMP_CANVAS_HOTSPOT(windowName, left, top, width, height, button, macroSpec);
```

Parameters:

windowName Logical modeless window name.

left Left screen column.

top Top screen row.

width Control or region width.

height Control or region height.

button Integer mouse-button identifier.

macroSpec Macro specification or callback target.

Registers a canvas-local left-click region and macro callback.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_HOTSPOT FROM EDIT;
    MMP_CANVAS_HOTSPOT('reference', 0, 0, 0, 0, 0, 'reference');
END_MACRO;
```

MMP_CANVAS_LINE

```
MMP_CANVAS_LINE(windowName, canvasName, x1, y1, x2, y2, style, glyph);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

x1 Starting canvas column.

y1 Starting canvas row.

x2 Ending canvas column.

y2 Ending canvas row.

style MRMAC style or attribute code.

glyph String-like glyph text used by the retained canvas.

Draws a retained line between two canvas coordinates.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_LINE FROM EDIT;
    MMP_CANVAS_LINE('reference', 'reference', 0, 0, 0, 0, 0, 'reference');
END_MACRO;
```

MMP_CANVAS_TEXT

```
MMP_CANVAS_TEXT(windowName, canvasName, column, row, style, text);
```

Parameters:

windowName Logical modeless window name.

canvasName Named retained canvas.

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

style MRMAC style or attribute code.

text String-like text.

Places retained text at a canvas coordinate.

Example macro.

```
$MACRO REFERENCE_MMP_CANVAS_TEXT FROM EDIT;  
    MMP_CANVAS_TEXT('reference', 'reference', 0, 0, 0, 'reference');  
END_MACRO;
```

MMP_INT_FIELD

```
MMP_INT_FIELD(column, row, width, fieldName, label, minimum, maximum, initialValue);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

fieldName Named retained field.

label User-visible field label.

minimum Inclusive minimum integer value.

maximum Inclusive maximum integer value.

initialValue Initial selected value.

Adds a range-bounded retained integer field.

Example macro.

```
$MACRO REFERENCE_MMP_INT_FIELD FROM EDIT;  
    MMP_INT_FIELD(0, 0, 0, 'reference', 'reference', 0, 0, 0);  
END_MACRO;
```

MMP_INT_SET

```
MMP_INT_SET(windowName, fieldName, value);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

value Value written or selected by the primitive.

Writes a retained modeless integer field.

Example macro.

```
$MACRO REFERENCE_MMP_INT_SET FROM EDIT;  
    MMP_INT_SET('reference', 'reference', 0);  
END_MACRO;
```

MMP_INT_VALUE

```
MMP_INT_VALUE(windowName, fieldName)
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Reads the retained integer value of a modeless field.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_INT_VALUE FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_INT_VALUE('reference', 'reference');  
END_MACRO;
```

MMP_LOG_APPEND

```
MMP_LOG_APPEND(windowName, fieldName, text);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

text String-like text.

Appends text to a retained modeless log.

Example macro.

```
$MACRO REFERENCE_MMP_LOG_APPEND FROM EDIT;  
    MMP_LOG_APPEND('reference', 'reference', 'reference');  
END_MACRO;
```

MMP_LOG_CLEAR

```
MMP_LOG_CLEAR(windowName, fieldName);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Clears a retained modeless log.

Example macro.

```
$MACRO REFERENCE_MMP_LOG_CLEAR FROM EDIT;  
    MMP_LOG_CLEAR('reference', 'reference');  
END_MACRO;
```

MMP_LOG_COUNT

```
MMP_LOG_COUNT(windowName, fieldName)
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Returns the retained entry count of a modeless log field.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_LOG_COUNT FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_LOG_COUNT('reference', 'reference');  
END_MACRO;
```

MMP_LOG_FIELD

MMP_LOG_FIELD(column, row, width, height, fieldName, label, capacity);

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

fieldName Named retained field.

label User-visible field label.

capacity Maximum retained log entry count.

Adds a bounded retained log field.

Example macro.

```
$MACRO REFERENCE_MMP_LOG_FIELD FROM EDIT;  
    MMP_LOG_FIELD(0, 0, 0, 0, 'reference', 'reference', 0);  
END_MACRO;
```

MMP_MENU_CLEAR

MMP_MENU_CLEAR(menuName);

Parameters:

menuName Named menu definition.

Clears a named retained action menu.

Example macro.

```
$MACRO REFERENCE_MMP_MENU_CLEAR FROM EDIT;  
    MMP_MENU_CLEAR('reference');  
END_MACRO;
```

MMP_MENU_ITEM

MMP_MENU_ITEM(menuName, label, value, detail);

Parameters:

menuName Named menu definition.

label User-visible field label.

value Value written or selected by the primitive.

detail String or character positional argument.

Adds one label, value and detail row to a retained action menu.

Example macro.

```
$MACRO REFERENCE_MMP_MENU_ITEM FROM EDIT;  
    MMP_MENU_ITEM('reference', 'reference', 'reference', 'reference');  
END_MACRO;
```

MMP_PROGRESS_FIELD

```
MMP_PROGRESS_FIELD(column, row, width, fieldName, label, maximum, initialValue);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

fieldName Named retained field.

label User-visible field label.

maximum Inclusive maximum integer value.

initialValue Initial selected value.

Adds a retained progress display with maximum and initial value.

Example macro.

```
$MACRO REFERENCE_MMP_PROGRESS_FIELD FROM EDIT;
    MMP_PROGRESS_FIELD(0, 0, 0, 'reference', 'reference', 0, 0);
END_MACRO;
```

MMP_PROGRESS_SET

```
MMP_PROGRESS_SET(windowName, fieldName, value);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

value Value written or selected by the primitive.

Writes a retained modeless progress value.

Example macro.

```
$MACRO REFERENCE_MMP_PROGRESS_SET FROM EDIT;
    MMP_PROGRESS_SET('reference', 'reference', 0);
END_MACRO;
```

MMP_PROGRESS_VALUE

```
MMP_PROGRESS_VALUE(windowName, fieldName)
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Reads the retained progress value of a modeless field.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_PROGRESS_VALUE FROM EDIT;
    DEF_INT(Result);
    Result := MMP_PROGRESS_VALUE('reference', 'reference');
END_MACRO;
```

MMP_SELECT_FIELD

```
MMP_SELECT_FIELD(column, row, width, height, fieldName, label, initialValue);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

height Control or region height.

fieldName Named retained field.

label User-visible field label.

initialValue Initial selected value.

Adds a retained selection field.

Example macro.

```
$MACRO REFERENCE_MMP_SELECT_FIELD FROM EDIT;  
    MMP_SELECT_FIELD(0, 0, 0, 0, 'reference', 'reference', 'reference');  
END_MACRO;
```

MMP_SELECT_OPTION

```
MMP_SELECT_OPTION(fieldName, value);
```

Parameters:

fieldName Named retained field.

value Value written or selected by the primitive.

Adds one selectable option to a retained selection field.

Example macro.

```
$MACRO REFERENCE_MMP_SELECT_OPTION FROM EDIT;  
    MMP_SELECT_OPTION('reference', 'reference');  
END_MACRO;
```

MMP_SELECT_SET

```
MMP_SELECT_SET(windowName, fieldName, value);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

value Value written or selected by the primitive.

Writes the selected value of a retained selection field.

Example macro.

```
$MACRO REFERENCE_MMP_SELECT_SET FROM EDIT;  
    MMP_SELECT_SET('reference', 'reference', 'reference');  
END_MACRO;
```

MMP_SELECT_VALUE

MMP_SELECT_VALUE(windowName, fieldName)

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Reads the selected value of a modeless selection field.
Returns a string.

Example macro.

```
$MACRO REFERENCE_MMP_SELECT_VALUE FROM EDIT;
    DEF_STR(Result);
    Result := MMP_SELECT_VALUE('reference', 'reference');
END_MACRO;
```

MMP_STATUS_FIELD

MMP_STATUS_FIELD(column, row, width, fieldName, initialText);

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

fieldName Named retained field.

initialText Initial text value.

Adds a retained status line.

Example macro.

```
$MACRO REFERENCE_MMP_STATUS_FIELD FROM EDIT;
    MMP_STATUS_FIELD(0, 0, 0, 'reference', 'reference');
END_MACRO;
```

MMP_STATUS_SET

MMP_STATUS_SET(windowName, fieldName, text);

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

text String-like text.

Writes a retained status line.

Example macro.

```
$MACRO REFERENCE_MMP_STATUS_SET FROM EDIT;
    MMP_STATUS_SET('reference', 'reference', 'reference');
END_MACRO;
```

MMP_STYLE_ACCENT

MMP_STYLE_ACCENT()

Parameters: None.

Returns the MMP accent style code.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_STYLE_ACCENT FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_STYLE_ACCENT();  
END_MACRO;
```

MMP_STYLE_MUTED

MMP_STYLE_MUTED()

Parameters: None.

Returns the MMP muted style code.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_STYLE_MUTED FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_STYLE_MUTED();  
END_MACRO;
```

MMP_STYLE_SURFACE

MMP_STYLE_SURFACE()

Parameters: None.

Returns the MMP surface style code.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_STYLE_SURFACE FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_STYLE_SURFACE();  
END_MACRO;
```

MMP_STYLE_TEXT

MMP_STYLE_TEXT()

Parameters: None.

Returns the MMP text style code.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_STYLE_TEXT FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_STYLE_TEXT();  
END_MACRO;
```

MMP_TEXT_FIELD

```
MMP_TEXT_FIELD(column, row, width, fieldName, label, initialText);
```

Parameters:

column Screen or editor column, as required by the primitive.

row Zero-based screen row.

width Control or region width.

fieldName Named retained field.

label User-visible field label.

initialText Initial text value.

Adds a retained named text field.

Example macro.

```
$MACRO REFERENCE_MMP_TEXT_FIELD FROM EDIT;
    MMP_TEXT_FIELD(0, 0, 0, 'reference', 'reference', 'reference');
END_MACRO;
```

MMP_TEXT_SET

```
MMP_TEXT_SET(windowName, fieldName, text);
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

text String-like text.

Writes a retained modeless text field.

Example macro.

```
$MACRO REFERENCE_MMP_TEXT_SET FROM EDIT;
    MMP_TEXT_SET('reference', 'reference', 'reference');
END_MACRO;
```

MMP_TEXT_VALUE

```
MMP_TEXT_VALUE(windowName, fieldName)
```

Parameters:

windowName Logical modeless window name.

fieldName Named retained field.

Reads the retained text value of a modeless text field.

Returns a string.

Example macro.

```
$MACRO REFERENCE_MMP_TEXT_VALUE FROM EDIT;
    DEF_STR(Result);
    Result := MMP_TEXT_VALUE('reference', 'reference');
END_MACRO;
```

MMP_TIMER_START

```
MMP_TIMER_START(windowName, timerName, milliseconds, macroSpec);
```

Parameters:

windowName Logical modeless window name.

timerName String or character positional argument.

milliseconds Positive delay or timer interval in milliseconds.

macroSpec Macro specification or callback target.

Starts a named logical modeless window timer and callback.

Example macro.

```
$MACRO REFERENCE_MMP_TIMER_START FROM EDIT;  
    MMP_TIMER_START('reference', 'reference', 0, 'reference');  
END_MACRO;
```

MMP_TIMER_STOP

```
MMP_TIMER_STOP(windowName, timerName);
```

Parameters:

windowName Logical modeless window name.

timerName String or character positional argument.

Stops a named logical modeless window timer.

Example macro.

```
$MACRO REFERENCE_MMP_TIMER_STOP FROM EDIT;  
    MMP_TIMER_STOP('reference', 'reference');  
END_MACRO;
```

MMP_WINDOW_EXISTS

```
MMP_WINDOW_EXISTS(windowName)
```

Parameters:

windowName Logical modeless window name.

Reports whether a named logical modeless window exists.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_EXISTS FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_WINDOW_EXISTS('reference');  
END_MACRO;
```

MMP_WINDOW_HEIGHT

```
MMP_WINDOW_HEIGHT(windowName)
```

Parameters:

windowName Logical modeless window name.

Returns the logical height of the named modeless window.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_HEIGHT FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_WINDOW_HEIGHT('reference');  
END_MACRO;
```

MMP_WINDOW_INSTANCE

MMP_WINDOW_INSTANCE(windowName)

Parameters:

windowName Logical modeless window name.

Returns the current instance identity of the named modeless window.
Returns a string.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_INSTANCE FROM EDIT;  
    DEF_STR(Result);  
    Result := MMP_WINDOW_INSTANCE('reference');  
END_MACRO;
```

MMP_WINDOW_WIDTH

MMP_WINDOW_WIDTH(windowName)

Parameters:

windowName Logical modeless window name.

Returns the logical width of the named modeless window.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_WIDTH FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_WINDOW_WIDTH('reference');  
END_MACRO;
```

MMP_WINDOW_X

MMP_WINDOW_X(windowName)

Parameters:

windowName Logical modeless window name.

Returns the logical x coordinate of the named modeless window.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_X FROM EDIT;  
    DEF_INT(Result);  
    Result := MMP_WINDOW_X('reference');  
END_MACRO;
```

MMP_WINDOW_Y

MMP_WINDOW_Y(windowName)

Parameters:

windowName Logical modeless window name.

Returns the logical y coordinate of the named modeless window.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_MMP_WINDOW_Y FROM EDIT;  
  DEF_INT(Result);  
  Result := MMP_WINDOW_Y('reference');  
END_MACRO;
```

Part IV

Runtime and Integration

Chapter 12

Globals, Catalogs and Execution Sessions

Local declarations belong to one running macro. Explicit macro globals, the loaded macro catalogue and execution sessions are different runtime concepts. This chapter covers their controlled primitive interfaces and enumerators.

12.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
FIRST_GLOBAL	intrinsic	Macro runtime state.	p. 147
NEXT_GLOBAL	intrinsic	Macro runtime state.	p. 148
GLOBAL_HASH	intrinsic	Macro runtime state.	p. 148
GLOBAL_INT	intrinsic	Macro runtime state.	p. 148
GLOBAL_STR	intrinsic	Macro runtime state.	p. 148
INQ_MACRO	intrinsic	Macro runtime state.	p. 149
CREATE_GLOBAL_STR	procedure	Macro runtime state.	p. 149
DELAY	procedure	Macro runtime state.	p. 149
EXEC_ASSIGN	procedure	Macro runtime state.	p. 150
EXEC_SESSION_LIST	procedure	Macro runtime state.	p. 150
EXEC_SESSION_STOP	procedure	Macro runtime state.	p. 150
LOAD_MACRO_FILE	procedure	Macro runtime state.	p. 150
RUN_MACRO	procedure	Macro runtime state.	p. 151
SET_GLOBAL_HASH	procedure	Macro runtime state.	p. 151
SET_GLOBAL_INT	procedure	Macro runtime state.	p. 151
SET_GLOBAL_STR	procedure	Macro runtime state.	p. 151
UNLOAD_MACRO	procedure	Macro runtime state.	p. 152

12.2 Reference Entries

FIRST_GLOBAL

`FIRST_GLOBAL(cursor)`

Parameters:

cursor Integer enumeration cursor variable.

Starts global-name enumeration with cursor and returns the first name.
Returns a string.

Example macro.

```
$MACRO REFERENCE_FIRST_GLOBAL FROM EDIT;  
    DEF_INT(Cursor);  
    DEF_STR(Result);  
    Result := FIRST_GLOBAL(Cursor);  
END_MACRO;
```

NEXT_GLOBAL

NEXT_GLOBAL(cursor)

Parameters:

cursor Integer enumeration cursor variable.

Continues global-name enumeration with cursor and returns the next name.
Returns a string.

Example macro.

```
$MACRO REFERENCE_NEXT_GLOBAL FROM EDIT;  
    DEF_INT(Cursor);  
    DEF_STR(Result);  
    Result := NEXT_GLOBAL(Cursor);  
END_MACRO;
```

GLOBAL_HASH

GLOBAL_HASH(name)

Parameters:

name String-like global name.

Reads a named hash global.
Returns a hash.

Example macro.

```
$MACRO REFERENCE_GLOBAL_HASH FROM EDIT;  
    DEF_HASH(State);  
    DEF_HASH(Result);  
    Result := GLOBAL_HASH('reference');  
END_MACRO;
```

GLOBAL_INT

GLOBAL_INT(name)

Parameters:

name String-like global name.

Reads a named integer global.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_GLOBAL_INT FROM EDIT;  
    DEF_INT(Result);  
    Result := GLOBAL_INT('reference');  
END_MACRO;
```

GLOBAL_STR

GLOBAL_STR(name)

Parameters:

name String-like global name.

Reads a named string global.
Returns a string.

Example macro.

```
$MACRO REFERENCE_GLOBAL_STR FROM EDIT;  
  DEF_STR(Result);  
  Result := GLOBAL_STR('reference');  
END_MACRO;
```

INQ_MACRO

INQ_MACRO(macroSpec)

Parameters:

macroSpec Macro specification or callback target.

Reports whether the macro catalog contains macroSpec.
Returns an integer.

Example macro.

```
$MACRO REFERENCE_INQ_MACRO FROM EDIT;  
  DEF_INT(Result);  
  Result := INQ_MACRO('reference');  
END_MACRO;
```

CREATE_GLOBAL_STR

CREATE_GLOBAL_STR(name, value);

Parameters:

name String-like global name.

value Value written or selected by the primitive.

Creates or replaces a named process-wide string global.

Example macro.

```
$MACRO REFERENCE_CREATE_GLOBAL_STR FROM EDIT;  
  CREATE_GLOBAL_STR('reference', 'reference');  
END_MACRO;
```

DELAY

DELAY(milliseconds);

Parameters:

milliseconds Positive delay or timer interval in milliseconds.

Yields macro execution for the requested number of milliseconds.

Example macro.

```
$MACRO REFERENCE_DELAY FROM EDIT;  
    DELAY(0);  
END_MACRO;
```

EXEC_ASSIGN

```
EXEC_ASSIGN(actionName, macroSpec, label);
```

Parameters:

actionName Named execution action.

macroSpec Macro specification or callback target.

label User-visible field label.

Assigns macroSpec to a named execution action.

Example macro.

```
$MACRO REFERENCE_EXEC_ASSIGN FROM EDIT;  
    EXEC_ASSIGN('reference', 'reference', 'reference');  
END_MACRO;
```

EXEC_SESSION_LIST

```
EXEC_SESSION_LIST(targetName);
```

Parameters:

targetName String or character positional argument.

Publishes execution-session information to targetName.

Example macro.

```
$MACRO REFERENCE_EXEC_SESSION_LIST FROM EDIT;  
    EXEC_SESSION_LIST('reference');  
END_MACRO;
```

EXEC_SESSION_STOP

```
EXEC_SESSION_STOP(sessionId);
```

Parameters:

sessionId Integer execution session identifier.

Requests cancellation of the named execution session.

Example macro.

```
$MACRO REFERENCE_EXEC_SESSION_STOP FROM EDIT;  
    EXEC_SESSION_STOP(0);  
END_MACRO;
```

LOAD_MACRO_FILE

```
LOAD_MACRO_FILE(path);
```

Parameters:

path A string-like file-system path.

Loads and compiles a macro source file into the catalog.

Example macro.

```
$MACRO REFERENCE_LOAD_MACRO_FILE FROM EDIT;  
    LOAD_MACRO_FILE('reference');  
END_MACRO;
```

RUN_MACRO

```
RUN_MACRO(macroSpec);
```

Parameters:

macroSpec Macro specification or callback target.

Runs a macro selected by macroSpec from the loaded catalog.

Example macro.

```
$MACRO REFERENCE_RUN_MACRO FROM EDIT;  
    RUN_MACRO('reference');  
END_MACRO;
```

SET_GLOBAL_HASH

```
SET_GLOBAL_HASH(name, value);
```

Parameters:

name String-like global name.

value Value written or selected by the primitive.

Writes a named process-wide hash global.

Example macro.

```
$MACRO REFERENCE_SET_GLOBAL_HASH FROM EDIT;  
    DEF_HASH(State);  
    SET_GLOBAL_HASH('reference', State);  
END_MACRO;
```

SET_GLOBAL_INT

```
SET_GLOBAL_INT(name, value);
```

Parameters:

name String-like global name.

value Value written or selected by the primitive.

Writes a named process-wide integer global.

Example macro.

```
$MACRO REFERENCE_SET_GLOBAL_INT FROM EDIT;  
    SET_GLOBAL_INT('reference', 0);  
END_MACRO;
```

SET_GLOBAL_STR

```
SET_GLOBAL_STR(name, value);
```

Parameters:

name String-like global name.

value Value written or selected by the primitive.

Writes a named process-wide string global.

Example macro.

```
$MACRO REFERENCE_SET_GLOBAL_STR FROM EDIT;  
    SET_GLOBAL_STR('reference', 'reference');  
END_MACRO;
```

UNLOAD_MACRO

```
UNLOAD_MACRO(macroSpec);
```

Parameters:

macroSpec Macro specification or callback target.

Removes macroSpec from the loaded macro catalog.

Example macro.

```
$MACRO REFERENCE_UNLOAD_MACRO FROM EDIT;  
    UNLOAD_MACRO('reference');  
END_MACRO;
```

Chapter 13

Settings, Keymaps and Themes

Settings and serialized UI descriptions cross a controlled VM boundary. They are not a macro-owned configuration store. This chapter documents the accepted settings, keymap and theme source forms.

13.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
CODECOLORS	procedure	Settings source.	p. 153
DEBUGGERCOLORS	procedure	Settings source.	p. 154
FILECOMPARECOLORS	procedure	Settings source.	p. 154
FILECOMPAREMINIMAPCOLORS	procedure	Settings source.	p. 154
HELPCOLORS	procedure	Settings source.	p. 154
KEYMAP_BIND	procedure	Settings source.	p. 155
KEYMAP_PROFILE	procedure	Settings source.	p. 155
KEYMAP_RESET	procedure	Settings source.	p. 155
KEYMAP_VERSION	procedure	Settings source.	p. 155
MENUDIALOGCOLORS	procedure	Settings source.	p. 156
MINIMAPCOLORS	procedure	Settings source.	p. 156
MRCOMPILERPROFILE	procedure	Settings source.	p. 156
MRFEPROFILE	procedure	Settings source.	p. 157
MRSETUP	procedure	Settings source.	p. 157
OTHERCOLORS	procedure	Settings source.	p. 158
SAVE_SETTINGS	procedure	Settings source.	p. 158
THEME_NAME	procedure	Settings source.	p. 158
THEME_RESET	procedure	Settings source.	p. 158
THEME_VERSION	procedure	Settings source.	p. 159
WINDOWCOLORS	procedure	Settings source.	p. 159

13.2 Reference Entries

CODECOLORS

```
CODECOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_CODECOLORS FROM EDIT;  
    CODECOLORS('reference');  
END_MACRO;
```

DEBUGGERCOLORS

```
DEBUGGERCOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_DEBUGGERCOLORS FROM EDIT;  
    DEBUGGERCOLORS('reference');  
END_MACRO;
```

FILECOMPARECOLORS

```
FILECOMPARECOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_FILECOMPARECOLORS FROM EDIT;  
    FILECOMPARECOLORS('reference');  
END_MACRO;
```

FILECOMPAREMINIMAPCOLORS

```
FILECOMPAREMINIMAPCOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_FILECOMPAREMINIMAPCOLORS FROM EDIT;  
    FILECOMPAREMINIMAPCOLORS('reference');  
END_MACRO;
```

HELPCOLORS

```
HELPCOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_HELPCOLORS FROM EDIT;
    HELPCOLORS('reference');
END_MACRO;
```

KEYMAP__BIND

```
KEYMAP_BIND(binding);
```

Parameters:

binding String or character positional argument.

Adds one serialized keymap binding definition for a defined profile. A binding names its profile, context, type, target, and key sequence. The current factory WordStar, Emacs, and Nano profiles bind Ctrl+A to the whole-document line-block action. The former edit-prefixed action remains a compatibility alias:

```
MR_BLOCK_MARK_ALL
MR_EDIT_MARK_ALL
```

Installed personal keymaps are not silently overwritten by a package update. The action is not an Emacs/Nano beginning-of-line binding.

Example macro. The WORDSTAR profile must already exist. This changes its Ctrl+A binding.

```
$MACRO REFERENCE_KEYMAP_BIND FROM EDIT;
    KEYMAP_BIND('name="WORDSTAR" context=EDIT type=ACTION ' +
                'target="MR_BLOCK_MARK_ALL" sequence="<Ctrl+A>"');
END_MACRO;
```

KEYMAP__PROFILE

```
KEYMAP_PROFILE(profileName);
```

Parameters:

profileName Serialized profile name.

Selects or defines the named serialized keymap profile.

Example macro.

```
$MACRO REFERENCE_KEYMAP_PROFILE FROM EDIT;
    KEYMAP_PROFILE('reference');
END_MACRO;
```

KEYMAP__RESET

```
KEYMAP_RESET();
```

Parameters: None.

Resets the serialized keymap source currently being assembled.

Example macro.

```
$MACRO REFERENCE_KEYMAP_RESET FROM EDIT;
    KEYMAP_RESET();
END_MACRO;
```

KEYMAP_VERSION

KEYMAP_VERSION(version);

Parameters:

version Serialized version text.

Sets the keymap source version.

Example macro.

```
$MACRO REFERENCE_KEYMAP_VERSION FROM EDIT;
    KEYMAP_VERSION('reference');
END_MACRO;
```

MENUDIALOGCOLORS

MENUDIALOGCOLORS(definition);

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_MENUDIALOGCOLORS FROM EDIT;
    MENUDIALOGCOLORS('reference');
END_MACRO;
```

MINIMAPCOLORS

MINIMAPCOLORS(definition);

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_MINIMAPCOLORS FROM EDIT;
    MINIMAPCOLORS('reference');
END_MACRO;
```

MRCOMPILERPROFILE

MRCOMPILERPROFILE(operation, profileId, argument3, argument4);

Parameters (all strings):

operation DEFINE to create a profile, or SET to assign one of its settings.

profileId New profile ID for DEFINE; existing ID for SET.

argument3 Display name for DEFINE; setting key for SET.

argument4 Supported toolchain for DEFINE; serialized setting value for SET.

Duplicate definitions and unknown settings are errors.

This is a startup settings directive. Ordinary edit macros cannot call it. Compatible compiled-program debugging requires a Debug profile and a current executable artifact; compiler selection alone does not start GDB. Supported GDB toolchains are GCC, CLANG, GCC_C, CLANG_C, RUST, GO, FPC, and FREEBASIC.

Example settings source. These lines illustrate a profile definition within controlled startup source; retain the surrounding canonical settings and build-version declaration.

```
MRCOMPILERPROFILE('DEFINE', 'C_DEBUG_DOC', 'C Debug', 'GCC_C');
MRCOMPILERPROFILE('SET', 'C_DEBUG_DOC', 'EXECUTABLE', '/usr/bin/gcc');
```

Use Compiler profiles and Automatic Setup to obtain the full build arguments and output configuration appropriate to the installed toolchain.

MRFEPROFILE

```
MRFEPROFILE(operation, profileId, argument3, argument4);
```

Parameters (all strings):

operation DEFINE to create a profile, EXT to add an extension, or SET to assign an editor setting.

profileId New profile ID for DEFINE; existing ID for EXT or SET.

argument3 Display name for DEFINE, filename extension for EXT, or setting key for SET.

argument4 Empty for DEFINE and EXT; serialized setting value for SET.

These are editor-profile directives, not source/output file arguments.

The directive is legal only in controlled startup settings source. A matching filename-extension profile can override the global automatic bracket-pairing option independently of other editor settings.

Example settings source.

```
MRFEPROFILE('DEFINE', 'C_DOC', 'C source', '');
MRFEPROFILE('EXT', 'C_DOC', 'c', '');
MRFEPROFILE('SET', 'C_DOC', 'AUTO_PAIR_BRACKETS', 'TRUE');
```

These lines belong in a complete canonical startup settings source, not in an ordinary edit macro. See the bracket-pairing semantics under MRSETUP.

MRSETUP

```
MRSETUP(key, value);
```

Parameters:

key String naming a supported settings key.

value String containing that key's serialized value. Boolean values accept the settings parser's boolean literals; numeric settings validate their own integer range.

Applies an assignment through the central settings model during controlled startup execution. Ordinary edit macros cannot use it to change arbitrary settings. The existing non-startup File Compare minimap colour route is a narrow exception, not general permission to call startup directives.

Hero destinations and threshold.

```
HERO_MESSAGES_ON_MESSAGE_LINE
HERO_MESSAGES_IN_LOGFILE
HERO_MESSAGES_FILE_THRESHOLD_MB
```

The first two keys independently enable the Hero message-line and log destinations; defaults are true and false. The third accepts integers 0–100, default eight, in units of 1,048,576 bytes. File-aware Hero notices require a file strictly larger than the threshold before either destination is used. Non-file Hero notices do not use that filter. Ordinary warning and validation owners are unaffected. The retired serialized MESSAGES key is a no-op; the runtime variable still controls the Hero message-line destination.

Automatic bracket pairing.**AUTO_PAIR_BRACKETS**

This boolean defaults to false and can also be overridden in a matching filename-extension profile. With insert mode on, no selection, and a single-character text event, an opening parenthesis, bracket, or brace inserts its closer and leaves the cursor between the pair. A matching closer already at the cursor is skipped. Quotes, bracketed paste, overwrite mode, and selected-text replacement do not take this route; syntax classification does not exclude comments or strings.

Example settings source. Keep the complete generated settings source and its version declaration; these are example assignments inside that source, not an edit macro.

```
MRSETUP('HERO_MESSAGES_ON_MESSAGELINE', 'TRUE');
MRSETUP('HERO_MESSAGES_IN_LOGFILE', 'FALSE');
MRSETUP('HERO_MESSAGES_FILE_THRESHOLD_MB', '8');
MRSETUP('AUTO_PAIR_BRACKETS', 'TRUE');
```

OTHERCOLORS

```
OTHERCOLORS(definition);
```

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_OTHERCOLORS FROM EDIT;
    OTHERCOLORS('reference');
END_MACRO;
```

SAVE_SETTINGS

```
SAVE_SETTINGS;
```

Parameters: None.

Persists the current runtime settings model through the VM boundary.

Example macro.

```
$MACRO REFERENCE_SAVE_SETTINGS FROM EDIT;
    SAVE_SETTINGS;
END_MACRO;
```

THEME_NAME

```
THEME_NAME(themeName);
```

Parameters:

themeName String or character positional argument.

Sets the serialized theme name.

Example macro.

```
$MACRO REFERENCE_THEME_NAME FROM EDIT;
    THEME_NAME('reference');
END_MACRO;
```

THEME_RESET

THEME_RESET();

Parameters: None.

Resets the serialized theme source currently being assembled.

Example macro.

```
$MACRO REFERENCE_THEME_RESET FROM EDIT;  
    THEME_RESET();  
END_MACRO;
```

THEME_VERSION

THEME_VERSION(version);

Parameters:

version Serialized version text.

Sets the theme source version.

Example macro.

```
$MACRO REFERENCE_THEME_VERSION FROM EDIT;  
    THEME_VERSION('reference');  
END_MACRO;
```

WINDOWCOLORS

WINDOWCOLORS(definition);

Parameters:

definition Serialized keymap or theme definition.

Adds the named serialized theme colour definition.

Example macro.

```
$MACRO REFERENCE_WINDOWCOLORS FROM EDIT;  
    WINDOWCOLORS('reference');  
END_MACRO;
```

Chapter 14

External Environment and Editor Commands

This chapter contains the bounded process-environment bridge and procedures that invoke established MR application commands. They request existing behaviour; they do not provide unrestricted native handles or a separate DOS shell.

14.1 Primitive Map

The map is a local index for this chapter. The final column is a generated page reference and hyperlink to the full entry, where the source form, parameters, semantics and a complete example macro appear.

Primitive	Role	Purpose	Reference
GET_ENVIRONMENT	intrinsic	MR command bridge.	p. 161
OS_BACK	intrinsic	MR command bridge.	p. 161
OS_COLOR	intrinsic	MR command bridge.	p. 161
OS_VERSION	intrinsic	Shell version query.	p. 161
SUBSHELL	intrinsic	MR command bridge.	p. 161
VERSION	intrinsic	MR command bridge.	p. 162
ACTIVE_KEYMAP_PROFILE	procedure	MR command bridge.	p. 162
EXIT_SAVE_ALL	procedure	MR command bridge.	p. 162
MACRO_TOGGLE_RECORDING	procedure	MR command bridge.	p. 162
MATCH_PARENTHESIS	procedure	MR command bridge.	p. 163
SETUP_BACKUPS_AUTOSAVE	procedure	MR command bridge.	p. 163
SETUP_COLOR	procedure	MR command bridge.	p. 163
SETUP_COMPILER_PROFILES	procedure	MR command bridge.	p. 163
SETUP_EDIT_SETTINGS	procedure	MR command bridge.	p. 164
SETUP_FILENAME_EXTENSIONS	procedure	MR command bridge.	p. 164
SETUP_KEYMAP	procedure	MR command bridge.	p. 164
SETUP_LIVE_LOGS	procedure	MR command bridge.	p. 164
SETUP_PATHS	procedure	MR command bridge.	p. 165
SETUP_USER_INTERFACE	procedure	MR command bridge.	p. 165
QUIT	procedure	MR command bridge.	p. 165
REST_OS_SCREEN	procedure	MR command bridge.	p. 165
SAVE_OS_SCREEN	procedure	MR command bridge.	p. 166
CHANGE_DIR	procedure	MR command bridge.	p. 166
FORK	procedure	MR command bridge.	p. 166
SHELL_TO_OS	procedure	MR command bridge.	p. 166
WRITE_SOD	procedure	MR command bridge.	p. 167

14.2 Reference Entries

GET_ENVIRONMENT

GET_ENVIRONMENT(variableName)

Parameters:

variableName String-like environment variable name.

Returns the named environment value.

Returns a string.

Example macro.

```
$MACRO REFERENCE_GET_ENVIRONMENT FROM EDIT;  
    DEF_STR(Result);  
    Result := GET_ENVIRONMENT('reference');  
END_MACRO;
```

OS_BACK

OS_BACK

Parameters: None.

Returns the supported system background attribute.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_OS_BACK FROM EDIT;  
    DEF_INT(Result);  
    Result := OS_BACK;  
END_MACRO;
```

OS_COLOR

OS_COLOR

Parameters: None.

Returns the supported system colour attribute.

Returns an integer.

Example macro.

```
$MACRO REFERENCE_OS_COLOR FROM EDIT;  
    DEF_INT(Result);  
    Result := OS_COLOR;  
END_MACRO;
```

OS_VERSION

OS_VERSION()

Parameters: None.

Invokes the configured shell with its version option through the controlled subprocess path when evaluated.

Returns the first output line as a string. If the shell produces no output, returns the shell executable name.

Example macro.

```
$MACRO REFERENCE_OS_VERSION FROM EDIT;  
    DEF_STR(Result);  
    Result := OS_VERSION();  
END_MACRO;
```

SUBSHELL

SUBSHELL(command, mode)

Parameters:

command Controlled command text.

mode Editor mode constant.

Runs the controlled command path and returns captured output.
Returns a string.

Example macro.

```
$MACRO REFERENCE_SUBSHELL FROM EDIT;
  DEF_STR(Result);
  Result := SUBSHELL('reference', 0);
END_MACRO;
```

VERSION

VERSION

Parameters: None.

Returns the MRMAC runtime version string.
Returns a string.

Example macro.

```
$MACRO REFERENCE_VERSION FROM EDIT;
  DEF_STR(Result);
  Result := VERSION;
END_MACRO;
```

ACTIVE_KEYMAP_PROFILE

ACTIVE_KEYMAP_PROFILE(profileName);

Parameters:

profileName Serialized profile name.

Sets the active serialized keymap profile.

Example macro.

```
$MACRO REFERENCE_ACTIVE_KEYMAP_PROFILE FROM EDIT;
  ACTIVE_KEYMAP_PROFILE('reference');
END_MACRO;
```

EXIT_SAVE_ALL

EXIT_SAVE_ALL;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_EXIT_SAVE_ALL FROM EDIT;
  EXIT_SAVE_ALL;
END_MACRO;
```

MACRO_TOGGLE_RECORDING

MACRO_TOGGLE_RECORDING;

Parameters: None.

Performs the documented MR operation named by this primitive in the active execution context.

Example macro.

```
$MACRO REFERENCE_MACRO_TOGGLE_RECORDING FROM EDIT;  
    MACRO_TOGGLE_RECORDING;  
END_MACRO;
```

MATCH_PARENTHESES

MATCH_PARENTHESES;

Parameters: None.

Moves the cursor to the matching parenthesis, square bracket, or brace. The origin is the delimiter at the cursor or immediately before it. Matching counts nesting of that delimiter pair in the document text; strings and comments are not excluded. Missing delimiters or matches produce feedback without moving to an invented location.

Example macro.

```
$MACRO REFERENCE_MATCH_PARENTHESES FROM EDIT;  
    MATCH_PARENTHESES;  
END_MACRO;
```

SETUP_BACKUPS_AUTOSAVE

SETUP_BACKUPS_AUTOSAVE;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_BACKUPS_AUTOSAVE FROM EDIT;  
    SETUP_BACKUPS_AUTOSAVE;  
END_MACRO;
```

SETUP_COLOR

SETUP_COLOR;

Parameters: None.

Opens Color setup. Select a role and edit its foreground/background RGB components with six sliders in the range 0–255. The hexadecimal pair is a readout; it and the role sample update immediately with each component change, including during a mouse drag. Role changes retain previous component edits in the same dialog draft. The sample uses the configured output mode. OK applies and persists changes; closing a dirty draft uses Save/Discard/Cancel gating. Load Theme activates a theme and Save Theme applies and writes the draft. Those explicit operations are not rolled back by later cancellation. RGB versus palette output is selected in User Interface Settings.

Example macro.

```
$MACRO REFERENCE_SETUP_COLOR FROM EDIT;  
    SETUP_COLOR;  
END_MACRO;
```

SETUP_COMPILER_PROFILES

SETUP_COMPILER_PROFILES;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_COMPILER_PROFILES FROM EDIT;  
    SETUP_COMPILER_PROFILES;  
END_MACRO;
```

SETUP_EDIT_SETTINGS

SETUP_EDIT_SETTINGS;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_EDIT_SETTINGS FROM EDIT;  
    SETUP_EDIT_SETTINGS;  
END_MACRO;
```

SETUP_FILENAME_EXTENSIONS

SETUP_FILENAME_EXTENSIONS;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_FILENAME_EXTENSIONS FROM EDIT;  
    SETUP_FILENAME_EXTENSIONS;  
END_MACRO;
```

SETUP_KEYMAP

SETUP_KEYMAP;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_KEYMAP FROM EDIT;  
    SETUP_KEYMAP;  
END_MACRO;
```

SETUP_LIVE_LOGS

SETUP_LIVE_LOGS;

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_LIVE_LOGS FROM EDIT;
    SETUP_LIVE_LOGS;
END_MACRO;
```

SETUP_PATHS

```
SETUP_PATHS;
```

Parameters: None.

Opens the corresponding MR setup dialog through the application command route.

Example macro.

```
$MACRO REFERENCE_SETUP_PATHS FROM EDIT;
    SETUP_PATHS;
END_MACRO;
```

SETUP_USER_INTERFACE

```
SETUP_USER_INTERFACE;
```

Parameters: None.

Opens User Interface Settings, including independent Hero message-line/log destinations, the synchronized 0–100 MB threshold slider and editable field, and Color Management. Color Management selects automatic 24-bit RGB output or terminal-palette output; the RGB role values remain the same settings in either mode. Invalid input prevents applying the dialog draft.

Example macro.

```
$MACRO REFERENCE_SETUP_USER_INTERFACE FROM EDIT;
    SETUP_USER_INTERFACE;
END_MACRO;
```

QUIT

```
QUIT();
QUIT(saveMode);
```

Parameters:

saveMode Optional integer quit/save policy.

Requests the editor exit route with optional save policy.

Example macro.

```
$MACRO REFERENCE_QUIT FROM EDIT;
    QUIT(0);
END_MACRO;
```

REST_OS_SCREEN

```
REST_OS_SCREEN;
```

Parameters: None.

Restores saved supported system-screen state.

Example macro.

```
$MACRO REFERENCE_REST_OS_SCREEN FROM EDIT;
    REST_OS_SCREEN;
END_MACRO;
```

SAVE_OS_SCREEN

```
SAVE_OS_SCREEN;
```

Parameters: None.

Saves supported system-screen state.

Example macro.

```
$MACRO REFERENCE_SAVE_OS_SCREEN FROM EDIT;
    SAVE_OS_SCREEN;
END_MACRO;
```

CHANGE_DIR

```
CHANGE_DIR(path);
```

Parameters:

path A string-like file-system path.

Changes the macro runtime working directory.

Example macro.

```
$MACRO REFERENCE_CHANGE_DIR FROM EDIT;
    CHANGE_DIR('reference');
END_MACRO;
```

FORK

```
FORK(command, argument1, argument2, argument3,
      argument4, argument5, argument6, argument7);
```

Parameters:

command Controlled command text.

argument1 First controlled-process string argument.

argument2 Second controlled-process string argument.

argument3 Third controlled-process string argument.

argument4 Fourth controlled-process string argument.

argument5 Fifth controlled-process string argument.

argument6 Sixth controlled-process string argument.

argument7 Seventh controlled-process string argument.

Starts a controlled external process with its command and positional string arguments.

Example macro.

```
$MACRO REFERENCE_FORK FROM EDIT;
    FORK('reference', 'reference', 'reference', 'reference',
        'reference', 'reference', 'reference', 'reference');
END_MACRO;
```

SHELL__TO__OS

```
SHELL_TO_OS(command, mode);
```

Parameters:

command Controlled command text.

mode Editor mode constant.

Executes a controlled system command using mode.

Example macro.

```
$MACRO REFERENCE_SHELL_TO_OS FROM EDIT;  
    SHELL_TO_OS('reference', 0);  
END_MACRO;
```

WRITE__SOD

```
WRITE_SOD(text);
```

Parameters:

text String-like text.

Writes text through the supported standard-output device path.

Example macro.

```
$MACRO REFERENCE_WRITE_SOD FROM EDIT;  
    WRITE_SOD('reference');  
END_MACRO;
```

Part V

Debugging

Chapter 15

Debugging MRMAC Macros

15.1 Starting a debug session

The MRMAC debugger starts a loaded macro from the macro catalog and compiles it with source-map information only for that debug run. Normal macro loading does not create source maps. A debugger session runs real MRMAC semantics and keeps the macro's normal execution profile. Background-safe code runs in finite Macro workers, staged code keeps the normal snapshot, conflict and UI-thread commit path, and remaining UI-affine code advances in bounded UI-pump slices. File, editor and UI side effects are therefore not a preview, but none of these routes may monopolise the UI thread.

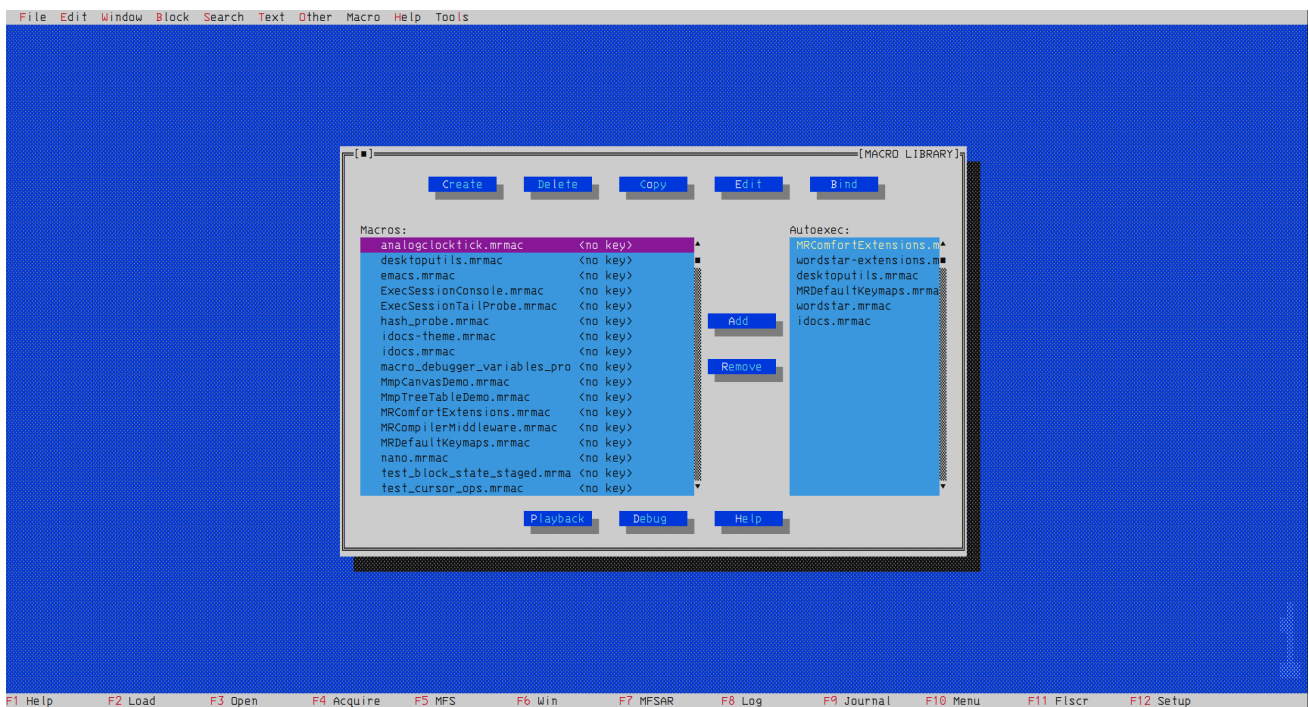


Figure 15.1: The Macro Library holds loaded macros. Select the macro to inspect, then choose Debug to create a debugger session for that macro.

The new session pauses at its first debuggable source span. Debugger Output identifies the macro, session, state and stop reason; the Variables and Watches panes show the state available at that pause.

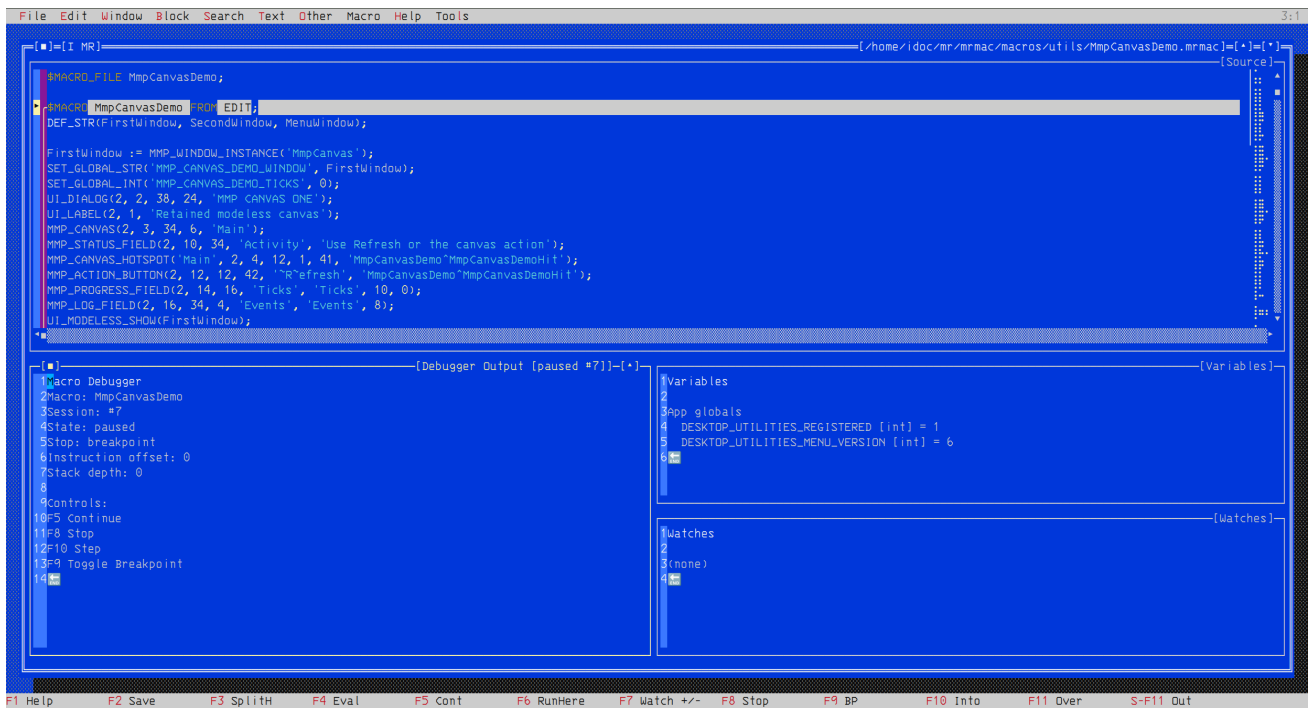


Figure 15.2: A newly started MRMAC debug session paused at a source span. The debugger panes identify the session and expose its current state.

15.2 Source-view context menu

A plain single right-click in the debugger source text opens the ordinary editor context menu. Edit contains Cut, Copy, and Unmark when text is marked, plus Paste; without marked text it contains Paste. Outline lists available structure from open source buffers. At a supported text target, References and Rename open the multi-file search or replacement dialog with that text prefilled. These operations use text matching, not MRMAC binding identity. Escape dismisses the menu; Enter selects an item or opens the Edit submenu.

The extra breakpoint/run/watch/evaluation/step group in the compiled-program GDB source menu is specific to that backend. Use the MRMAC debugger controls below for macro execution. The left-button gutter double-click is shared by both debuggers.

15.3 Breakpoints, running and stopping

F9 sets a breakpoint at the cursor source line or clears one already there. Alternatively, double-click with the left mouse button on the desired line number in the source view's visible line-number gutter. The gutter must be visible and debugger controls active. This invokes the same action as F9 and resolves folded viewport rows to document lines; it is not a folding-gutter gesture. The same mouse action is available in the compiled-program GDB debugger, with that backend's source-location rules. The MRMAC debugger maps the line to its debuggable source span, so a comment, declaration or other non-executable line cannot receive a breakpoint. Shift-F9 enables or disables the breakpoint at the cursor. Its Alt-Shift-F9 and Ctrl-Shift-F9 forms respectively enable or disable all breakpoints, or clear all breakpoints, in the current macro file.

F6 Run Here closes any live session and starts the selected macro again, with a temporary breakpoint at the cursor line. The cursor line must be debuggable; another enabled breakpoint reached first still stops execution first. F5 continues a paused session. While that session is running, F5 requests a pause. F8 closes a live session, clears its current-location marker and removes its variable state. With no live session, F8 resets the debugger by starting a new session which pauses at the first debuggable source span.

Breakpoints and watches are debugger state, separate from the general settings. Their definitions are stored with the debugger Bento's workspace configuration and rebound when that workspace is restored. The persisted definition contains the source and macro identity, source line, enabled state, optional condition and watch expression. It contains no live session, instruction pointer, call stack, variable value, generated source map or bytecode offset. Restore therefore creates a cold debugger with its breakpoints and watches, not a resurrected paused VM.

Exactly one debugger may own a source identity at a time. Internally that identity is the normalized resolved source path plus the macro name; the UI shows the familiar `Filename~MacroName` form. If a stored breakpoint

cannot bind to a debuggable span after the source is recompiled, it remains visible with the Debugger group's unbound colour, is logged, and is retried by a later debug start.

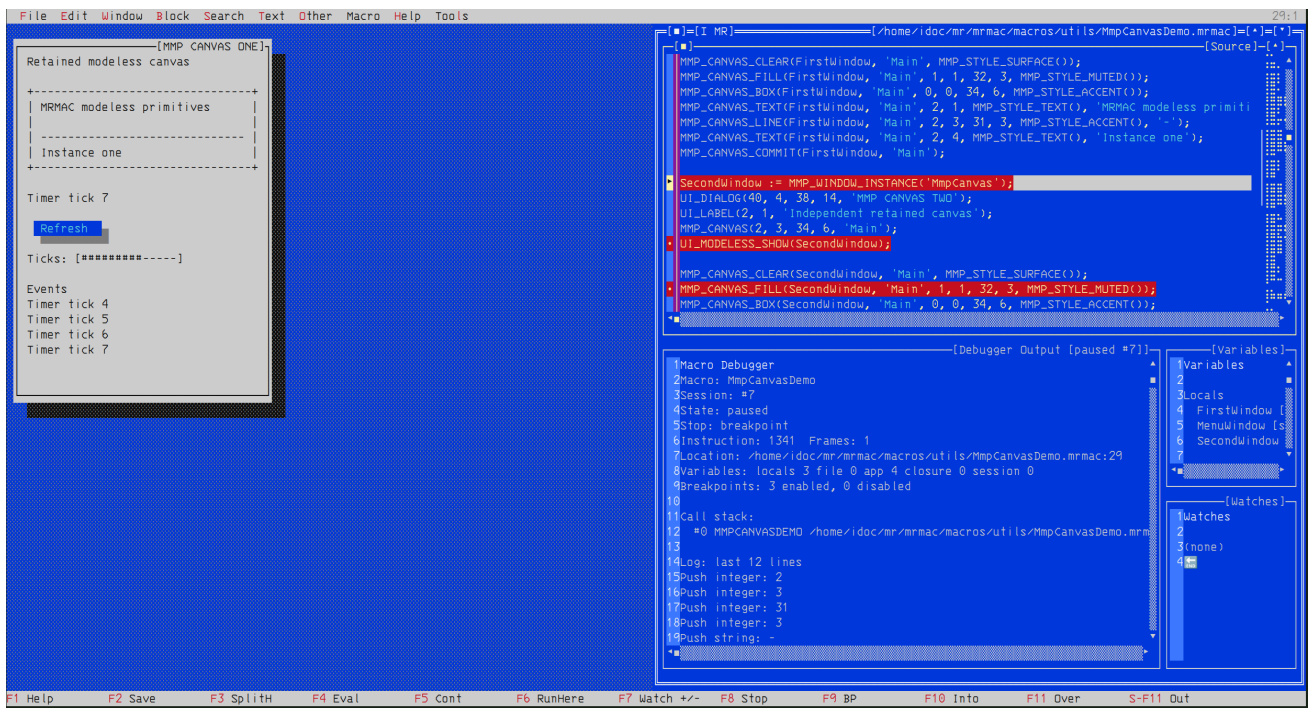


Figure 15.3: A breakpoint pauses the macro at the marked source span while the displayed modeless window remains the real UI created by that session.

An MMP timer callback starts as its own runtime execution session. If its source is open in an observing debugger Bento and the callback has an enabled breakpoint, it stops in that Bento like any other macro. The source pane, Debugger Output, Variables and Watches remain the controls for that paused callback; F5 resumes it. Without an observing debugger, timer callbacks retain their normal scheduling behaviour.

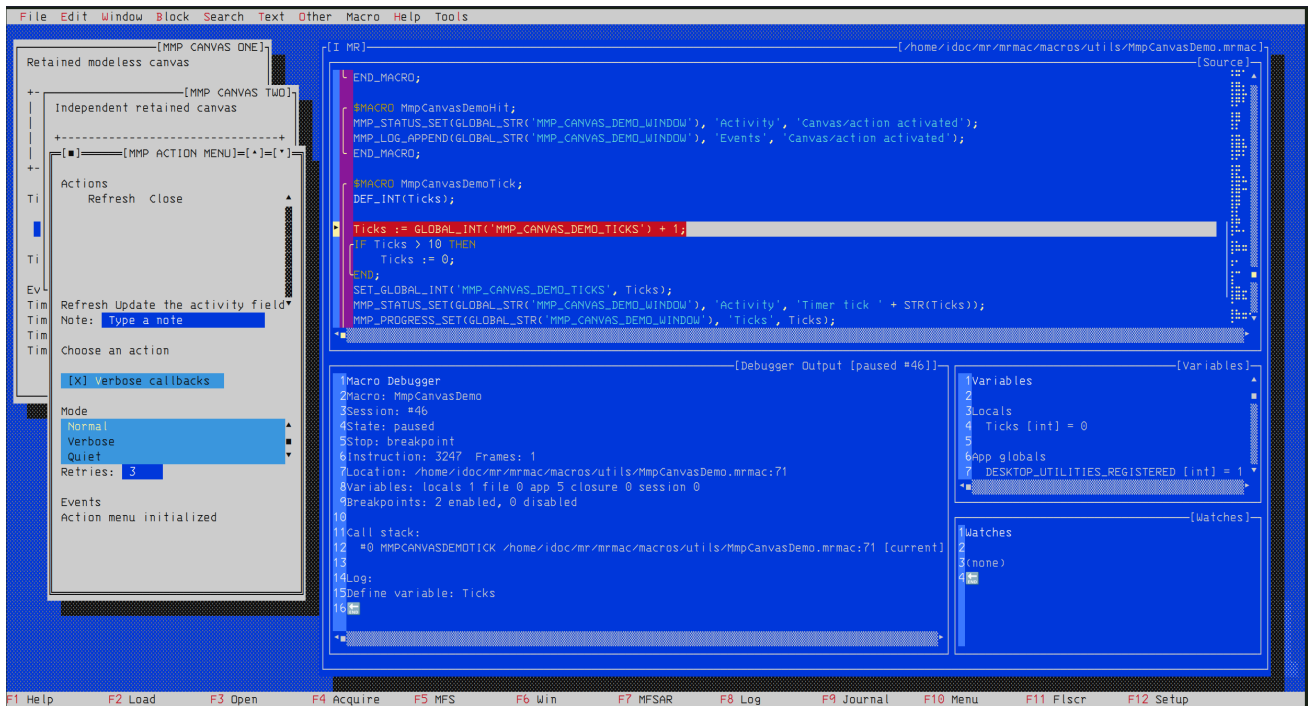


Figure 15.4: An enabled breakpoint in the `MmpCanvasDemoTick` timer callback stops the independently scheduled callback at its source line. The modeless MMP windows remain visible while the debugger exposes the callback session and its local `Ticks` value.

15.4 Stepping

Stepping requires a paused live session. **F10 Step Into** advances to the next source span and enters a call when that is the next operation. **F11 Step Over** executes a called operation without stopping inside it, then pauses at the next source span in the current frame. **Shift-F11 Step Out** continues the current frame to its return and then pauses in its caller. Continue runs until a breakpoint, pause request, stop request, runtime error or normal completion. Nested local calls and starts of loaded macros contribute frames to the displayed debug location when source mapping is available.

Intrinsics, external I/O and deferred UI playback are black boxes for stepping and cooperative pause. The debugger never interrupts one halfway through. When background or staged execution reaches a breakpoint, pause or step stop, its finite worker ends and the VM remains parked with the Execution Session; the paused debugger consumes no worker. Continue or Step starts another finite worker action. UI-affine execution uses the same stop policy in bounded event-pump slices.

At every pause, the source pane follows the mapped instruction, marks its source span and centres the line. Debugger Output reports the macro, session, state, stop reason, instruction offset, frame count, location, variable and breakpoint counts, call stack and the latest log lines.

Debugger Output lists the call stack below its summary. Frame #0 is the paused macro. Later frames are callers: `[CALL]` identifies a nested call in that macro, while `[RUN_MACRO]` identifies the macro that started a loaded child macro. Each frame includes its mapped source location when available.

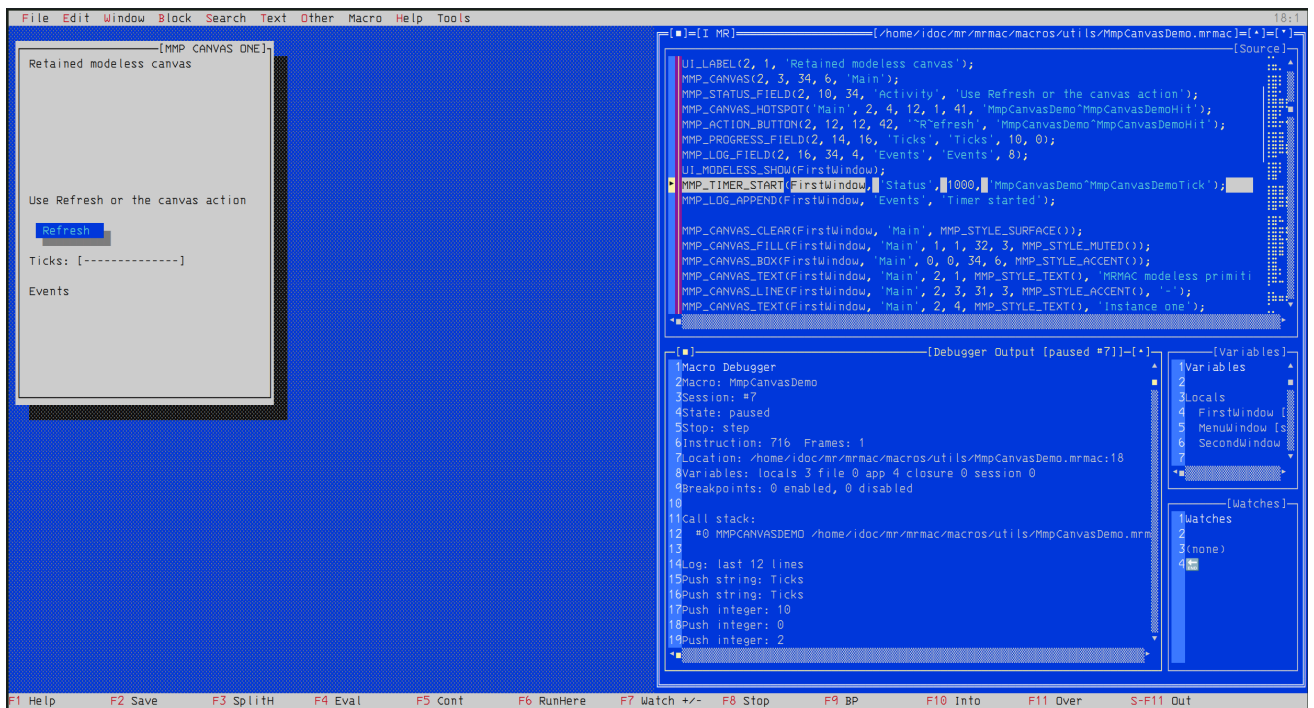


Figure 15.5: A step stop advances the current source location and records *step* as the stop reason in Debugger Output.

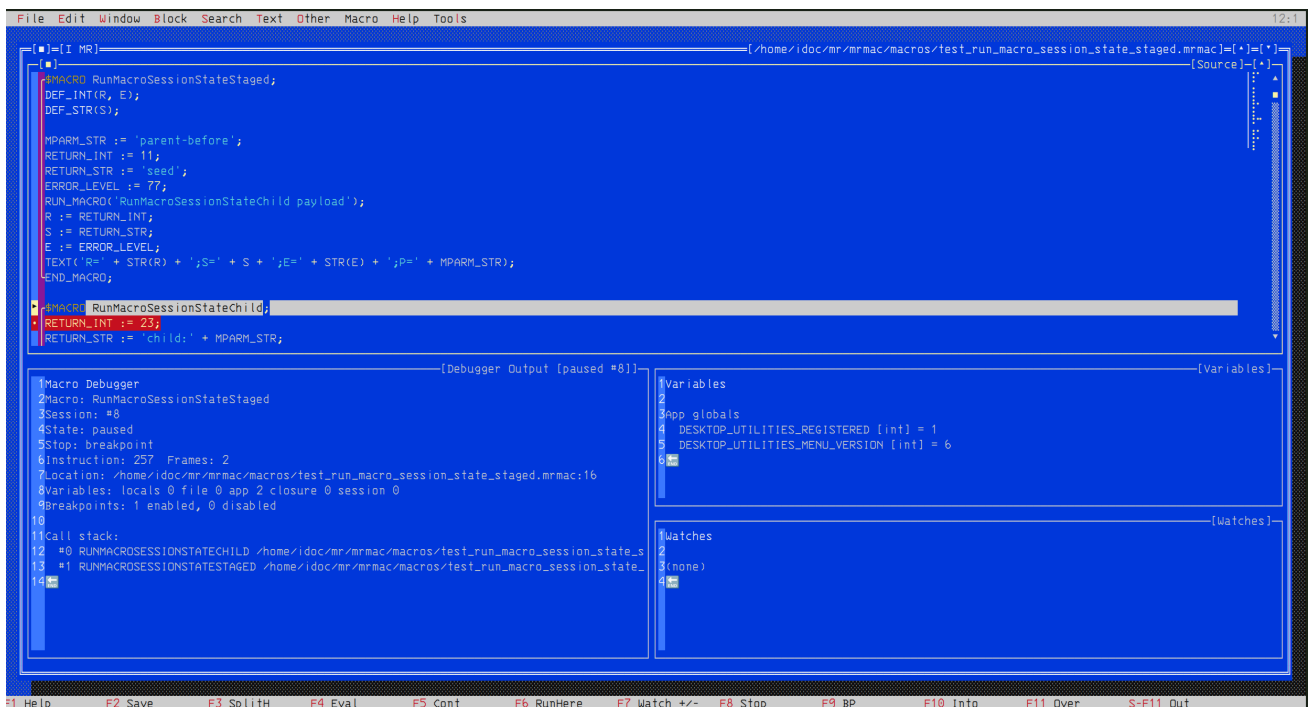


Figure 15.6: A breakpoint in a child macro started by `RUN_MACRO` leaves the child at frame #0 and its parent at frame #1 in the Call stack.

Chapter 16

Inspecting State and Diagnosing Failures

16.1 Variables and collections

The debugger panes show the scopes that MRMAC actually implements: locals, application globals, closure variables and execution-session variables. MRMAC has no file-global scope; a `DEF_*` declaration belongs to its current VM, closure or long-lived session, while an explicit global belongs to the process-wide application-global store.

While a session is paused, select an existing integer, real, string or character scalar in Variables to open its inline value field. Enter validates and writes the new value, then refreshes the complete Variables tree and Watches; Esc cancels the edit. The VM validates the root name, actual scope, type and complete collection path again at the write boundary. The pane is not a writable shadow of VM state.

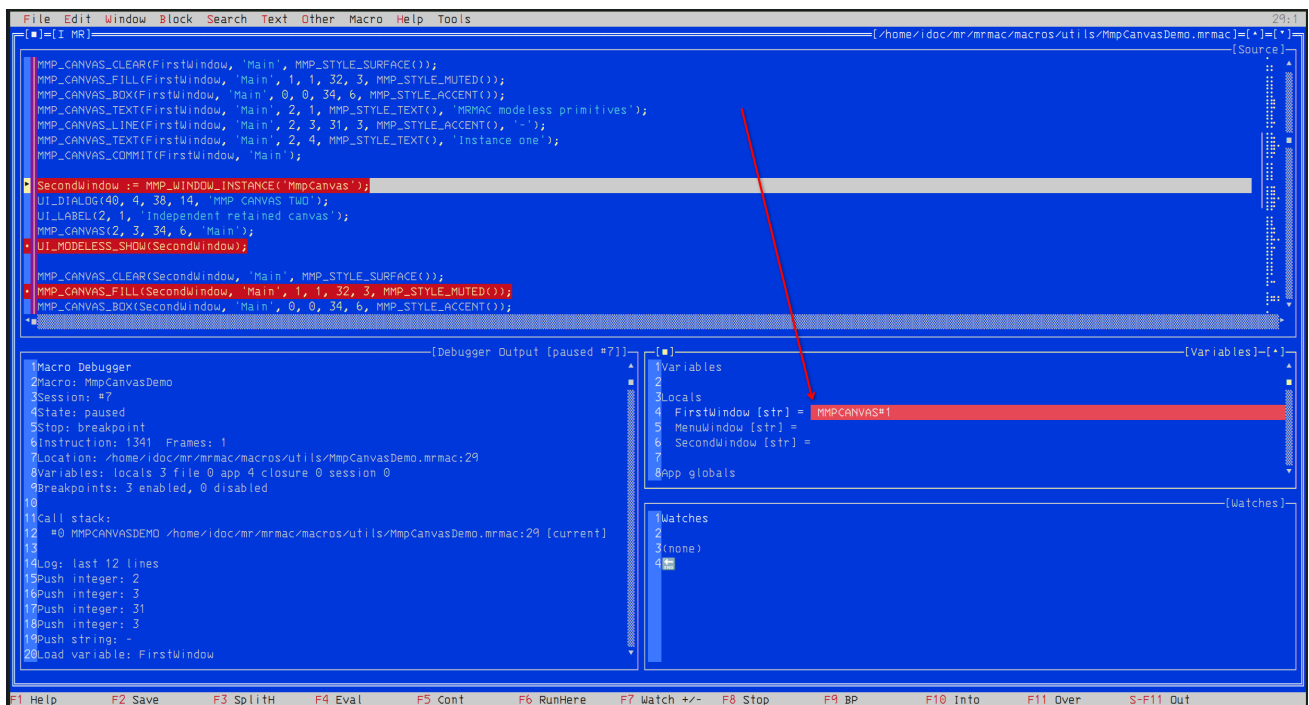


Figure 16.1: The source span and Variables pane identify the same scalar. Existing scalar values may be changed while the session is paused.

Hashes and arrays appear as complete indented trees, including nested collections and explicit cycle references. A container row still shows its type and size, but every key or one-based array element follows as its own selectable row; there is no arbitrary collection snapshot limit.

Selecting a hash container opens an insertion field. Enter `key=value` for a string entry, or `key:type=value` where `type` is `int`, `real`, `str`, `char`, `hash`, `int []`, `real []`, `str []`, `char []` or `hash []`. Selecting an array container appends one element of that array's declared type. On a non-root element, `:delete` removes it; `:rename=newKey` renames a hash entry. Nested scalar rows use the normal typed scalar edit. To store a scalar string beginning with one colon, enter two leading colons. Every successful structural change is applied to the paused VM's local, closure, session or application-global backing store and is then projected afresh.

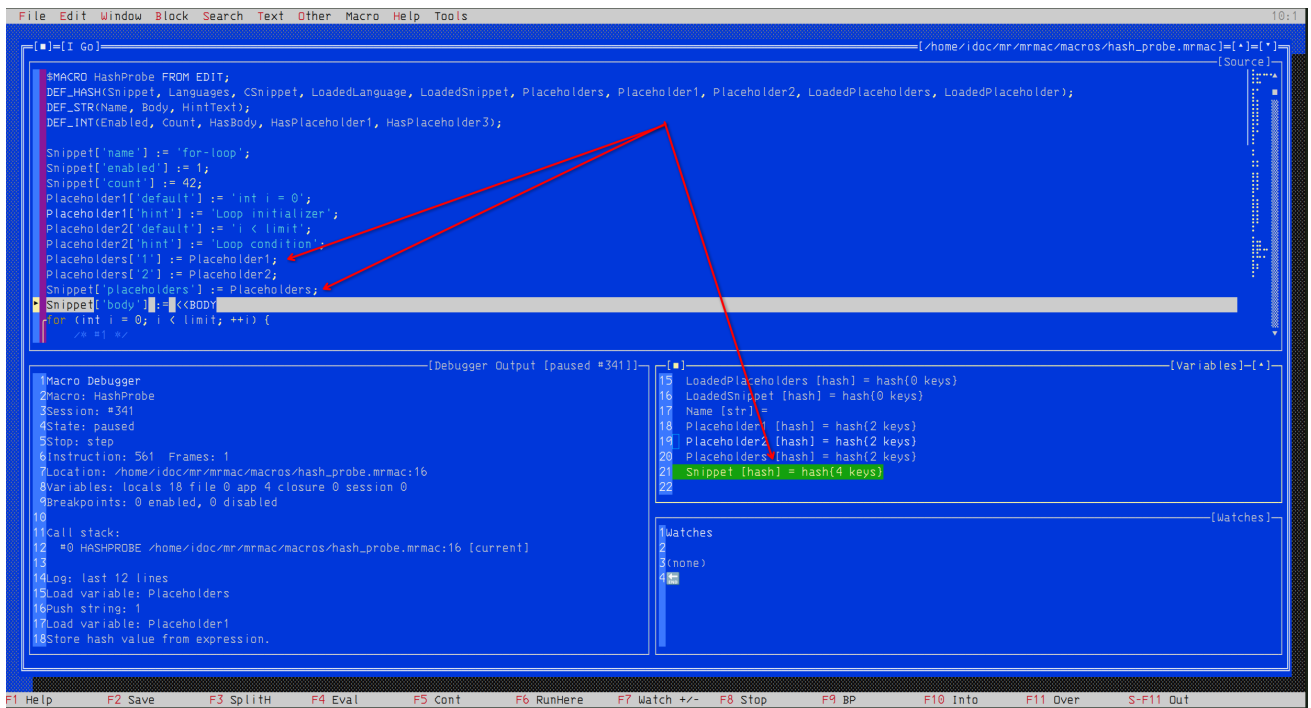


Figure 16.2: A nested hash is projected as a typed hierarchy in Variables. Container summaries remain visible while keys and nested values are individually selectable at the paused location.

16.2 Watches and expression evaluation

F7 opens Watch expression and stores the entered expression for the current debug macro; it does not require a live session. **Shift-F7** opens Remove watch and removes the watch whose expression exactly matches the entered text. A stored watch is evaluated only for a paused live session; otherwise its pane reports that no live debug session is available. Watches are compiled through the canonical compiler in a restricted pure-expression mode. **F4** evaluates an expression against the paused VM. A watch therefore reports a value or a compiler/runtime diagnostic; it does not execute an arbitrary procedure or create a second parser.

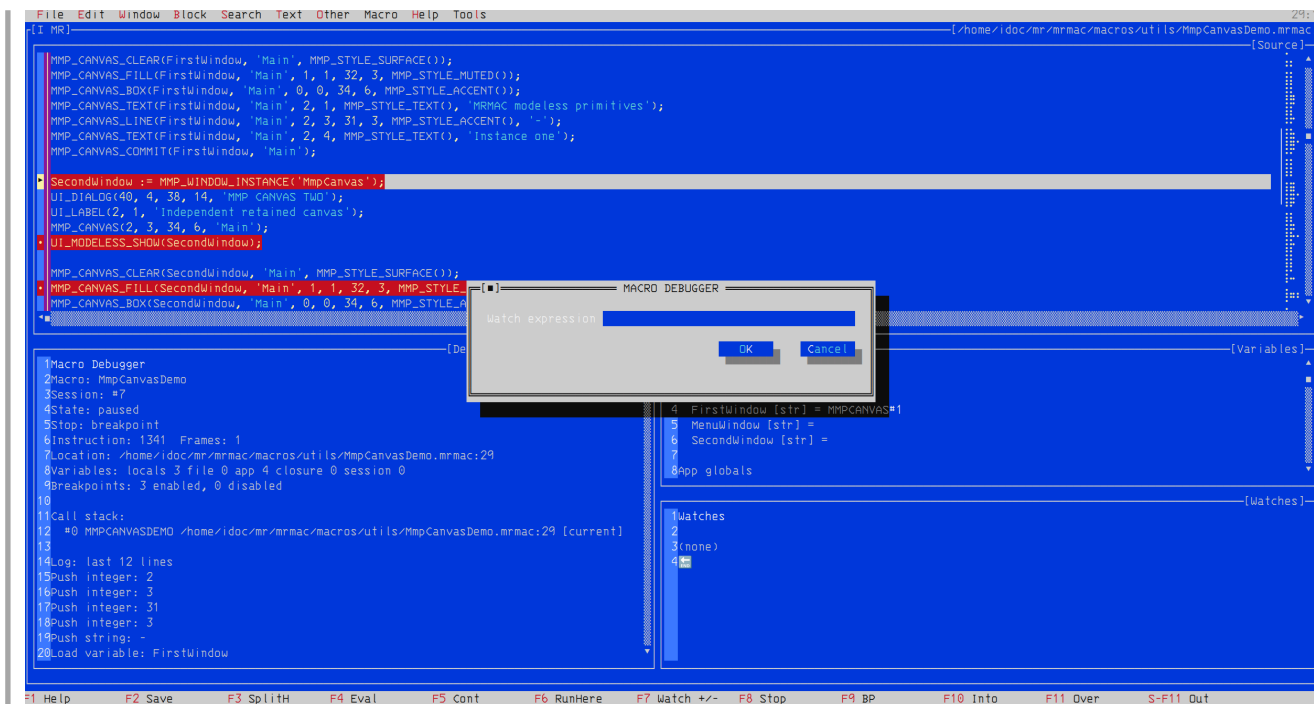


Figure 16.3: F7 opens the Watch expression dialog for the selected debug macro. The expression is compiled in restricted pure-expression mode and is evaluated when the macro has a paused live session.

F4 evaluates a pure expression immediately against the paused state. Debugger Output records the expression and result without invoking an arbitrary macro procedure.

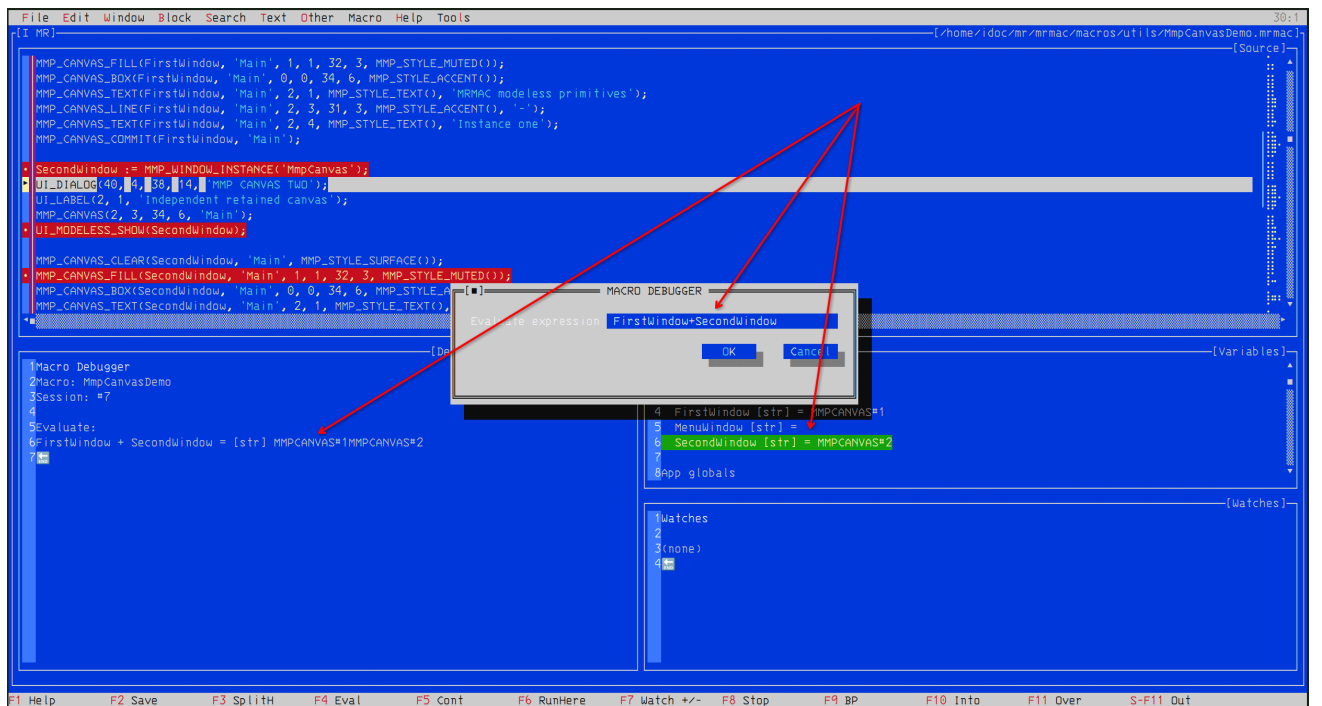


Figure 16.4: F4 evaluates an expression against paused values. Debugger Output records its result and the Variables pane shows the participating state.

16.3 Diagnostic model

Compiler diagnostics identify a source line and a concrete syntax, type or declaration problem. Runtime diagnostics identify the failed operation while the debugger retains the current source location and snapshots. MRMAC has no stable MEMAC-style numeric error catalogue; treat the displayed diagnostic text and the paused state as the authoritative failure report.

Chapter 17

Example Macros

17.1 Insert the current date

```
$MACRO INSERT_DATE FROM EDIT;  
    TEXT(DATE);  
END_MACRO;
```

17.2 A compact modeless status panel

```
$MACRO SHOW_STATUS FROM EDIT;  
    UI_DIALOG(2, 2, 42, 7, 'MRMAC status');  
    MMP_STATUS_FIELD(2, 2, 36, 'state', 'ready');  
    MMP_ACTION_BUTTON(2, 4, 14, 1, 'Refresh', 'status^REFRESH_STATUS');  
    UI_MODELESS_SHOW('StatusPanel');  
END_MACRO;  
  
$MACRO REFRESH_STATUS FROM EDIT;  
    MMP_STATUS_SET('StatusPanel', 'state', 'updated');  
END_MACRO;
```

Appendix A

MEMAC Compatibility Notes

MRMAC retains a broad, practical subset of MEMAC syntax and editor commands, but it does not emulate the DOS shell, EMS/video-card probing or a separate word-processing runtime. In particular, historical DOS directory-shell commands and device-specific video operations are not macro interfaces in MR. The nearby footnotes in this manual identify comparable names where migration is likely.

Appendix B

Command Index

Index

\$CLOSURE, 6
\$MACRO, 5
\$MACRO_FILE, 6

ACTIVE_KEYMAP_PROFILE, 162
ALL
 CONSTANT, 23
APPEND_BLOCK, 82
ASCII, 49
assignment
 SOURCE FORM, 13
AT_EOF
 VARIABLE, 37
AT_EOL
 VARIABLE, 37
AUTO_PAIR_BRACKETS, 158
AUTOSAVE
 VARIABLE, 29

BACK_COLOR
 VARIABLE, 33
BACK_HOME, 70
BACK_SPACE
 CONSTANT, 14
BACK_SPACE, 66
BACK_WORD, 70
BACKUPS
 VARIABLE, 29
BAR_MENU, 50
BEEP, 107
BLACK
 CONSTANT, 14
BLOCK_BEGIN
 CONSTANT, 15
BLOCK_BEGIN, 82
BLOCK_COL1
 VARIABLE, 38
BLOCK_COL2
 VARIABLE, 38
BLOCK_END
 CONSTANT, 15
BLOCK_END, 82
BLOCK_LINE1
 VARIABLE, 38
BLOCK_LINE2
 VARIABLE, 38
BLOCK_MARK_LINES, 83
BLOCK_OFF
 CONSTANT, 15
BLOCK_OFF, 83
BLOCK_STAT
 VARIABLE, 38

BLOCK_STAT, 83
BLOCK_TEXT, 82
BLOCK_TOGGLE_MARKING, 83
BLOCK_TOGGLE_VISIBILITY, 83
BLUE
 CONSTANT, 14
BOTTOM_OF_WINDOW, 70
BRAIN, 107
breakpoint
 gutter double-click, 170
BROWN
 CONSTANT, 14
BUFFER_ID
 VARIABLE, 25

C_COL
 VARIABLE, 36
C_LINE
 VARIABLE, 36
C_PAGE
 VARIABLE, 37
C_ROW
 VARIABLE, 36
CALL, 60
CAPS, 50
CENTER_LINE, 70
CENTER_LINE_ON_SCREEN, 71
CHANGE_COLOR
 VARIABLE, 33
CHANGE_DIR, 166
CHAR, 50
CHECK_KEY, 51
CLEAR_SCREEN, 107
CLR_LINE, 108
CMD_TO_KEY, 111
CODECOLORS, 153
COL_BLOCK_BEGIN
 CONSTANT, 15
COL_BLOCK_BEGIN, 84
COMSPEC
 VARIABLE, 44
context menu
 debugger source, 170
COPY, 51
COPY_BLOCK
 CONSTANT, 16
COPY_BLOCK, 84
COPY_BLOCK_TO_CLIPBOARD, 84
COPY_FILE, 90
CPU
 VARIABLE, 36
CR

CONSTANT, 16
 CR, 66
 CREATE_GLOBAL_STR, 149
 CREATE_WINDOW, 93
 CTRL_HELP
 VARIABLE, 32
 CUR_CHAR
 VARIABLE, 47
 CUR_FILE_ATTR
 VARIABLE, 26
 CUR_FILE_SIZE
 VARIABLE, 27
 CUR_WINDOW
 VARIABLE, 40
 CUT_APPEND_BLOCK, 84
 CUT_BLOCK, 85
 CYAN
 CONSTANT, 14
 CYCLIC_VIRTUAL_DESKTOPS
 VARIABLE, 41

 DARKGRAY
 CONSTANT, 14
 DATE
 VARIABLE, 43
 DEBUGGERCOLORS, 154
 DEF_CHAR, 12
 DEF_HASH, 12
 DEF_INT, 11
 DEF_REAL, 12
 DEF_STR, 12
 DEF_TICK, 6
 DEFAULT_FORMAT
 VARIABLE, 46
 DEL_CHAR
 CONSTANT, 16
 DEL_CHAR, 66
 DEL_CHAR_OR_BLOCK, 85
 DEL_CHARS, 67
 DEL_EOL, 85
 DEL_FILE, 102
 DEL_LINE
 CONSTANT, 17
 DEL_LINE, 66
 DEL_WORD, 85
 DELAY, 149
 DELETE_BLOCK
 CONSTANT, 16
 DELETE_BLOCK, 85
 DELETE_WINDOW, 93
 DESKTOP_BLIT, 109
 DESKTOP_CLEAR, 109
 DESKTOP_MOVE_WINDOW_LEFT, 93
 DESKTOP_MOVE_WINDOW_RIGHT, 93
 DESKTOP_PUT_CHAR, 110
 DESKTOP_PUT_STRING, 110
 DESKTOP_SET_COLOR, 110
 DESKTOP_VIEWPORT_LEFT, 94
 DESKTOP_VIEWPORT_RIGHT, 94
 DO
 KEYWORD, 62

DOC_MODE
 VARIABLE, 27
 DOS_SHELL
 CONSTANT, 14
 DOWN
 CONSTANT, 17
 DOWN, 71
 DUMP
 HEADER KEYWORD, 7

 EDIT
 CONSTANT, 14
 ELSE
 KEYWORD, 61
 END
 KEYWORD, 61
 END_CLOSURE
 KEYWORD, 6
 END_MACRO
 KEYWORD, 6
 END_OF_BLOCK, 86
 EOF
 CONSTANT, 17
 EOF, 71
 EOL
 CONSTANT, 17
 EOL, 71
 ERASE_WINDOW, 94
 ERROR_COLOR
 VARIABLE, 34
 ERROR_LEVEL
 VARIABLE, 24
 EXEC_ASSIGN, 150
 EXEC_SESSION_LIST, 150
 EXEC_SESSION_STOP, 150
 EXISTS, 51
 EXIT_SAVE_ALL, 162
 EXPAND_TABS, 69
 EXPLOSIONS
 VARIABLE, 29
 EXTEND_BLOCK_BY_MOTION, 86

 FALSE
 CONSTANT, 13
 FILE_ATTR, 91
 FILE_CHANGED
 VARIABLE, 25
 FILE_EXISTS, 91
 FILE_NAME
 VARIABLE, 44
 FILECOMPARECOLORS, 154
 FILECOMPAREMINIMAPCOLORS, 154
 FIRST_FILE, 91
 FIRST_GLOBAL, 147
 FIRST_MACRO
 VARIABLE, 43
 FIRST_RUN
 VARIABLE, 24
 FIRST_SAVE
 VARIABLE, 25
 FIRST_WORD

- CONSTANT, 18
- FIRST_WORD, 71
- FLABEL, 111
- FORCE_SAVE, 94
- FORK, 166
- FORMAT_LINE
 - VARIABLE, 45
- FORMAT_RULER
 - VARIABLE, 31
- FORMAT_STAT
 - VARIABLE, 30
- FOUND_STR
 - VARIABLE, 45
- FOUND_X
 - VARIABLE, 46
- FOUND_Y
 - VARIABLE, 47
- FROM
 - HEADER KEYWORD, 7
- GET_ENVIRONMENT, 161
- GET_EXTENSION, 64
- GET_LINE
 - VARIABLE, 45
- GET_PATH, 65
- GET_RANDOM_MARK, 72
- GET_WORD, 65
- GLOBAL_HASH, 148
- GLOBAL_INT, 148
- GLOBAL_STR, 149
- GOTO, 60
- GOTO_COL, 80
- GOTO_LINE, 80
- GOTO_MARK
 - CONSTANT, 18
- GOTO_MARK, 72
- GOTOXY, 111
- GREEN
 - CONSTANT, 14
- HAS_VALUE, 52
- HELPCOLORS, 154
- HERO_MESSAGES_FILE_THRESHOLD_MB, 157
- HERO_MESSAGES_IN_LOGFILE, 157
- HERO_MESSAGES_ON_MESSAGE_LINE, 157
- HOME
 - CONSTANT, 18
- HOME, 72
- IF, 59
- IGNORE_CASE
 - VARIABLE, 24
- INDENT
 - CONSTANT, 18
- INDENT, 72
- INDENT_BLOCK, 73
- INDENT_LEVEL
 - VARIABLE, 39
- INDENT_STYLE
 - VARIABLE, 31
- indexing
 - SOURCE FORM, 13
- INQ_MACRO, 149
- INS_CURSOR
 - VARIABLE, 32
- INSERT_MODE
 - VARIABLE, 39
- INT_R, 52
- JUSTIFY_PARAGRAPH, 73
- KEY1
 - VARIABLE, 42
- KEY2
 - VARIABLE, 42
- KEY_IN, 111
- KEY_RECORD
 - CONSTANT, 19
- KEY_RECORD, 112
- KEYMAP_BIND, 155
- KEYMAP_PROFILE, 155
- KEYMAP_RESET, 155
- KEYMAP_VERSION, 156
- KEYS, 52
- KILL_BOX, 112
- label
 - SOURCE FORM, 62
- LAST_FILE_ATTR
 - VARIABLE, 26
- LAST_FILE_NAME
 - VARIABLE, 43
- LAST_FILE_SIZE
 - VARIABLE, 26
- LAST_FILE_TIME
 - VARIABLE, 26
- LAST_PAGE_BREAK
 - CONSTANT, 19
- LAST_PAGE_BREAK, 73
- LEFT
 - CONSTANT, 19
- LEFT, 73
- LEFT_MARGIN
 - VARIABLE, 31
- LEN, 52
- LENGTH, 53
- LIGHTBLUE
 - CONSTANT, 14
- LIGHTCYAN
 - CONSTANT, 14
- LIGHTGRAY
 - CONSTANT, 14
- LIGHTGREEN
 - CONSTANT, 14
- LIGHTMAGENTA
 - CONSTANT, 14
- LIGHTRED
 - CONSTANT, 14
- line-number gutter
 - breakpoints, 170
- LINK_STAT
 - VARIABLE, 40

- LINK_WINDOW, 95
- LIST_MATCHED_FILES, 95
- LOAD_BLOCK, 88
- LOAD_FILE, 102
- LOAD_MACRO_FILE, 150
- LOAD_WORKSPACE, 95
- LOGO_SCREEN
 - VARIABLE, 29
- MACRO_TO_KEY, 112
- MACRO_TOGGLE_RECORDING, 163
- MAGENTA
 - CONSTANT, 14
- MAKE_MESSAGE, 112
- MARK_POS
 - CONSTANT, 19
- MARK_POS, 73
- MARK_WORD_RIGHT, 86
- MARKING
 - VARIABLE, 39
- MARQUEE, 113
- MARQUEE_ERROR, 113
- MARQUEE_WARNING, 113
- MATCH_PARENTHESIS, 163
- MAX_WINDOW_ROW
 - VARIABLE, 35
- MEM_ALLOC
 - VARIABLE, 30
- MENU_COLOR
 - VARIABLE, 34
- MENUDIALOGCOLORS, 156
- MESSAGE_ROW
 - VARIABLE, 35
- MESSAGES
 - VARIABLE, 28
- MIN_WINDOW_ROW
 - VARIABLE, 35
- MINIMAPCOLORS, 156
- MMP_ACTION_BUTTON, 130
- MMP_ACTION_MENU, 130
- MMP_BOOL_FIELD, 130
- MMP_BOOL_SET, 131
- MMP_BOOL_VALUE, 131
- MMP_CANVAS, 131
- MMP_CANVAS_BOX, 132
- MMP_CANVAS_CLEAR, 132
- MMP_CANVAS_COMMIT, 132
- MMP_CANVAS_FILL, 133
- MMP_CANVAS_GLYPH, 133
- MMP_CANVAS_HOTSPOT, 134
- MMP_CANVAS_LINE, 134
- MMP_CANVAS_TEXT, 134
- MMP_INT_FIELD, 135
- MMP_INT_SET, 135
- MMP_INT_VALUE, 135
- MMP_LOG_APPEND, 136
- MMP_LOG_CLEAR, 136
- MMP_LOG_COUNT, 136
- MMP_LOG_FIELD, 137
- MMP_MENU_CLEAR, 137
- MMP_MENU_ITEM, 137
- MMP_PROGRESS_FIELD, 138
- MMP_PROGRESS_SET, 138
- MMP_PROGRESS_VALUE, 138
- MMP_SELECT_FIELD, 139
- MMP_SELECT_OPTION, 139
- MMP_SELECT_SET, 139
- MMP_SELECT_VALUE, 140
- MMP_STATUS_FIELD, 140
- MMP_STATUS_SET, 140
- MMP_STYLE_ACCENT, 141
- MMP_STYLE_MUTED, 141
- MMP_STYLE_SURFACE, 141
- MMP_STYLE_TEXT, 141
- MMP_TEXT_FIELD, 142
- MMP_TEXT_SET, 142
- MMP_TEXT_VALUE, 142
- MMP_TIMER_START, 143
- MMP_TIMER_STOP, 143
- MMP_WINDOW_EXISTS, 143
- MMP_WINDOW_HEIGHT, 143
- MMP_WINDOW_INSTANCE, 144
- MMP_WINDOW_WIDTH, 144
- MMP_WINDOW_X, 144
- MMP_WINDOW_Y, 145
- MODIFY_WINDOW, 95
- MOUSE
 - VARIABLE, 28
- MOUSE_H_SENSE
 - VARIABLE, 32
- MOUSE_V_SENSE
 - VARIABLE, 32
- MOVE_BLOCK
 - CONSTANT, 20
- MOVE_BLOCK, 86
- MOVE_VIEWPORT_LEFT, 74
- MOVE_VIEWPORT_RIGHT, 74
- MOVE_WIN_TO_NEXT_DESKTOP, 95
- MOVE_WIN_TO_PREV_DESKTOP, 96
- MPARM_STR
 - VARIABLE, 42
- MR_BLOCK_MARK_ALL, 155
- MR_EDIT_MARK_ALL, 155
- MR_PATH
 - VARIABLE, 45
- MRCOMPILERPROFILE, 156
- MRFEPROFILE, 157
- MRSETUP, 157
- MULTI_FILE_SEARCH, 96
- MULTI_FILE_SEARCH_REPLACE, 96
- NAME_LINE
 - VARIABLE, 35
- NEW_SCREEN, 96
- NEXT_FILE, 91
- NEXT_GLOBAL, 148
- NEXT_MACRO
 - VARIABLE, 43
- NEXT_PAGE_BREAK
 - CONSTANT, 20
- NEXT_PAGE_BREAK, 74
- NEXT_SEARCH_RESULT, 97

- OS_BACK, 161
- OS_COLOR, 161
- OS_VERSION, 161
- OTHERCOLORS, 158
- OVR_CURSOR
 - VARIABLE, 32
- PAGE_DOWN
 - CONSTANT, 20
- PAGE_DOWN, 74
- PAGE_STR
 - VARIABLE, 46
- PAGE_UP
 - CONSTANT, 20
- PAGE_UP, 75
- PARAM_COUNT
 - VARIABLE, 36
- PARAM_STR, 61
- PARSE_INT, 53
- PARSE_STR, 53
- PASS_KEY, 113
- PASTE_BLOCK, 87
- PASTE_FROM_CLIPBOARD, 87
- PERM
 - HEADER KEYWORD, 7
- PG_LINE
 - VARIABLE, 37
- PLAY_KEY_MACRO, 114
- POP_LABELS, 114
- POP_MARK, 75
- POS, 54
- PRINT_MARGIN
 - VARIABLE, 27
- PUSH_KEY, 114
- PUSH_LABELS, 114
- PUT_BOX, 115
- PUT_COL_NUM, 115
- PUT_LINE, 67
- PUT_LINE_NUM, 115
- QUIT, 165
- READ_KEY, 116
- READ_ONLY
 - VARIABLE, 27
- REAL_I, 54
- RED
 - CONSTANT, 14
- REDO, 87
- REDRAW, 97
- REFORMAT_DOCUMENT, 75
- REFORMAT_PARAGRAPH, 75
- REFRESH
 - VARIABLE, 28
- REG_EXP_STAT
 - VARIABLE, 25
- REGISTER_MENU_ITEM, 116
- REMOVE_MENU_ITEM, 116
- REMOVE_SPACE, 54
- RENAME_FILE, 92
- REPEAT_SEARCH, 97
- REPLACE, 102
- REST_OS_SCREEN, 165
- RET, 60
- RETURN_INT
 - VARIABLE, 24
- RETURN_STR
 - VARIABLE, 42
- REVERSE_TAB, 75
- REVERT_FILE, 97
- RIGHT
 - CONSTANT, 21
- RIGHT, 76
- RIGHT_MARGIN
 - VARIABLE, 31
- RSTR, 54
- RUN_MACRO, 151
- RVAL, 49
- SAVE_ALL, 97
- SAVE_BLOCK, 88
- SAVE_FILE
 - CONSTANT, 21
- SAVE_FILE, 102
- SAVE_OS_SCREEN, 166
- SAVE_SETTINGS, 158
- SAVE_WORKSPACE, 98
- SCREEN_LENGTH, 55
- SCREEN_WIDTH, 55
- SCROLL_BOX_DN, 116
- SCROLL_BOX_UP, 117
- SCROLL_DOWN, 76
- SCROLL_UP, 76
- SEARCH, 98
- SEARCH_BWD, 92
- SEARCH_FILE
 - VARIABLE, 45
- SEARCH_FWD, 92
- SEARCH_REPLACE, 98
- SET_CLIPBOARD_TEXT, 117
- SET_FILE_ATTR, 103
- SET_GLOBAL_HASH, 151
- SET_GLOBAL_INT, 151
- SET_GLOBAL_STR, 152
- SET_INDENT_LEVEL, 76
- SET_LEFT_MARGIN, 77
- SET_RANDOM_MARK, 77
- SET_RIGHT_MARGIN, 77
- SETUP_BACKUPS_AUTOSAVE, 163
- SETUP_COLOR, 163
- SETUP_COMPILER_PROFILES, 164
- SETUP_EDIT_SETTINGS, 164
- SETUP_FILENAME_EXTENSIONS, 164
- SETUP_KEYMAP, 164
- SETUP_LIVE_LOGS, 164
- SETUP_PATHS, 165
- SETUP_USER_INTERFACE, 165
- SHADOW_CHAR
 - VARIABLE, 28
- SHADOW_COLOR
 - VARIABLE, 34
- SHELL_TO_OS, 167

- SIZE_WINDOW, 103
- SORT_COLUMN_BLOCK_TOGGLE, 87
- START_OF_BLOCK, 87
- STAT_COLOR
 - VARIABLE, 34
- STATUS_ROW
 - VARIABLE, 34
- STR, 55
- STR_BLOCK_BEGIN
 - CONSTANT, 21
- STR_BLOCK_BEGIN, 88
- STR_DEL, 56
- STR_INS, 56
- STRING_IN, 56
- SUBSHELL, 162
- SWITCH_FILE, 93
- SWITCH_WINDOW, 98
- TAB_EXPAND
 - VARIABLE, 39
- TAB_LEFT
 - CONSTANT, 21
- TAB_LEFT, 77
- TAB_RIGHT
 - CONSTANT, 22
- TAB_RIGHT, 77
- TABS_TO_SPACES, 69
- TEMP_PATH
 - VARIABLE, 44
- TEXT, 67
- TEXT_COLOR
 - VARIABLE, 33
- THEME_NAME, 158
- THEME_RESET, 159
- THEME_VERSION, 159
- THEN
 - KEYWORD, 61
- TIME
 - VARIABLE, 43
- TMP_FILE
 - VARIABLE, 25
- TMP_FILE_NAME
 - VARIABLE, 44
- TO
 - HEADER KEYWORD, 7
- TOF
 - CONSTANT, 22
- TOF, 78
- TOGGLE_FORMAT_RULER, 78
- TOGGLE_WORD_WRAP, 78
- TOP_OF_WINDOW, 78
- TRANS
 - HEADER KEYWORD, 7
- TRUE
 - CONSTANT, 13
- TRUNCATE_EXTENSION, 65
- TRUNCATE_PATH, 65
- TRUNCATE_SPACES
 - VARIABLE, 29
- UI_BUTTON, 119
- UI_DIALOG, 120
- UI_DISPLAY, 120
- UI_EXEC, 120
- UI_GRID, 121
- UI_INDEX, 121
- UI_INPUT, 121
- UI_LABEL, 122
- UI_LIST_ADD, 122
- UI_LIST_CLEAR, 123
- UI_LISTBOX, 122
- UI_MESSAGEBOX, 123
- UI_MODELESS_CLOSE, 128
- UI_MODELESS_DISPLAY, 128
- UI_MODELESS_ON, 129
- UI_MODELESS_SHOW, 129
- UI_MODELESS_UPDATE, 129
- UI_TABLE, 123
- UI_TABLE_CLEAR, 124
- UI_TABLE_COLUMN, 124
- UI_TABLE_ROW, 124
- UI_TEXT, 124
- UI_TREE, 125
- UI_TREE_CLEAR, 125
- UI_TREE_NODE, 125
- UNASSIGN_ALL_KEYS, 117
- UNASSIGN_KEY, 118
- UNDENT
 - CONSTANT, 22
- UNDENT, 79
- UNDENT_BLOCK, 79
- UNDO
 - CONSTANT, 22
- UNDO, 88
- UNDO_STAT
 - VARIABLE, 30
- UNLINK_WINDOW, 99
- UNLOAD_MACRO, 152
- UP
 - CONSTANT, 23
- UP, 79
- UTF8, 57
- V_MENU, 57
- VAL, 49
- VALUES, 57
- VERSION, 162
- VIRTUAL_DESKTOPS
 - VARIABLE, 41
- WHEREX, 58
- WHEREY, 58
- WHILE, 60
- WHITE
 - CONSTANT, 14
- WIN_X1
 - VARIABLE, 40
- WIN_X2
 - VARIABLE, 41
- WIN_Y1
 - VARIABLE, 40
- WIN_Y2

- VARIABLE, 41
- WINDOW_ATTR
 - VARIABLE, 33
- WINDOW_CASCADE, 99
- WINDOW_CLOSE, 99
- WINDOW_COPY, 103
- WINDOW_COPY_BLOCK, 99
- WINDOW_COUNT
 - VARIABLE, 41
- WINDOW_LIST, 99
- WINDOW_MOVE, 104
- WINDOW_MOVE_BLOCK, 100
- WINDOW_NEXT, 100
- WINDOW_OPEN, 100
- WINDOW_PREVIOUS, 100
- WINDOW_SPLIT_HORIZONTAL, 101
- WINDOW_SPLIT_VERTICAL, 101
- WINDOW_TILE, 101
- WINDOW_ZOOM, 101
- WINDOWCOLORS, 159
- WORD_DELIMITS
 - VARIABLE, 46
- WORD_LEFT
 - CONSTANT, 23
- WORD_LEFT, 79
- WORD_RIGHT
 - CONSTANT, 23
- WORD_RIGHT, 79
- WORD_WRAP_LINE, 80
- WORKING, 118
- WRAP_STAT
 - VARIABLE, 30
- WRITE, 118
- WRITE_SOD, 167

- XPOS, 58

- YELLOW
 - CONSTANT, 14

- ZOOM, 101